Topic 2

Bitcoin

Bitcoin System

Model		Application
Distributed System		Bitcoin System
Consensus		Global Ledger

Bitcoin System

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

Published online in 2008

The first coin implemented in 2009

First *decentralized* digital currency

Bitcoin System

Satoshi Nakamoto

Paper: **"Bitcoin: A Peer-to-Peer Electronic Cash System"** (2008)

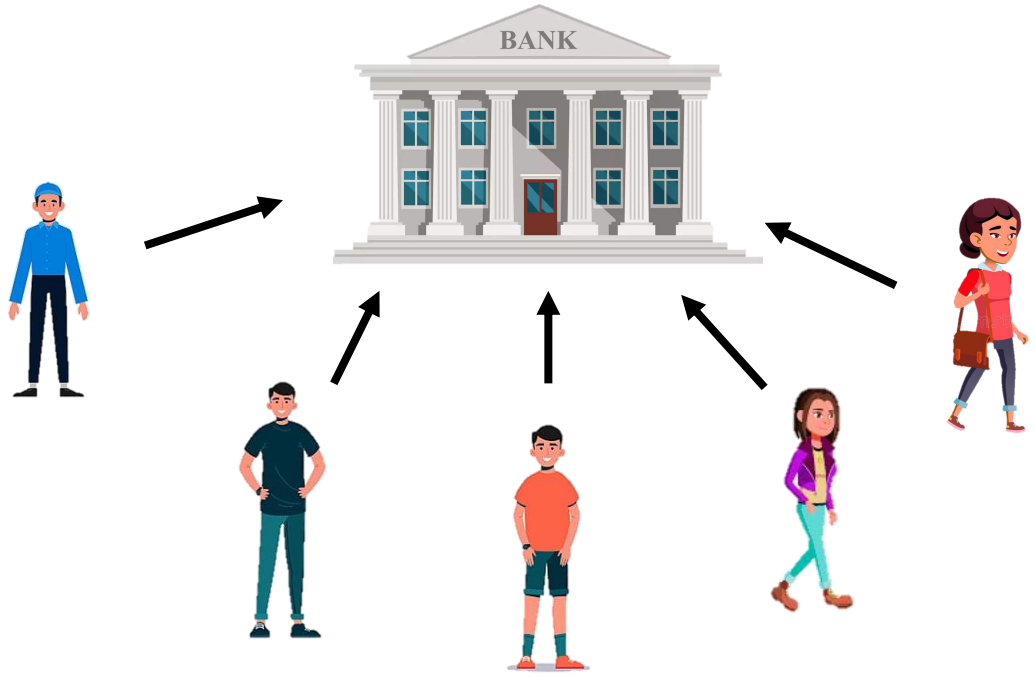
" (Bitcoin system) is an electronic payment system based on cryptographic proof instead of trust, allowing any two willing parties to transact directly with each other without the need for a trusted third party."

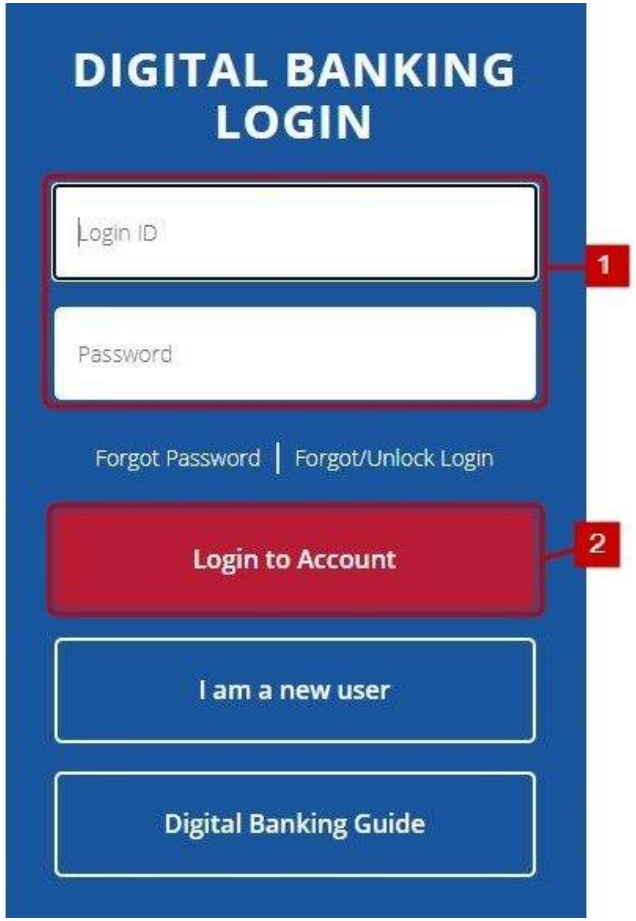
Before everything, you need a …

Wallet / Bank Account

Conventional Banking/E-Cash System

- Account in the bank
- Digital account and password in e-bank





Bitcoin System

A direct payment system must possess the following properties:

1. Each user is assigned a unique identifier, which they can use to transfer value to the intended recipient.
2. Only the transaction sender has the access to create the related transaction, preventing other users from impersonating and creating identical transactions.

Bitcoin System

A decentralized payment system must possess the following properties:

1. Each user is assigned a unique identifier, which they can use to transfer value to the intended recipient.
2. Only the transaction sender has the access to create the related transaction, preventing other users from impersonating and creating identical transactions.
3. Members within this system can easily verify whether a transaction has been created by the designated sender and not by any other user.

Bitcoin System

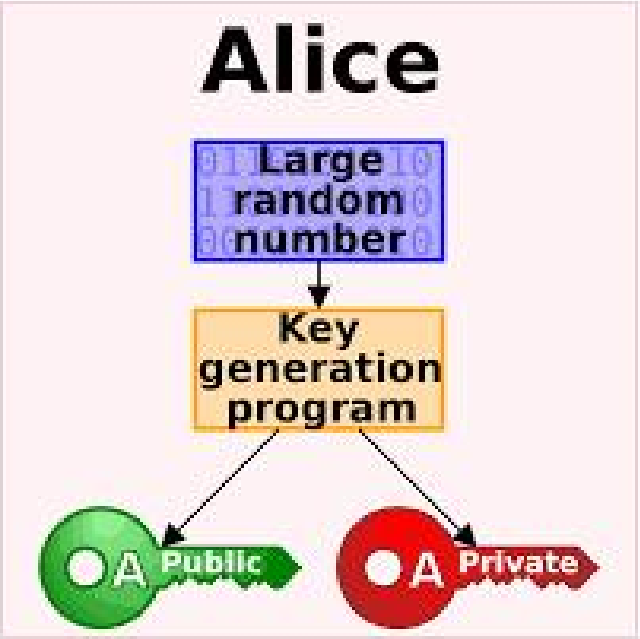
A decentralized payment system must possess the following properties:

1. Each user is assigned a unique identifier, which they can use to transfer value to the intended recipient.
2. Only the transaction sender has the access to create the related transaction, preventing other users from impersonating and creating identical transactions.
3. Members within this system can easily verify whether a transaction has been created by the designated sender and not by any other user.

One of the solutions is to use **asymmetric cryptography (asymmetric key pair).**

Bitcoin System

Asymmetric cryptography



In this method, there exists a pair of keys, i.e., a **public key** and a **private key**, and a pair of functions, an **encryption function** and a **decryption function**. The private key is always kept locally and should not be shared, while the public key can be provided to anyone. When encrypting a message using one key, it is necessary to use the other key to decrypt it.

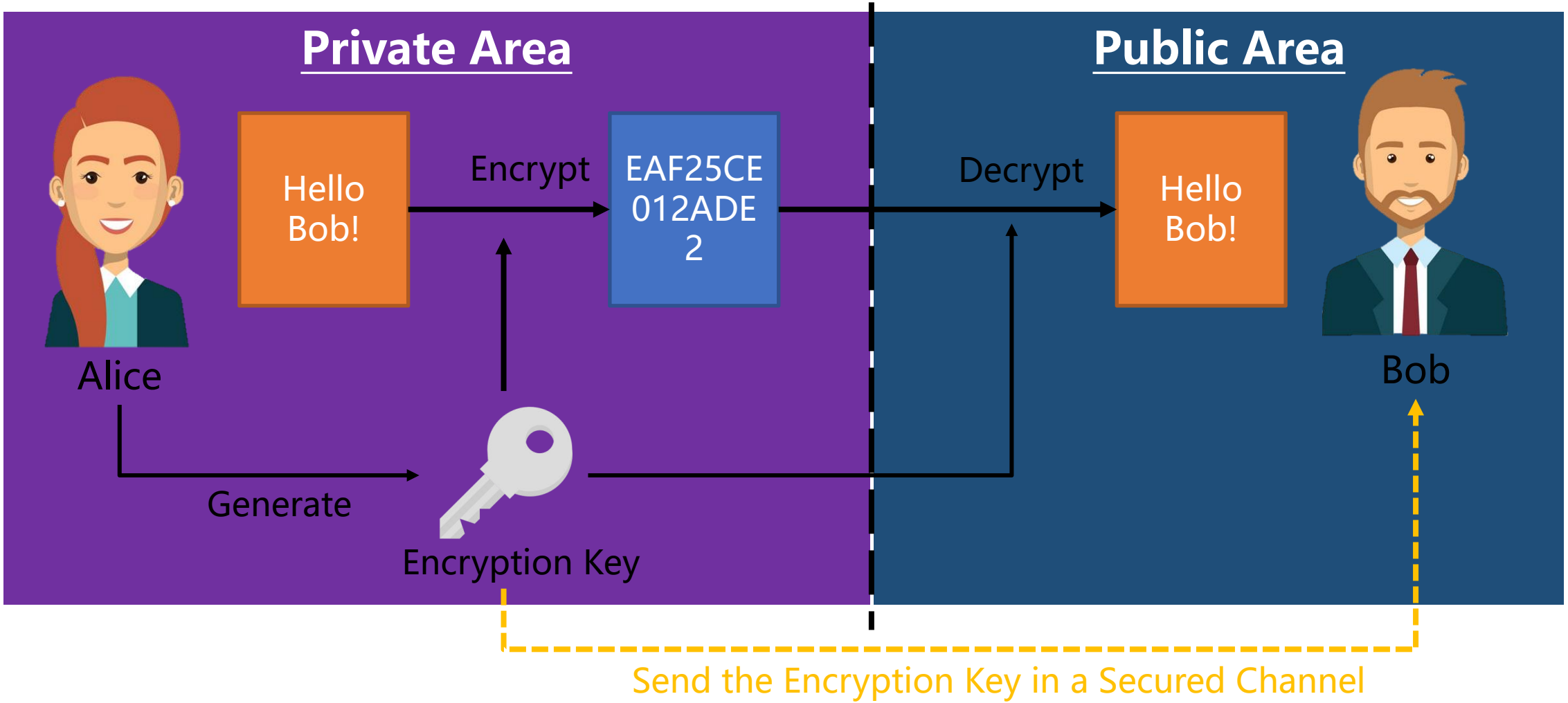
Encryption:

Encrypted_Message = E(Message, private_key)

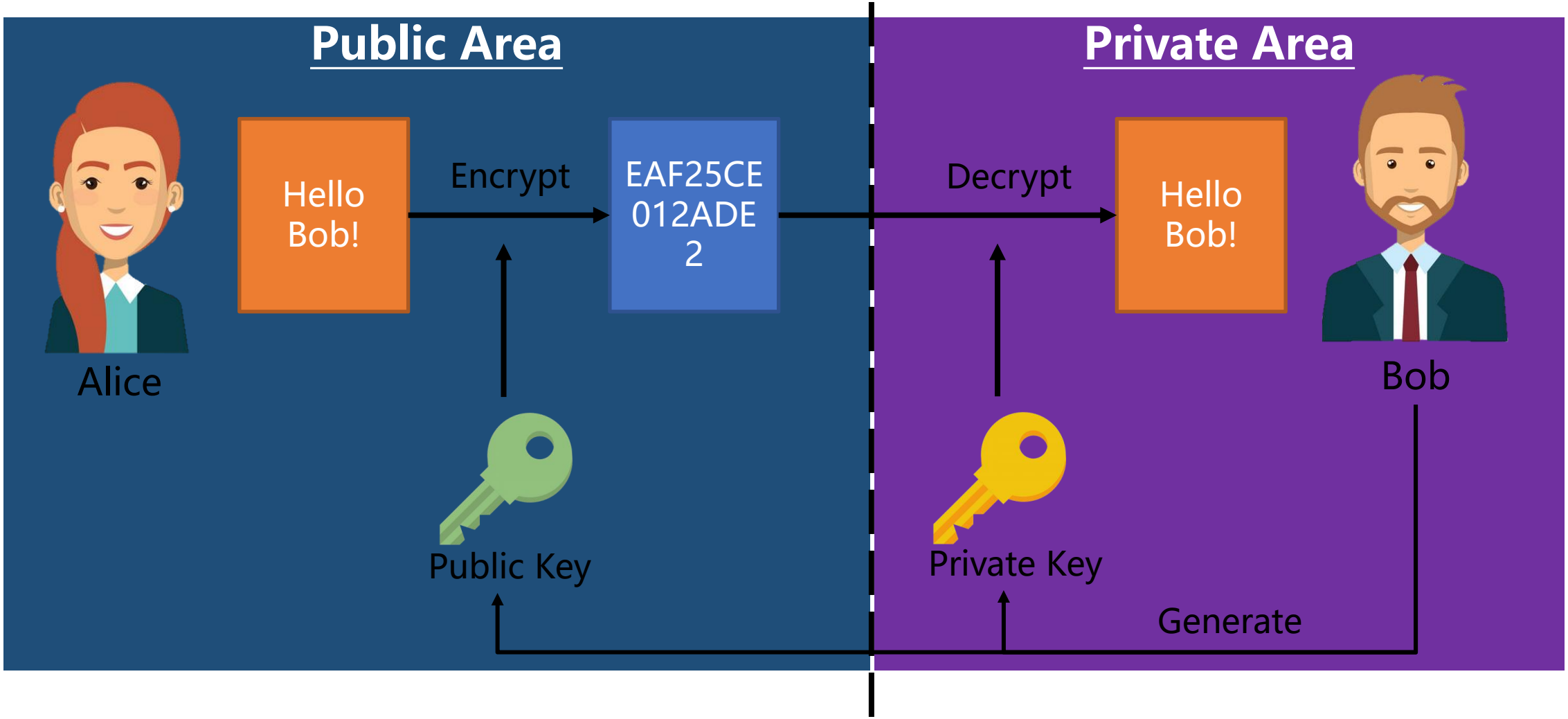
Decryption:

Message = D(Encrypted_Message, public_key)

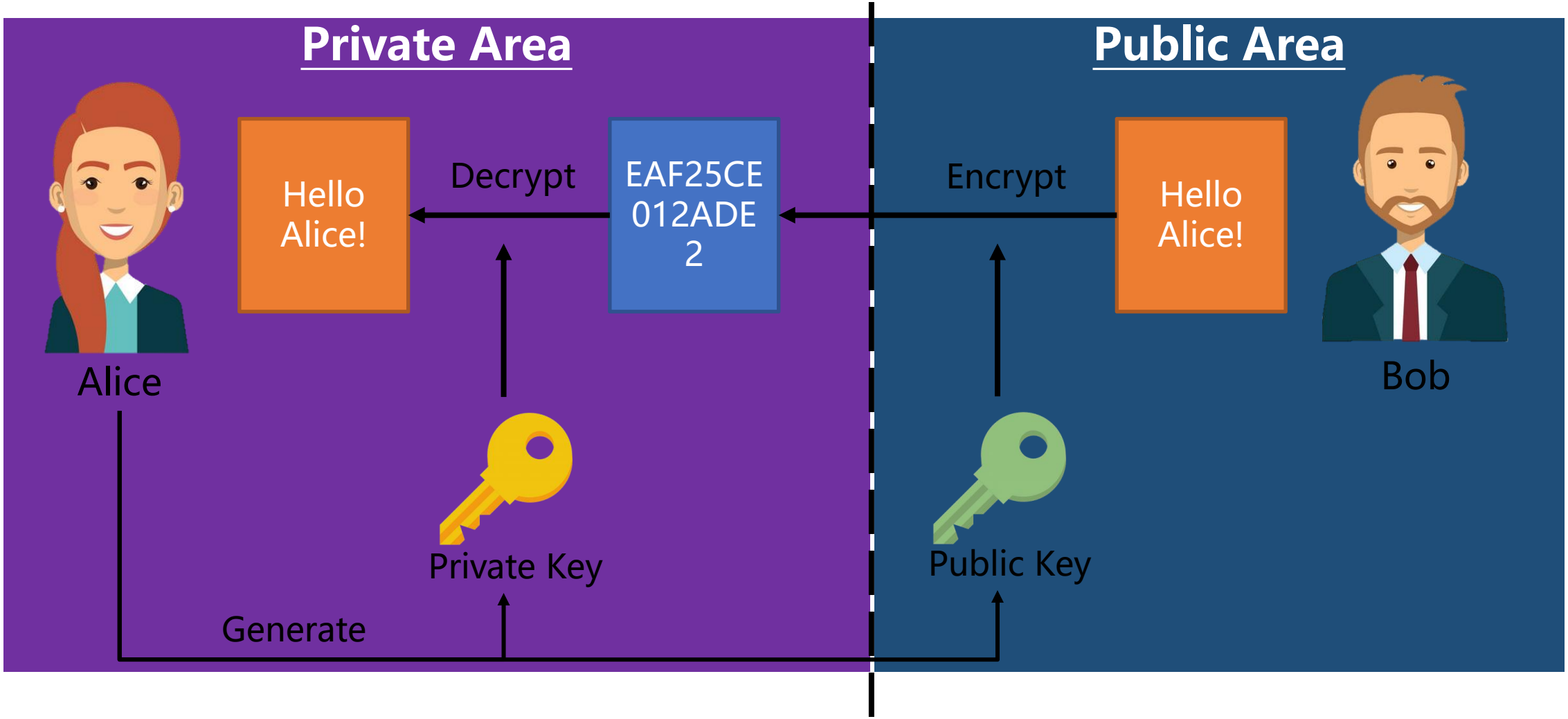
Symmetric Cryptography



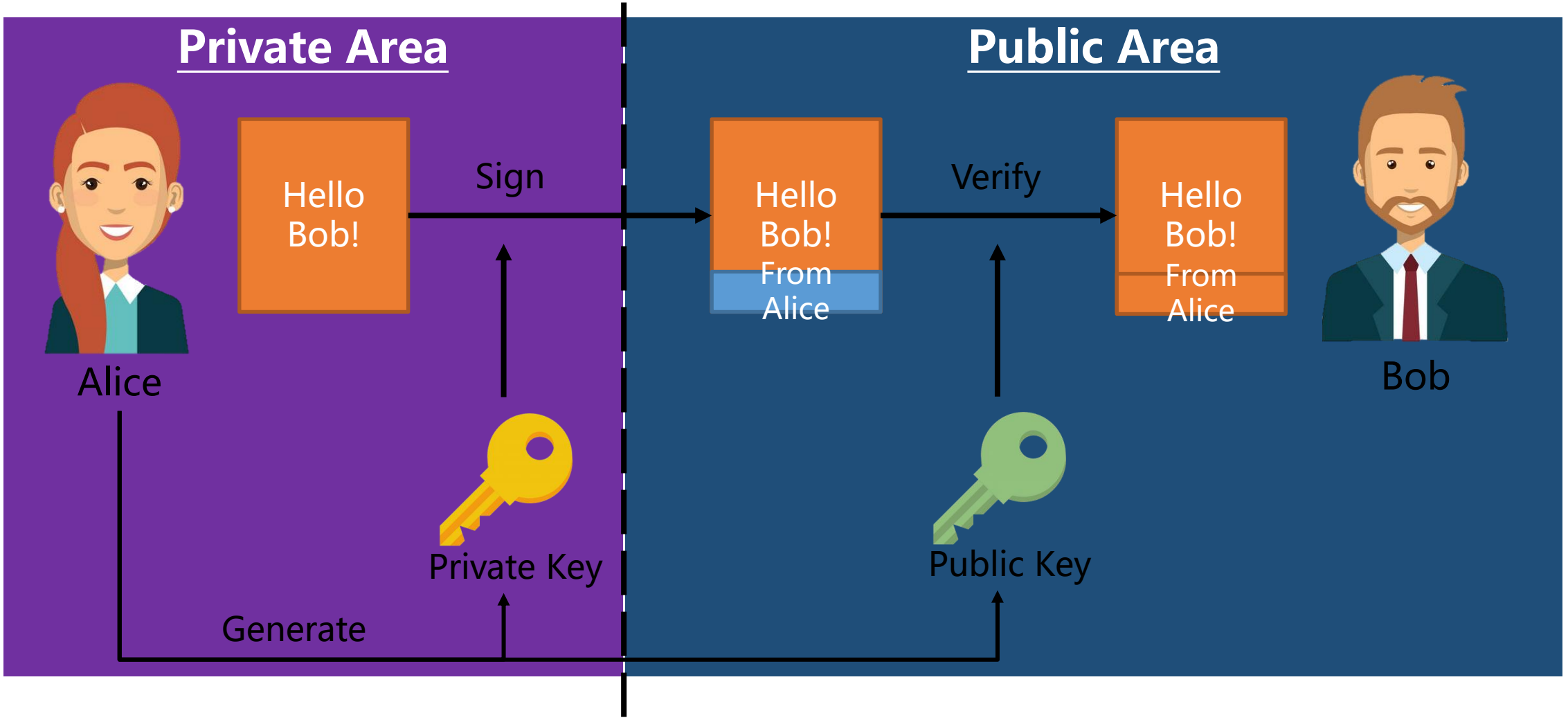
Asymmetric Cryptography: Encryption



Asymmetric Cryptography: Encryption 2



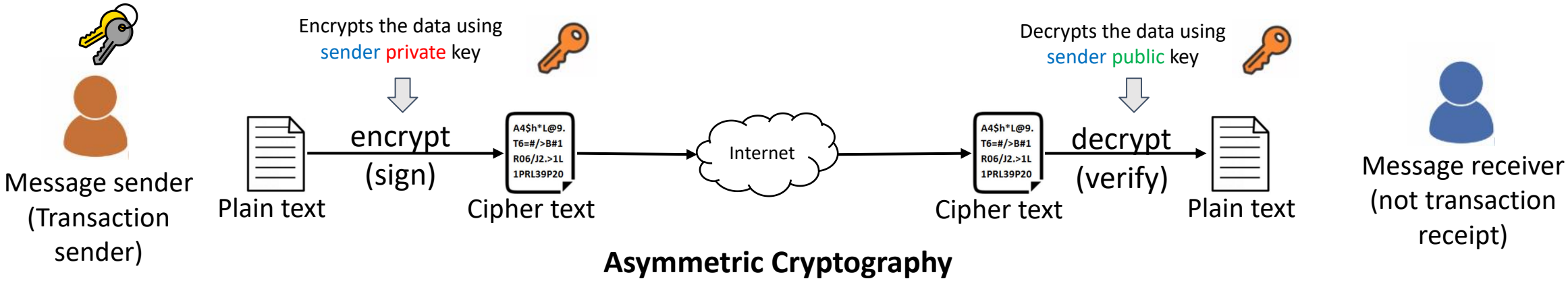
Asymmetric Cryptography: Signature



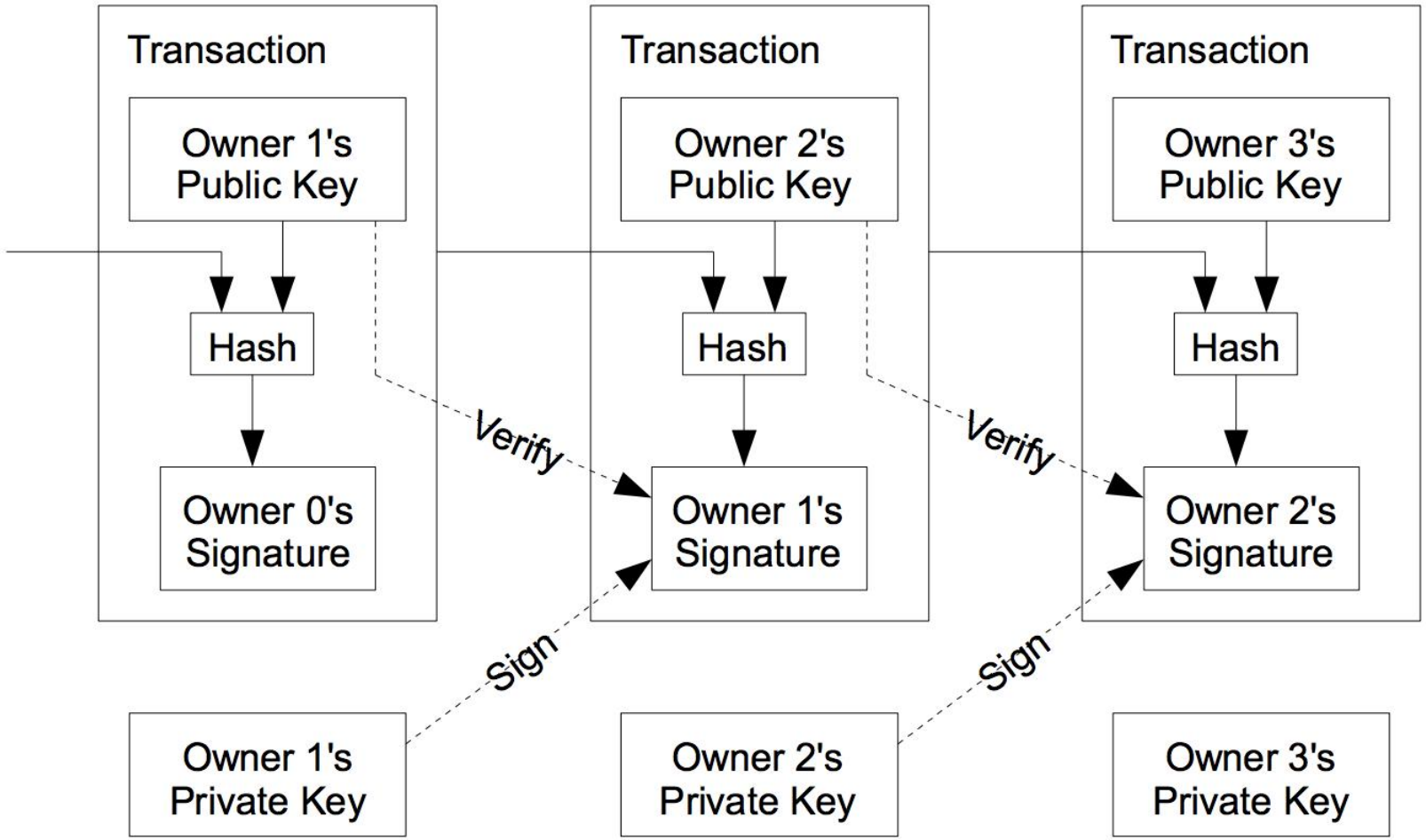
Bitcoin System

Transaction

- In order to transfer value, sender signs a message (transaction) using his **private key**.
  This transaction is then **broadcast** to **the rest nodes in Bitcoin network** where nodes verify that sender’s private key is able to access the inputs (by checking that sender’s encrypted message can decrypted by the **public key (address)** he is claiming to own).



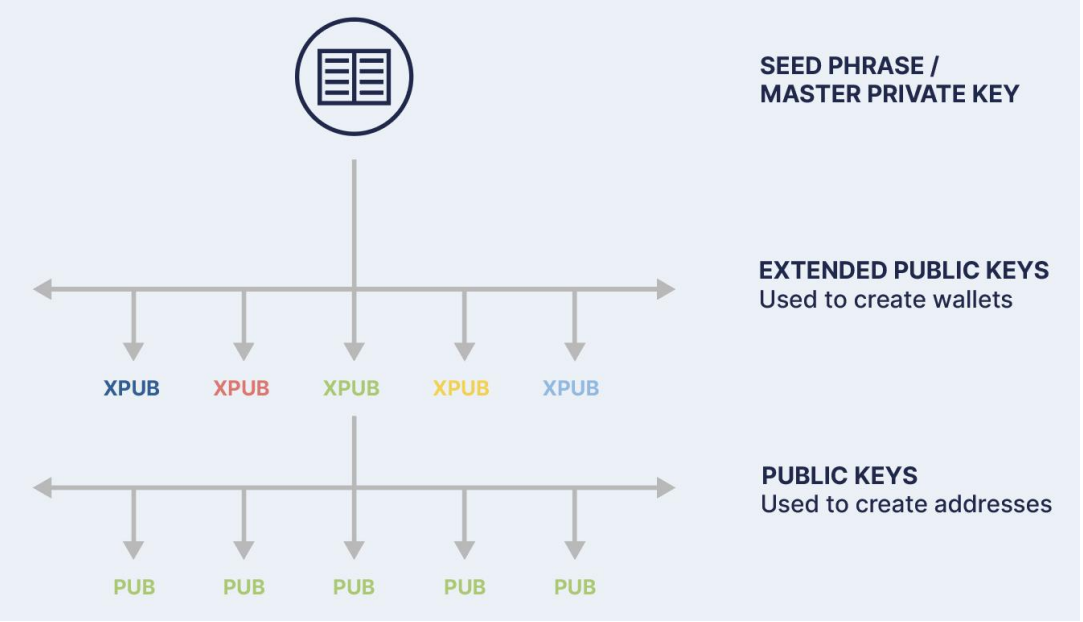
Signing Bitcoin Transactions



Bitcoin System

Address

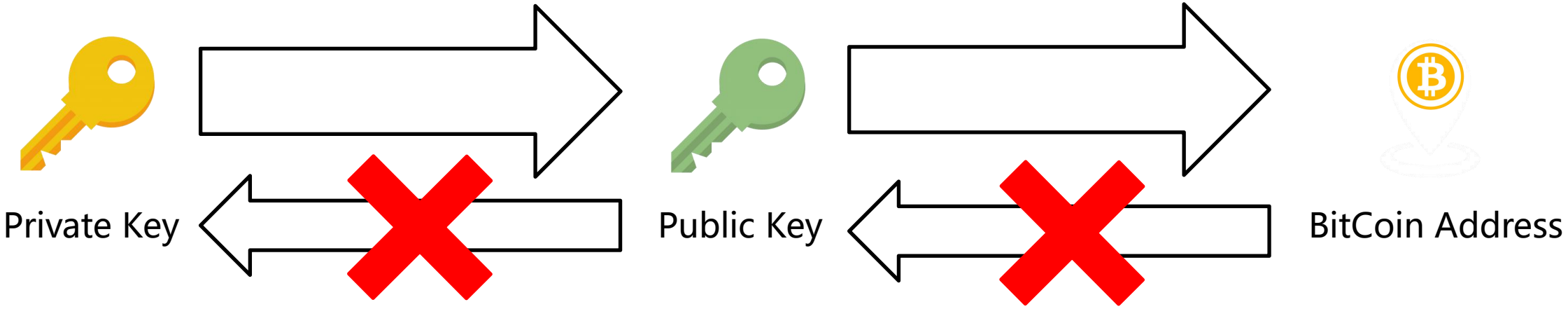
- A bitcoin address is a **unique identifier** that represents a virtual location that cryptocurrencies can be used to send or receive bitcoin.
- Every address is the public key of an **asymmetric key pair**.
- Bitcoin addresses typically consist of a string of 26 to 35 alphanumeric characters.
- One private key can generate multi public keys.
- Examples:
  - 1Lbcfr7sAHTD9CgdQo3HTMTkV8LK4ZnX71
  - 3E13MQrZvPHqSSTsdQaZzZiYPzjEDT5VKE



Reference:
https://unchained.com/blog/what-is-a-bitcoin-address/

Bitcoin System

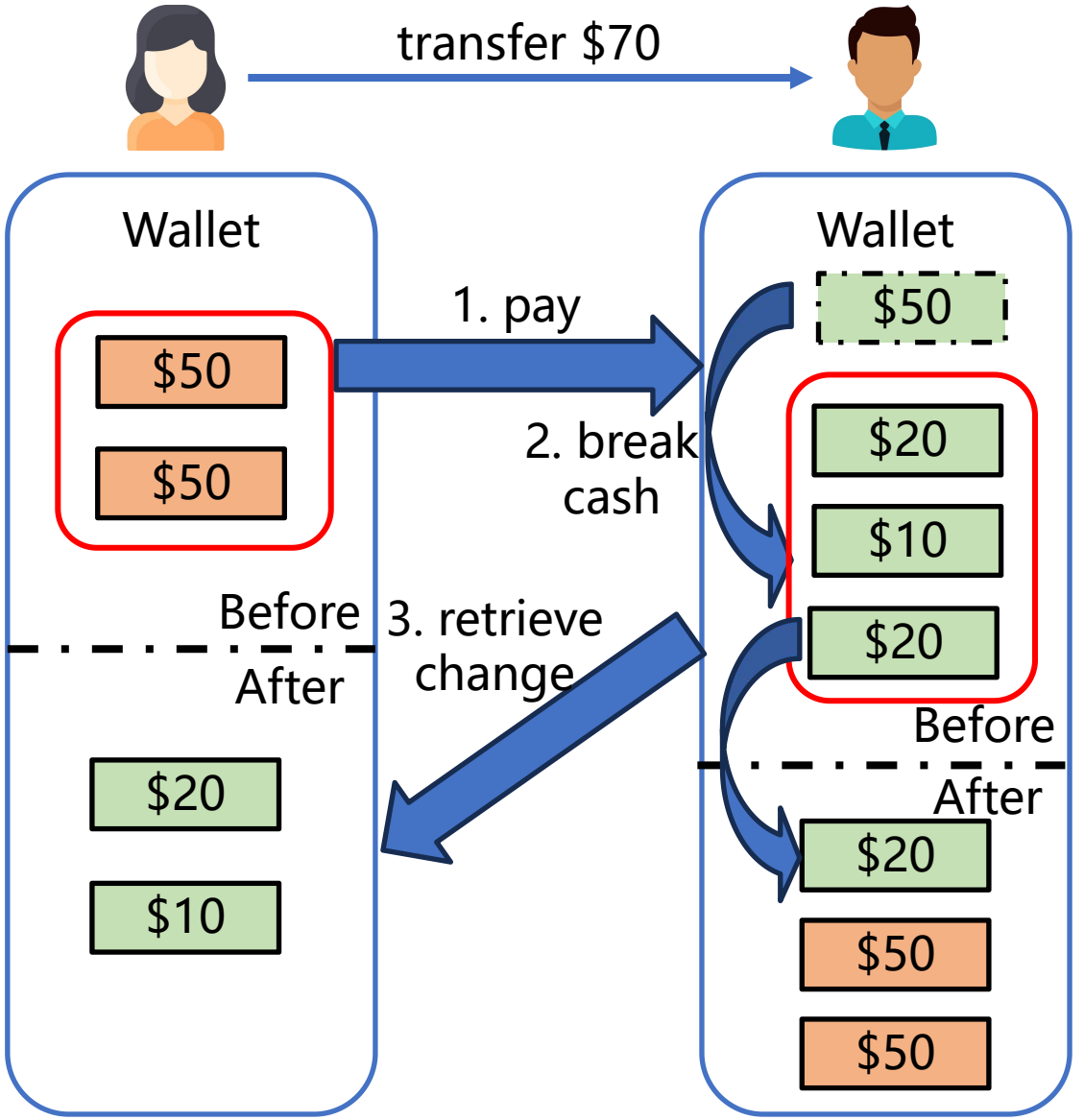
Address



Bitcoin System

Cash Transaction System

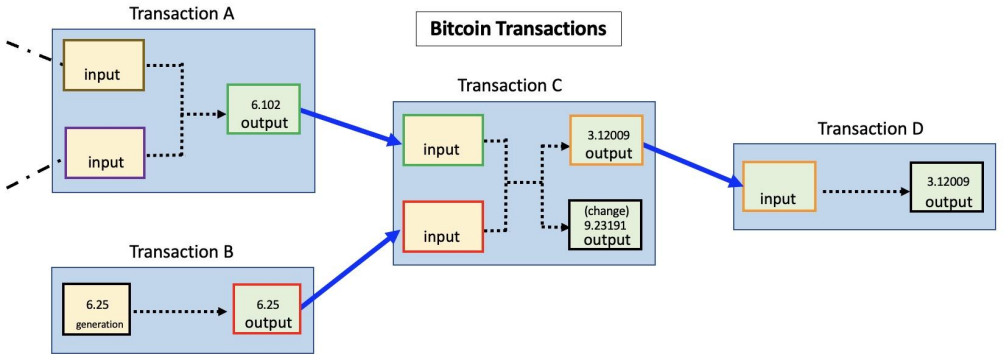
- The cash transaction system was well-known in our daily lives before using the electronic payment system.
- In this system, each person carry a wallet containing several cash notes of various denominations, which have been obtained from previous transactions.

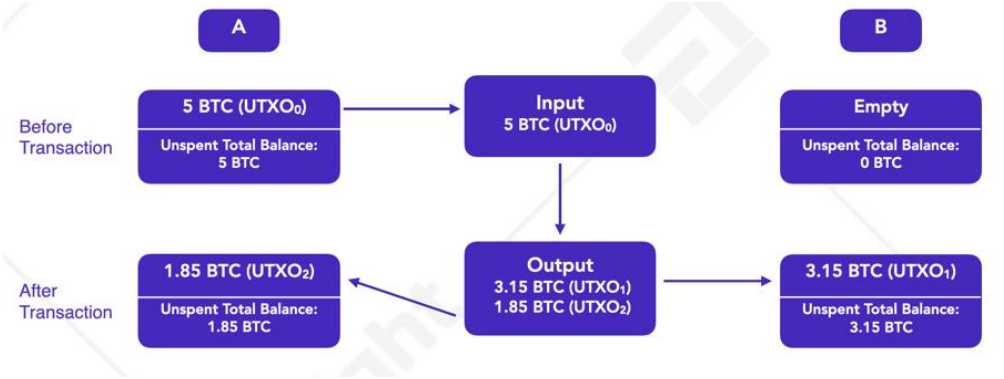


Bitcoin System

Unspent Transaction Output (UTXO) Model

- An Unspent Transaction Output (UTXO) is an unused token.
- Every time a transaction is executed, the input UTXOs are deleted and the output UTXOs are generated. Any output of the transaction as a new UTXO that can be later spent in a new transaction.
- The main differences between the cash and the UTXO model is that bitcoin isn’ t bound by set amounts, i.e. $5, $10, $20, etc. Any amount of bitcoin (up to eight decimal places) can be an UTXO.
- Double Spent is impossible in this model as every UTXO will have the state whether it is spent already in the same global ledger.



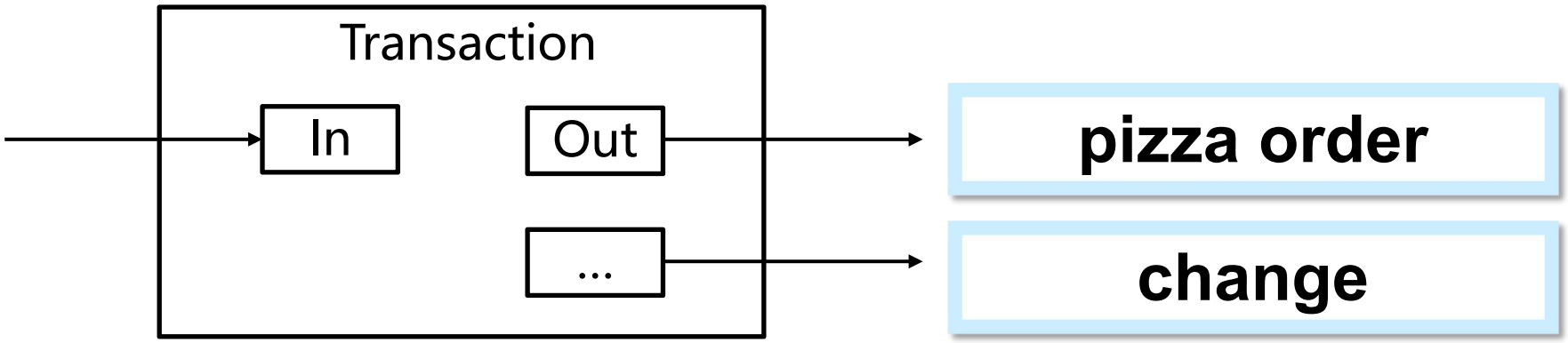


Why UTXO Model?

no need to check balance before spend

Use UTXO in a Transaction

UTXO is indivisible: it must be consumed in its entirety



Transaction View information about a bitcoin transaction

cc455ae816e6cdafdb58d54e35d4f46d860047458eacf1c7405dc634631c570d

1QgTYzMYqStzZBQx8gguYaJQMjFRbagbh	→	1DsB8CHc7fa87GUm6qs6YGoCRgQpBo2TJZ	0.0001 BTC

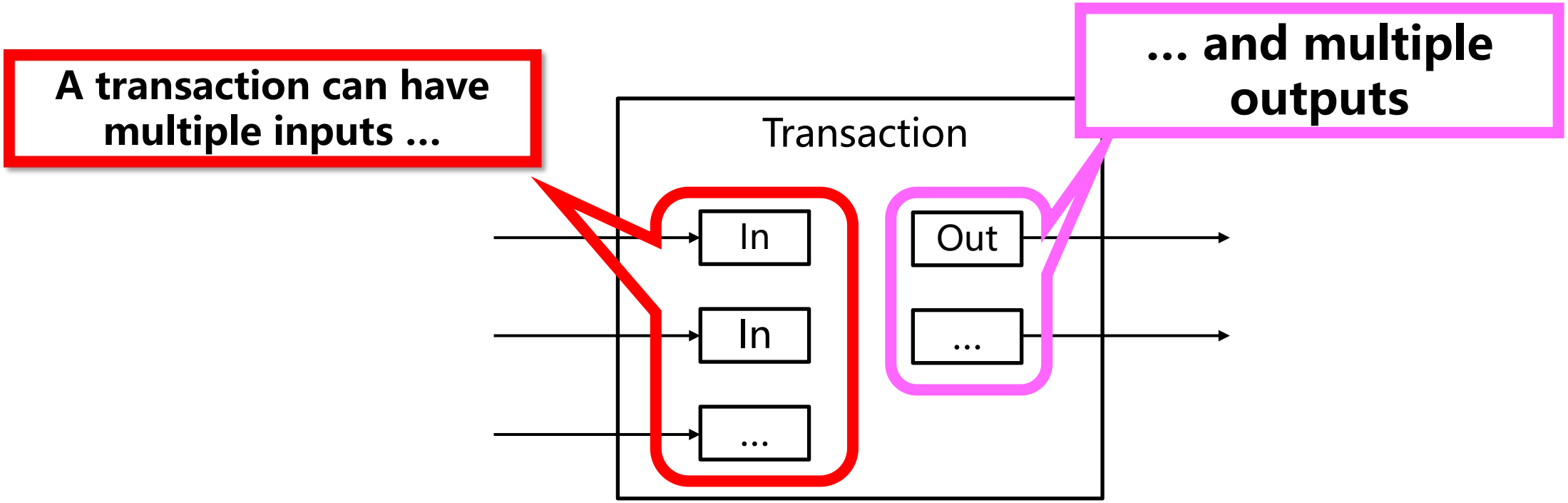
SPONSORED
Crypto Credit
0.0001 BTC

Summary	
Size	1963 (bytes)
Weight	7852
Received Time	2016-04-26 14:13:36
Included In Blocks	409008 (2016-04-26 14:15:22 + 2 minutes)
Confirmations	185027
Visualize	View Tree Chart

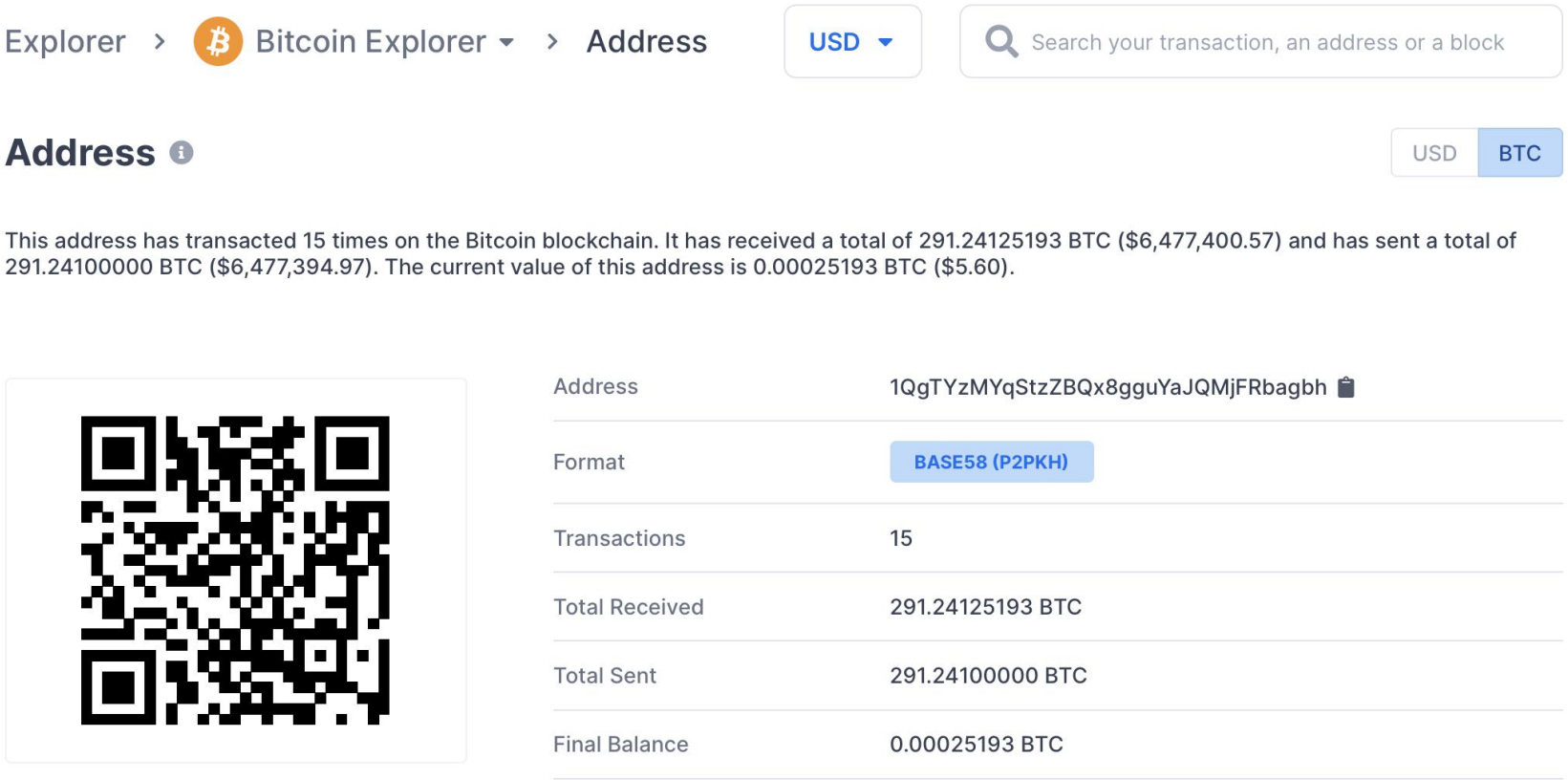
Inputs and Outputs	
Total Input	291.241 BTC
Total Output	0.0001 BTC
Fees	291.2409 BTC
Fee per byte	14,836,520.632 sat/B
Fee per weight unit	3,709,130.158 sat/WU
Estimated BTC Transacted	0.0001 BTC
Scripts	Show scripts & coinbase

Oooops:
https://blockchain.info/tx/cc455ae816e6cdafdb58d54e35d4f46d860047458eacf1c7405dc634631c570d

Multiple Inputs/Outputs in a Transaction



Balance of An Address



the balance of an address is calculated as the sum of UTXOs

Bitcoin System

Transaction Format

- A transaction is a transfer message record of bitcoin value on the bitcoin system, from sender address to receiver address.
- At a high level, a transaction has 2 main parts:
  - **Input:**

    (1) Previous UTXO’ s Transaction ID and index in this transaction (previous output)

    (2) input script (action)
  - **Output:**

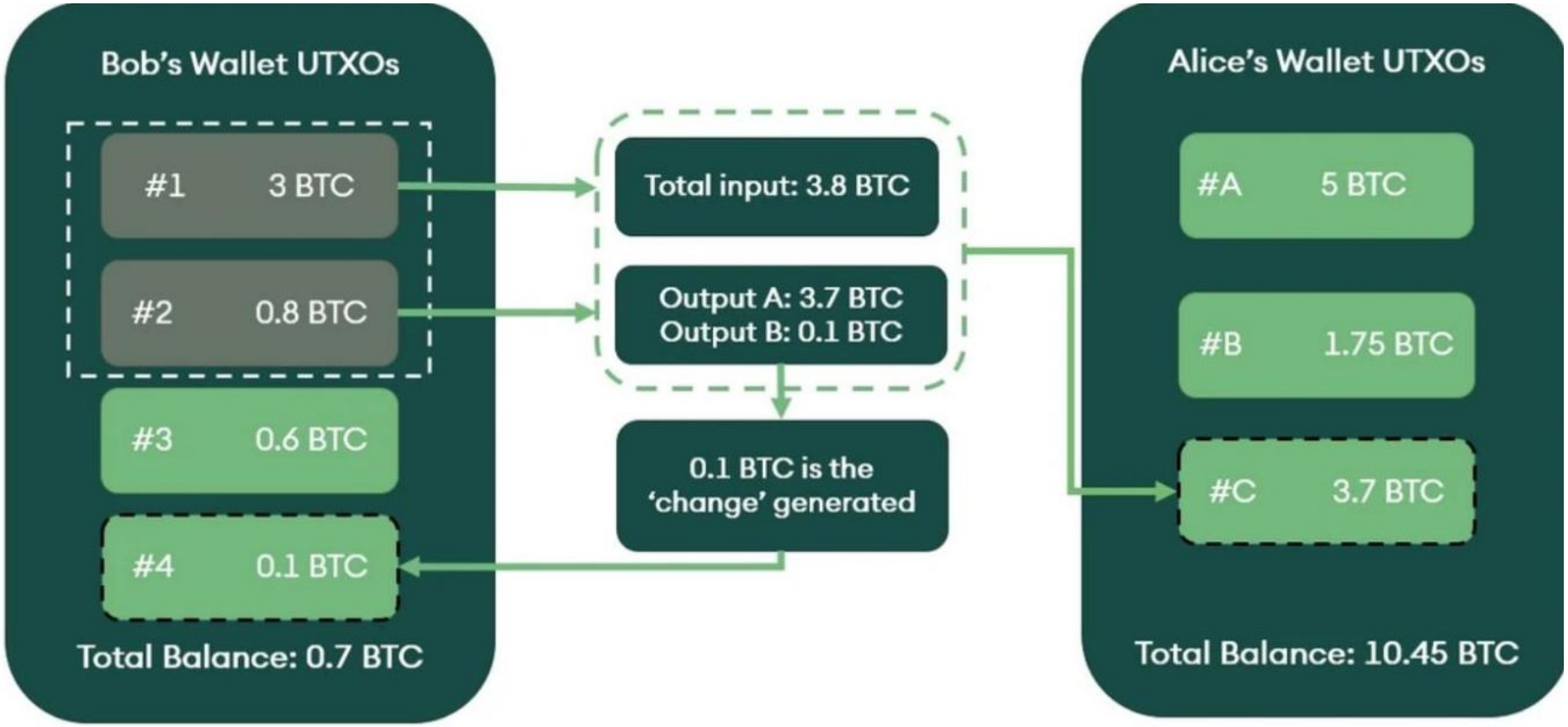
    (1) new UTXO’ s amount

    (2) Receipt’ s address

Bitcoin System

Example of transaction and UTXO model:

- Bob’ s address transfers 3.7 BTC to Alice’ s address (ignore transaction fees).



Bitcoin System

Real Transaction Example:

Tx Hash(ID) = 0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2

address: 1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK → address 1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA

transfer amount = 0.015 BTC

Transaction View information about a bitcoin transaction

0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2

1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK (0.1 BTC - Output)

1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA - (Unspent) 0.015 BTC

1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK - (Unspent) 0.0845 BTC

97 Confirmations | 0.0995 BTC

Summary	
Size	258 (bytes)
Received Time	2013-12-27 23:03:05
Included In Blocks	277316 (2013-12-27 23:11:54 +9 minutes)

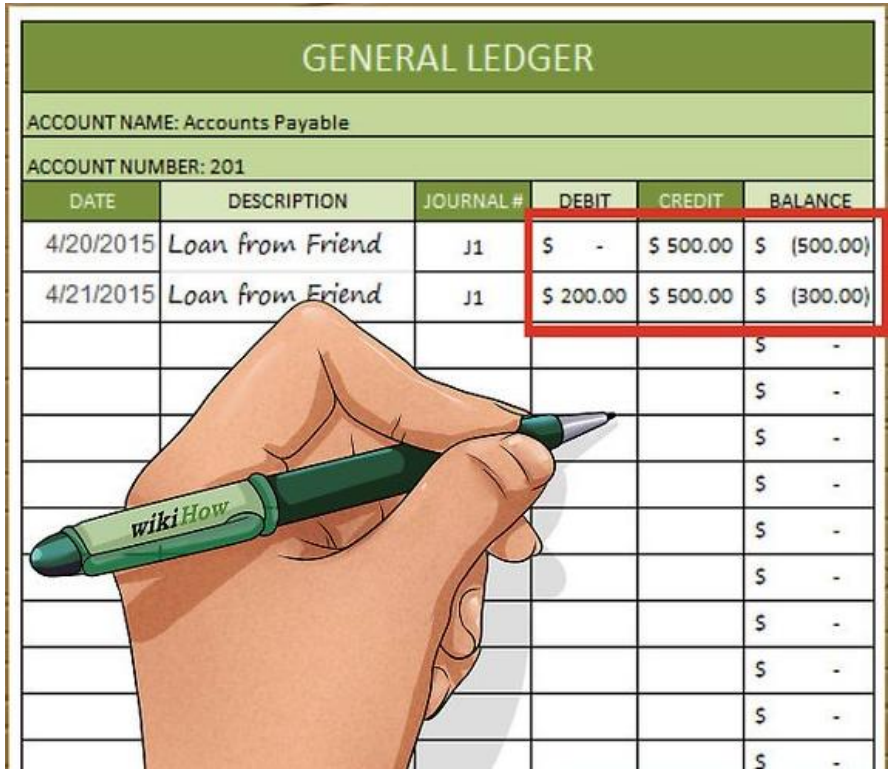
Inputs and Outputs	
Total Input	0.1 BTC
Total Output	0.0995 BTC
Fees	0.0005 BTC
Estimated BTC Transacted	0.015 BTC


```
{
  "txid": "0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2",
  "size": 258,
  "version": 1,
  "locktime": 0,
  "fee": 50000,
  "inputs": [
    {
      "coinbase": false,
      "txid": "7957a35fe64f80d234d76d83a2a8f1a0d8149a41d81de548f0a65a8a999f6f18",
      "output": 0,
      "sigscript":
"483045022100884d142d86652a3f47ba4746ec719bbfbd040a570b1deccbb6498c75c4ae24cb02204b9f039ff08df09cbe9f6add
ac960298cad530a863ea8f53982c09db8f6e381301410484ecc0d46f1918b30928fa0e4ed99f16a0fb4fde0735e7ade8416ab9fe4
23cc54123376376789d172787ec3457eee41c04f4938de5cc17b4a10fa336a8d752adf",
      "sequence": 4294967295,
      "pkscript": "76a9147f9b1a7fb68d60c536c2fd8aeaa53a8f3cc025a888ac",
      "value": 10000000,
      "address": "1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK",
      "witness": []
    }
  ],
  "outputs": [
    {
      "address": "1GdK9UzpHBzqzX2A9JFP3Di4weBwqgmoQA",
      "pkscript": "76a914ab68025513c3dbd2f7b92a94e0581f5d50f654e788ac",
      "value": 1500000,
      "spent": false,
      "spender": null
    },
    {
      "address": "1Cdid9KFAaatwczBwBttQcwXYCpvK8h7FK",
      "pkscript": "76a9147f9b1a7fb68d60c536c2fd8aeaa53a8f3cc025a888ac",
      "value": 8450000,
      "spent": false,
      "spender": null
    }
  ],
```


UTXO index 0

UTXO index 1

Bitcoin Network is a Shared Global Ledger



Ledger:
Append-only List of Transactions

Transaction:
Addr1 send Addr2 1.2 $BTC on Sep 1, 2022

Bitcoin System

Solutions for Consistency in the Bitcoin System

Q. How to confirm the global ledger having same transactions across all nodes of the bitcoin network (keep consistency) and ensure its scalability for millions of participants?

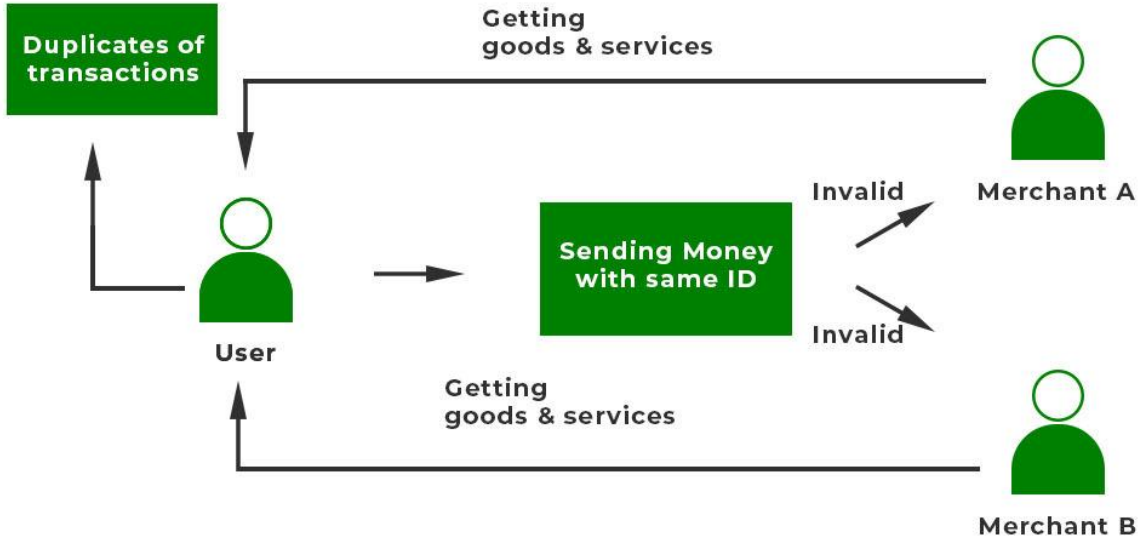
Example: Double-spending problem

- Sender writes and broadcasts message 1 that it transfer a UTXO to receiver A.
- Meanwhile, sender also writes and broadcasts message 2 that it transfer the same UTXO to receiver B.

How to handle or avoid this problem?

Bitcoin solutions:

1. Miner
2. Proof-of-Work (PoW) Consensus
3. Block & Blockchain
4. Miner Reward Mechanism

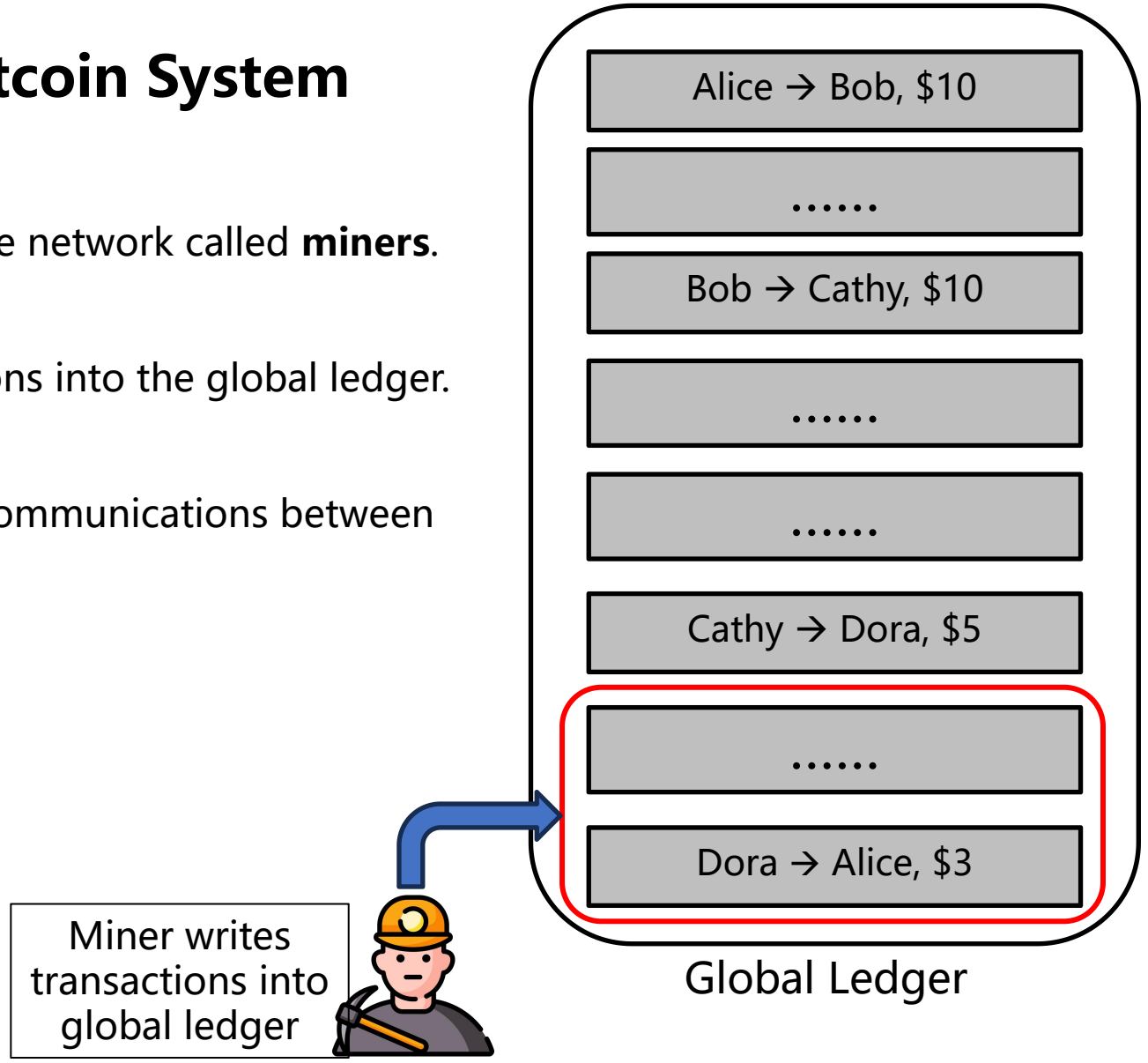


Bitcoin System

Solutions for Consistency in the Bitcoin System

Miner

- Beside senders and recipients, there are nodes in the network called **miners**.
- Miners can collect and record a bundle of transactions into the global ledger.
- In traditional distributed system, there are a lot of communications between nodes to achieve consensus.

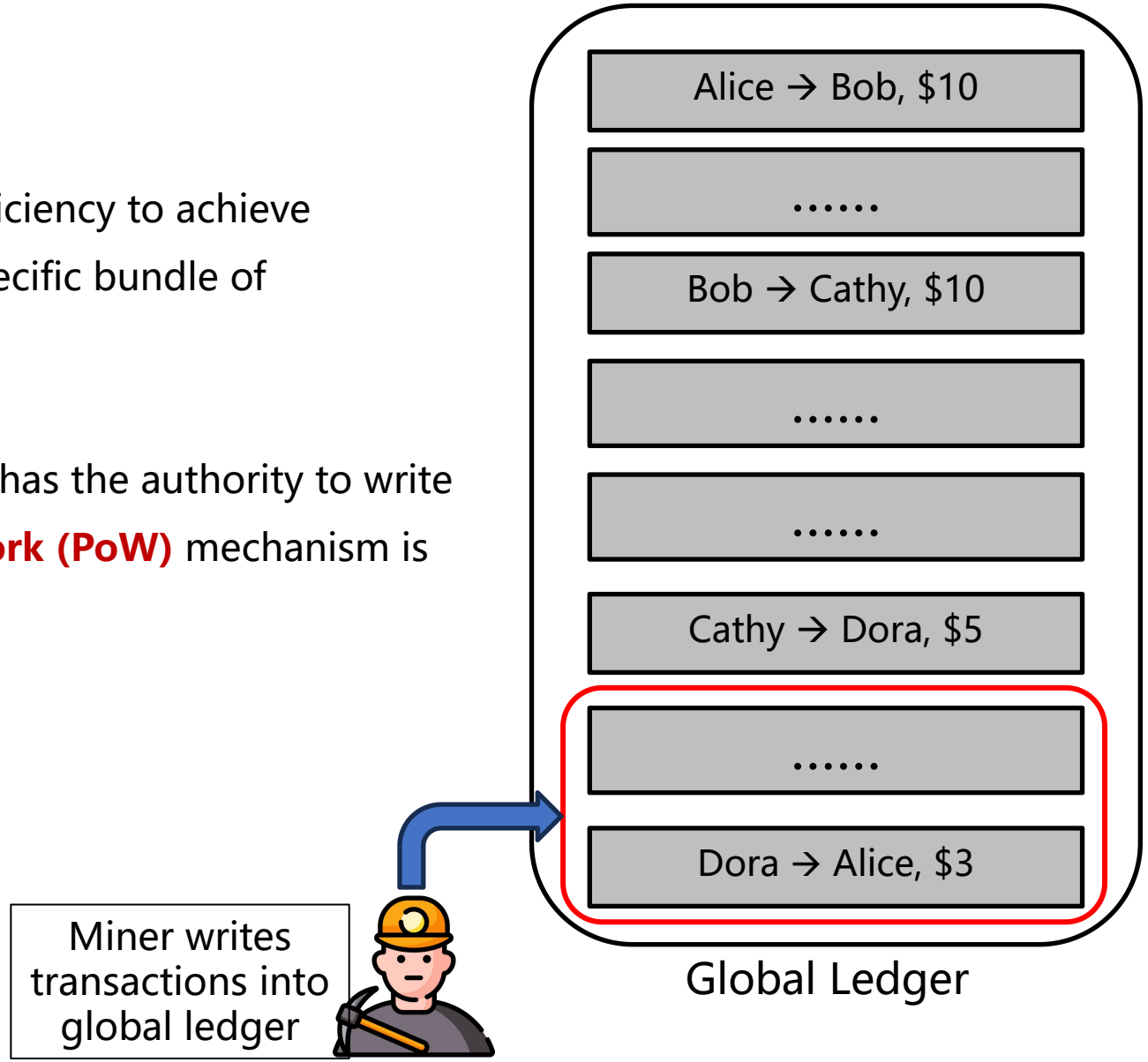


Bitcoin System

Miner

- To avoid conflicts in data recording and improve efficiency to achieve consensus, **ONLY one node** is allowed to write a specific bundle of transactions. (dictatorship mechanism)

- A mechanism is required to determine which miner has the authority to write transactions. In the Bitcoin system, the **Proof-of-Work (PoW)** mechanism is utilized to confirm the specific miner.



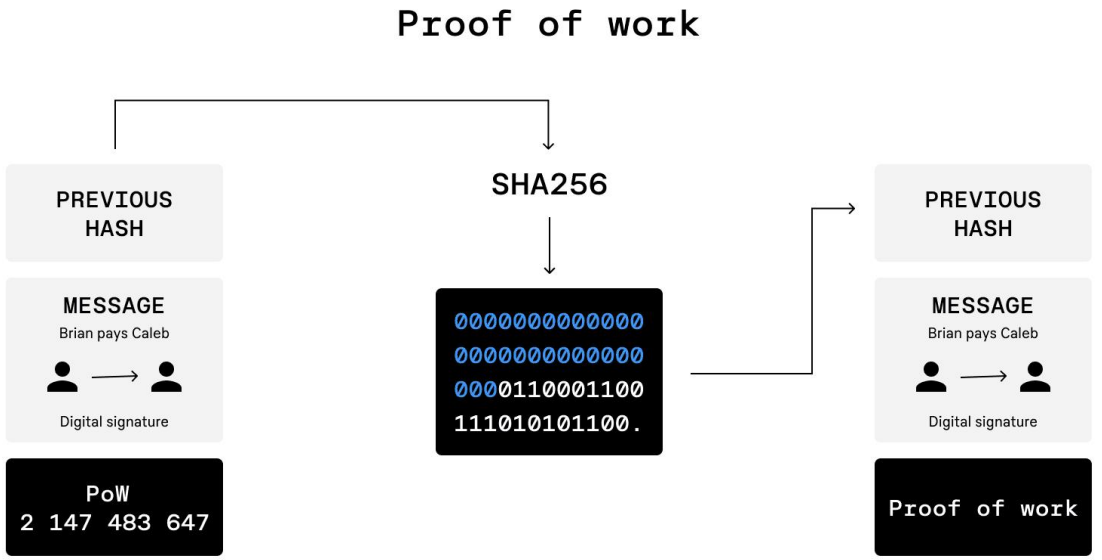
Bitcoin System

Proof-of-Work (PoW) Consensus

Mechanism

- Proof-of-Work (PoW) is a consensus mechanism in the Bitcoin system to validate miners. It involves calculating a valid block hash to verify Bitcoin transactions and add the subsequent block to the blockchain.

- To ensure that only valid blocks are accepted into the blockchain and to prevent unauthorized transaction inclusion, the Bitcoin system mandates that blocks must have a valid block hash. This is achieved using a **cryptographic hash function**, which secures the integrity of the blockchain.



Bitcoin System

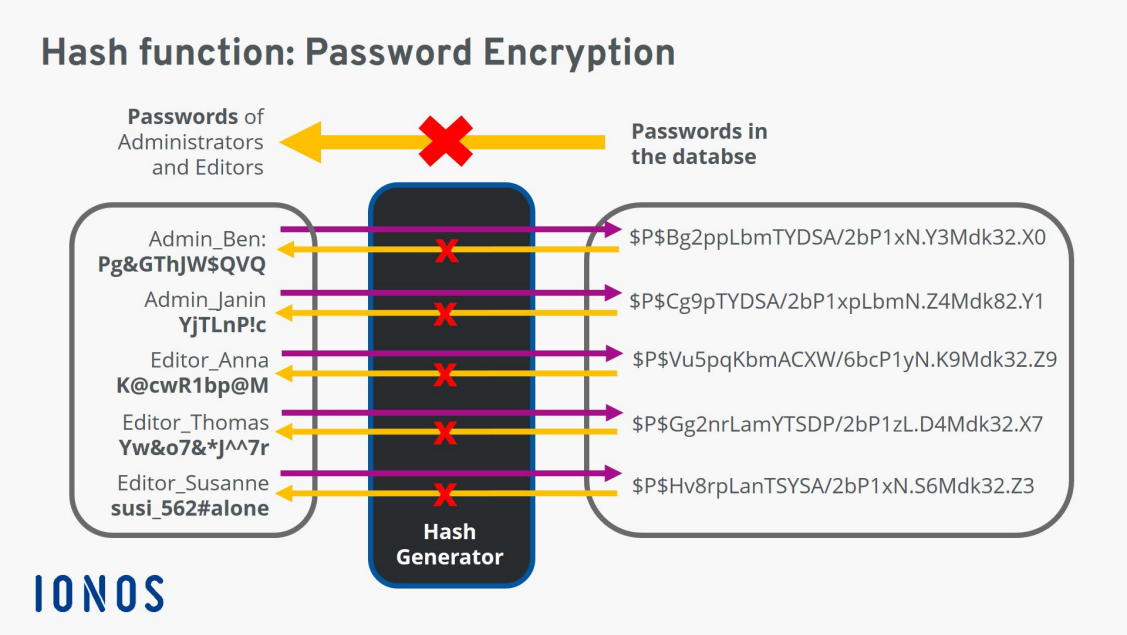
PoW

Hash Function(e.g. SHA256)

- A hash function is a mathematical function that converts any digital data into an output string (called hash) with a fixed number of characters.

- Hashing is a one-way process that input into the output. While it is straightforward to generate the hash from the input, it is impossible to retrieve the original data from the output.

- Hashing is useful to ensure the authenticity of a piece of data, as any small change to the message will result in a completely different hash value.



SHA256 online tool:
https://emn178.github.io/online-tools/sha256.html

Hash Function

Hash Function



- hash (blender)
  - to chop (food, such as meat and potatoes) into small pieces

- function

$$y = f(x)$$

input

output

function

Feature 1: Irreversible

- Irreversible process from input to output:
  - You can derive y from x, but not from y to x

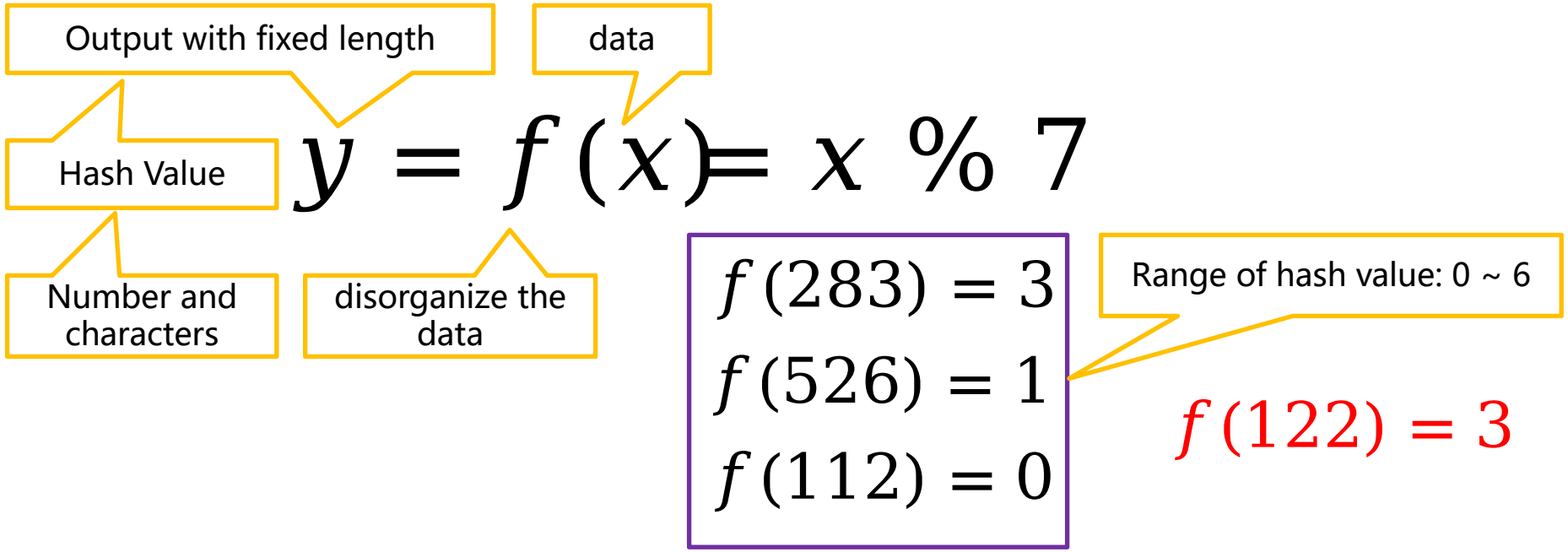


Feature 2: Deterministic

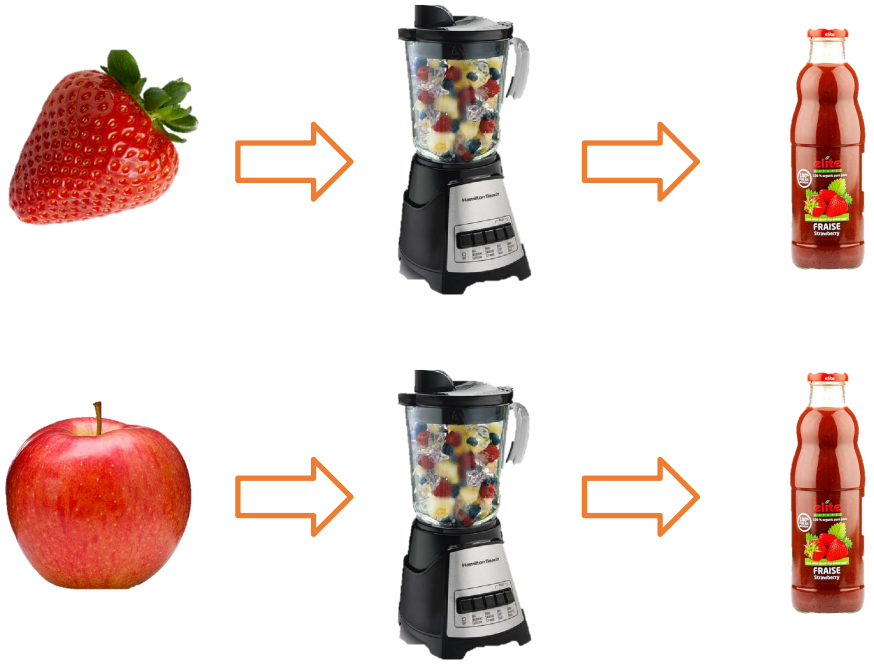
- Same inputs lead to same outputs



Implementation of Hash Function

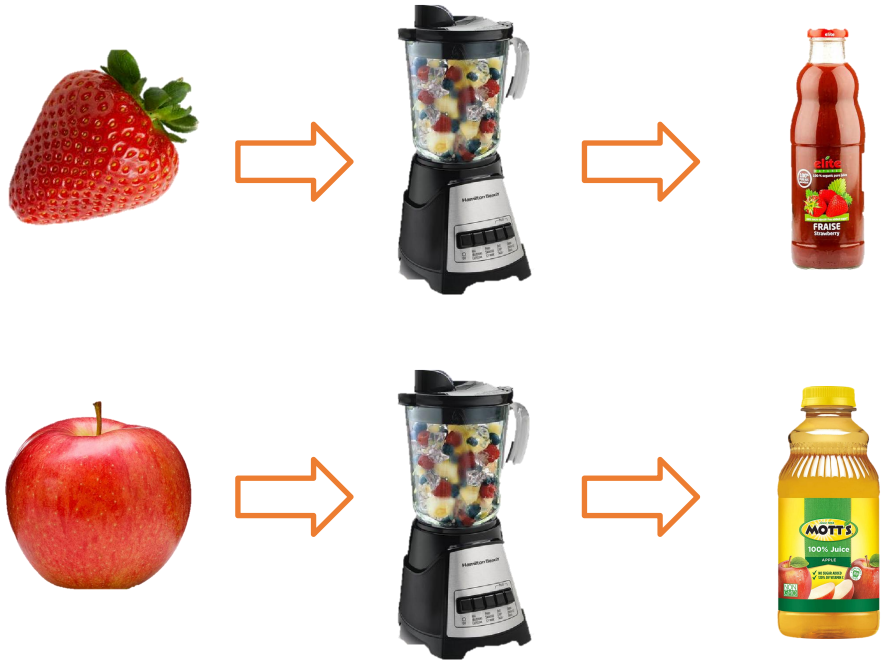


Collision of Hash Function



Feature 3: Minimal Collision

- Well designed hash function avoids output collisions
  - Different inputs lead to different outputs



Implementation of Hash Function

- How?

	$f(x) = x \ mod \ 1000$	$g(x)$
9189000	$f(9189000) = 0$	$009 + 189 + 000 = 198$
3152000	$f(3152000) = 0$	$003 + 152 + 000 = 155$

Secure Hash Algorithm (SHA)

- The SHA is a family of cryptographic hash functions

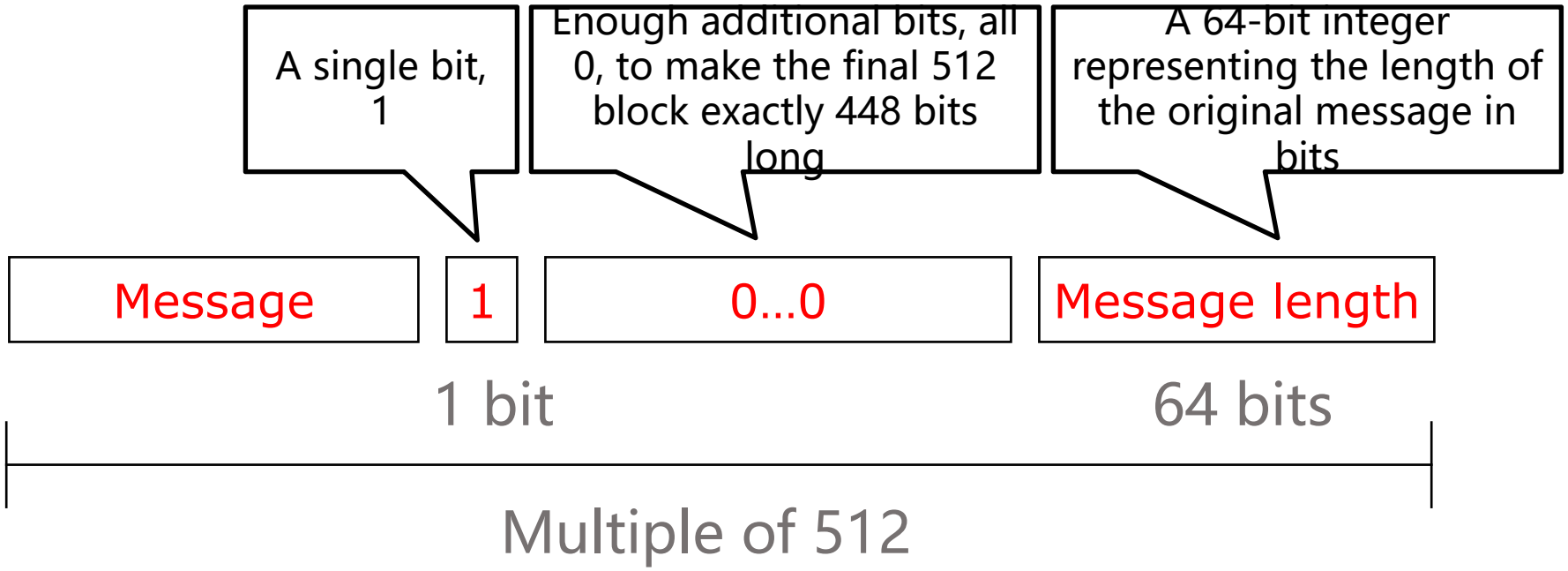




- Any change to a message will, with high probability, result in a different message digest.

Step 1 -- Padding

- *Padding* → the total length of a padded message is multiple of 512
  - Every message is padded even if its length is already a multiple of 512
- Padding is done by appending to the input



Example 1

- M = 01100010 11001010 1001 (20 bits)

- Padding is done by appending to the input
  - A single bit, 1
  - 427 0s
  - A 64-bit integer representing 20

- Pad(M) = 01100010 11001010 10011000 … 00010100

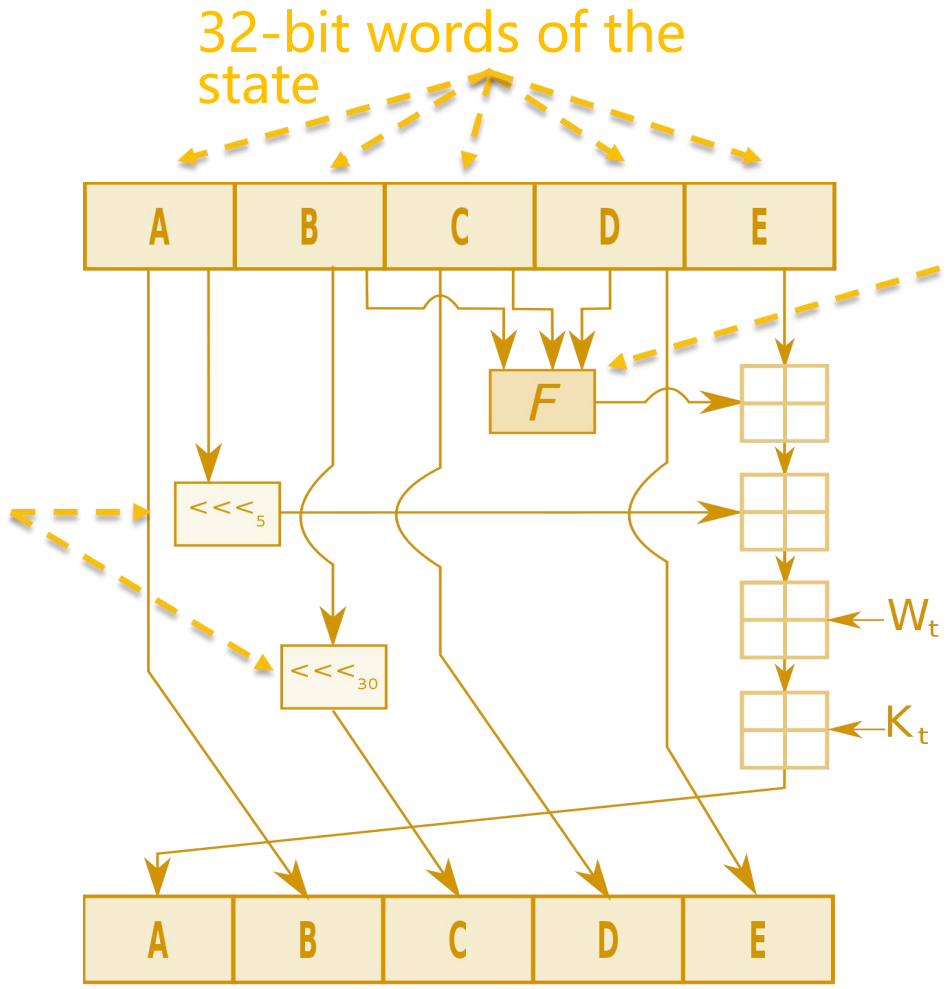
Example 2

- Length of M = 500 bits
- Padding is done by appending to the input:
  - A single bit, 1
  - 459 0s
  - A 64-bit integer representing 500
- Length of Pad(M) = 1024 bits

Step 2 -- Dividing Pad(M)

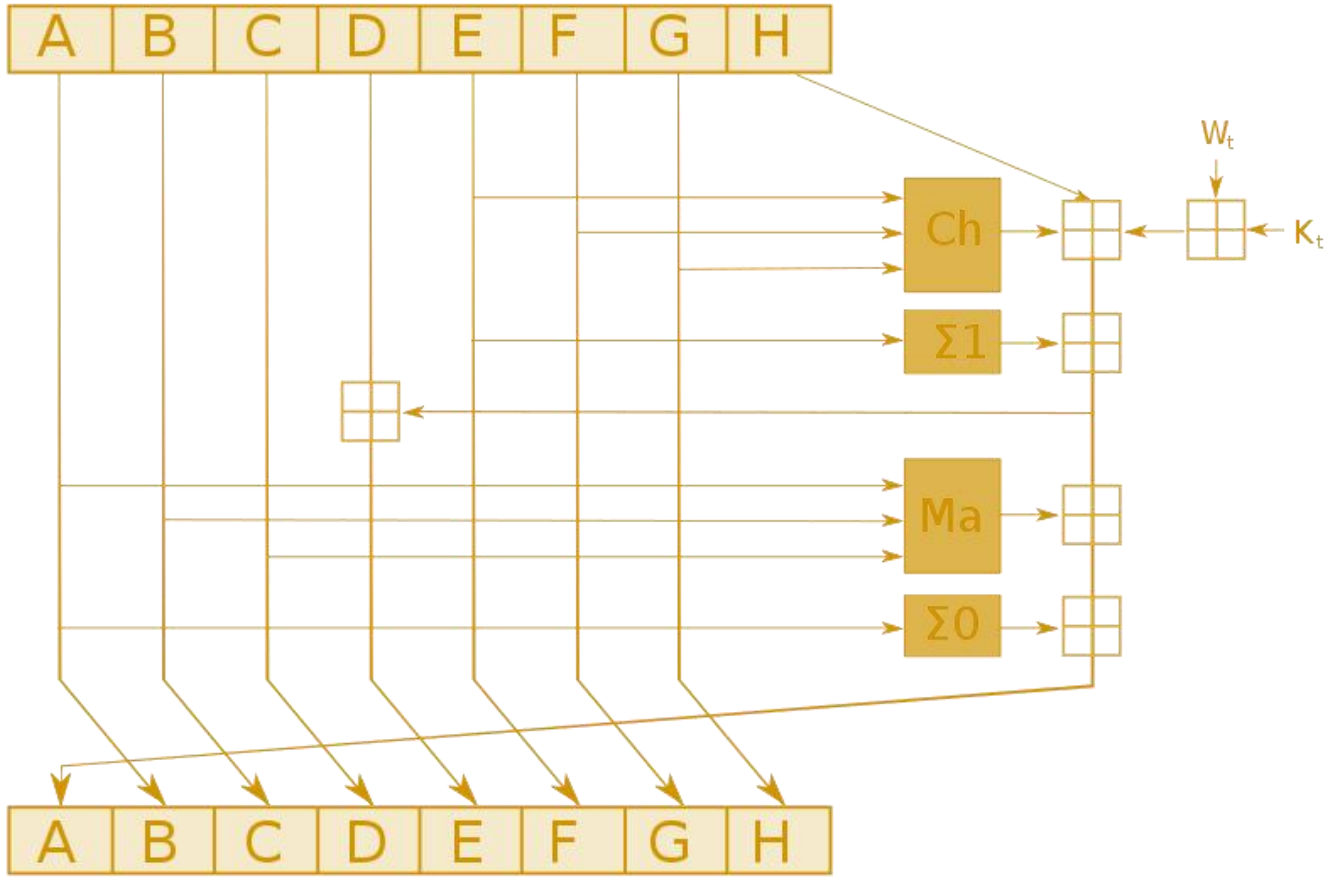
- Pad (M) = B_1, B_2, B_3, …, B_n
- Each B_i denote a 512-bit block
- Each B_i is divided into 16 32-bit words
  - W_0, W_1, …, W_{15}

Iteration within the SHA-1 Compression



a left bit rotation by n places

SHA-2 Family



SHA Family

	Output size (bits)	Internal state size (bits)	Block size (bits)	Max message size (bits)	Word size (bits)	Rounds	Operations	Collisions found
SHA-0	160	160	512	$2^{64} - 1$	32	80	+, and, or, xor, rot	Yes
SHA-1	160	160	512	$2^{64} - 1$	32	80	+, and, or, xor, rot	None (2^{52} attack)
SHA-2	256/224	256	512	$2^{64} - 1$	32	64	+, and, or, xor, shr, rot	None
	512/384	512	1024	$2^{128} - 1$	64	80	+, and, or, xor, shr, rot	None

Example Hashes with SHA1

SHA1("The quick brown fox jumps over the lazy dog")
= 2fd4e1c6 7a2d28fc ed849ee1 bb76e739 1b93eb12

SHA1("The quick brown fox jumps over the lazy cog")
= de9f2c7f d25e1b3a fad3e85a 0bd17d9b 100db4b3

The hash of the zero-length message is:
SHA1("")
= da39a3ee 5e6b4b0d 3255bfef 95601890 afd80709

Example Hashes with Different SHA

SHA1("The quick brown fox jumps over the lazy dog")
= 2fd4e1c6 7a2d28fc ed849ee1 bb76e739 1b93eb12

SHA-256
SHA256("The quick brown fox jumps over the lazy dog")
= d7a8fbb307d7809469ca9abcb0082e4f8d5651e46d3cdb762d02d0bf37c9e592

SHA-512
SHA512("The quick brown fox jumps over the lazy dog")
= 07e547d9586f6a73f73fbac0435ed76951218fb7d0c8d788a309d785436bbb64
2e93a252a954f23912547d1e8a3b5ed6e1bfd7097821233fa0538f3db854fee6

Bitcoin Cryptocurrency System

3. Solutions for Consistency in the Bitcoin System

3.2 PoW

- By utilizing a **cryptographic hash function** (bitcoin: SHA256), the block hash offers 2 key features:
    1. **Data integrity assurance:** Any slight change in transactional data will result in a completely different block hash, ensuring data integrity and tamper resistance.

    2. **Hash verification simplicity:** It is straight forward for any node to verify the correctness of a hash. By comparing the computed hash with the provided one, nodes can easily validate the integrity of the underlying transactional data.

Proof of Work



Proposed by Dwork & Naor 1993

Design for Junk Mail Prevention

Adopted by Bitcoin

Nakamoto PoW Consensus

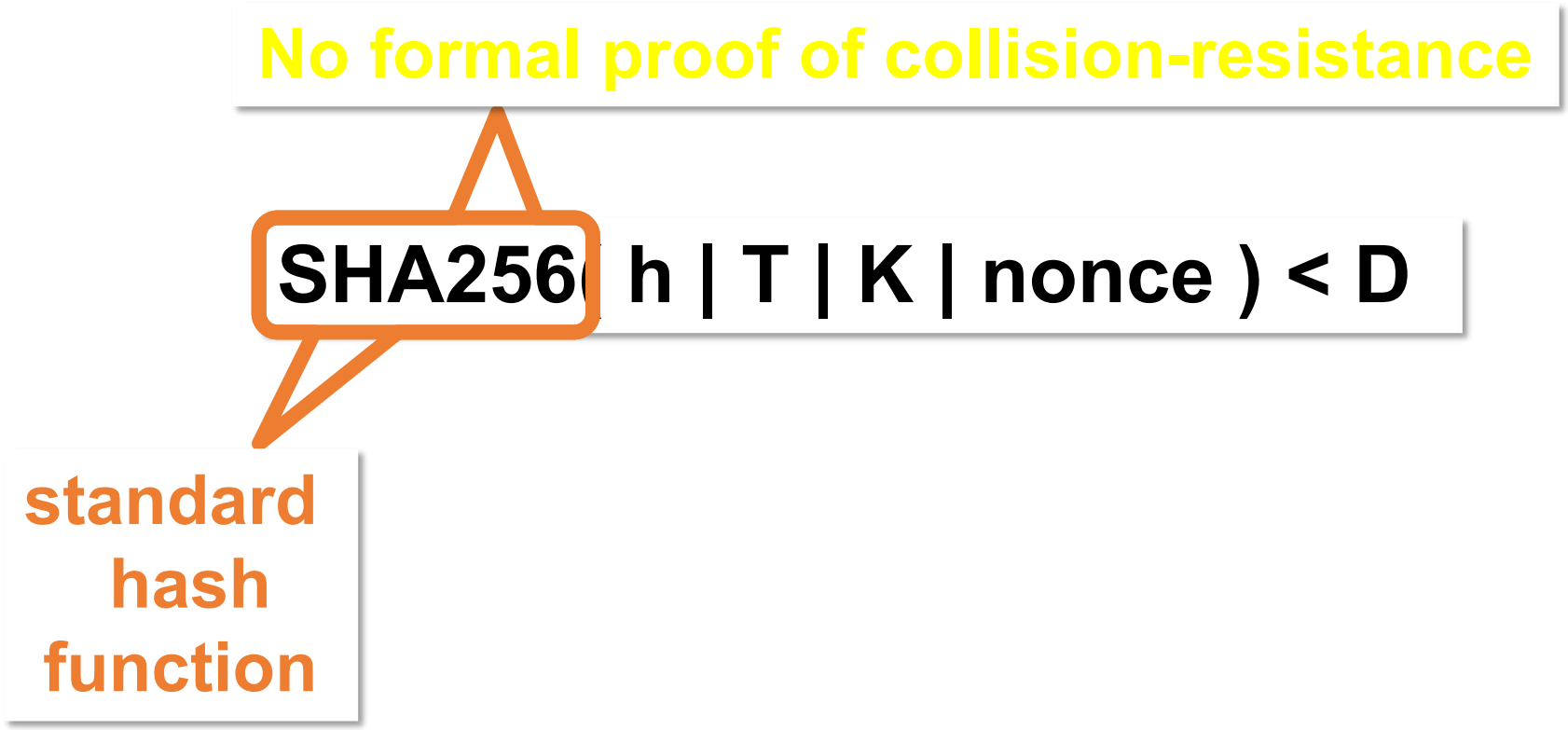
Pricing via Processing
or
Combatting Junk Mail*

Cynthia Dwork * Moni Naor †

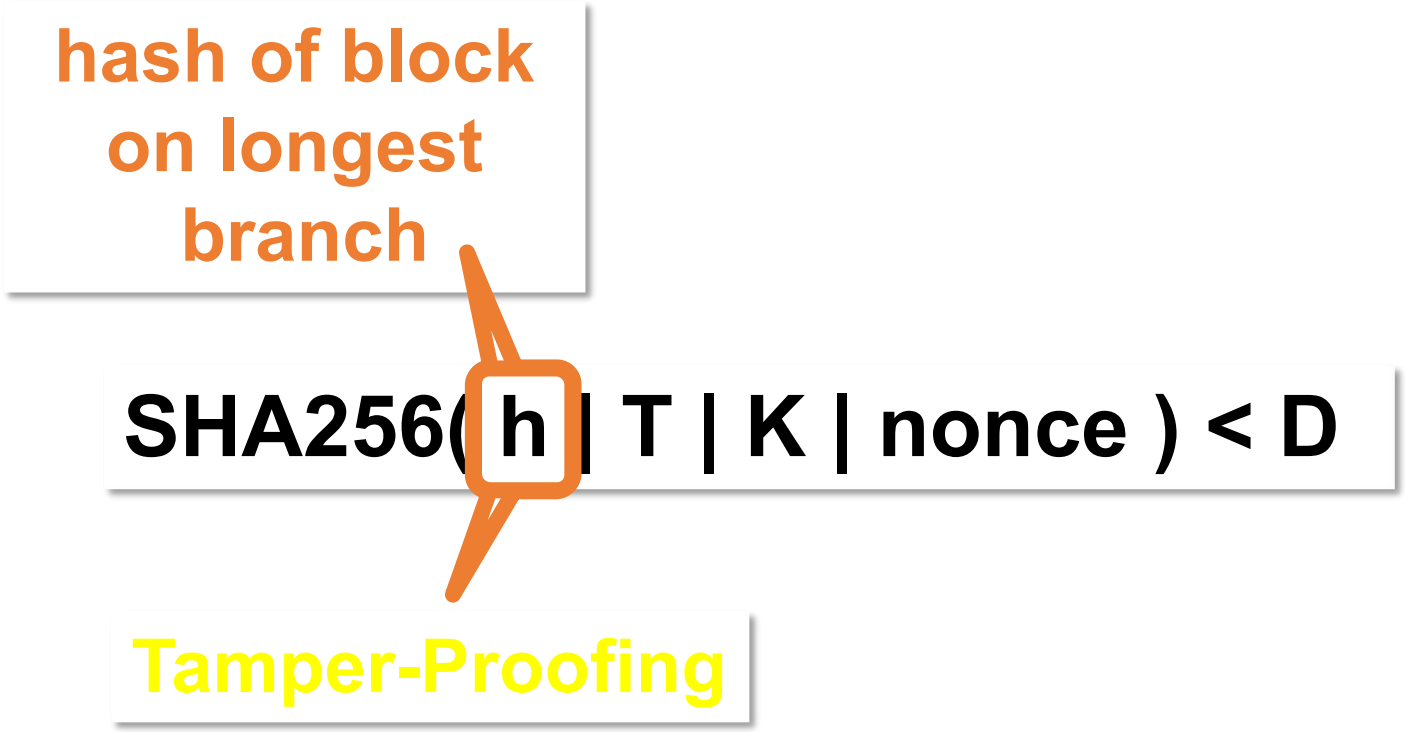
Abstract

We present a computational technique for combatting junk mail, in particular, and controlling access to a shared resource, in general. The main idea is to require a user to compute a moderately hard, but not intractable, function in order to gain access to the resource, thus preventing frivolous use. To this end we suggest several *pricing functions*, based on, respectively, extracting square roots modulo a prime, the Fiat-Shamir signature scheme, and the Ong-Schnorr-Shamir (cracked) signature scheme.

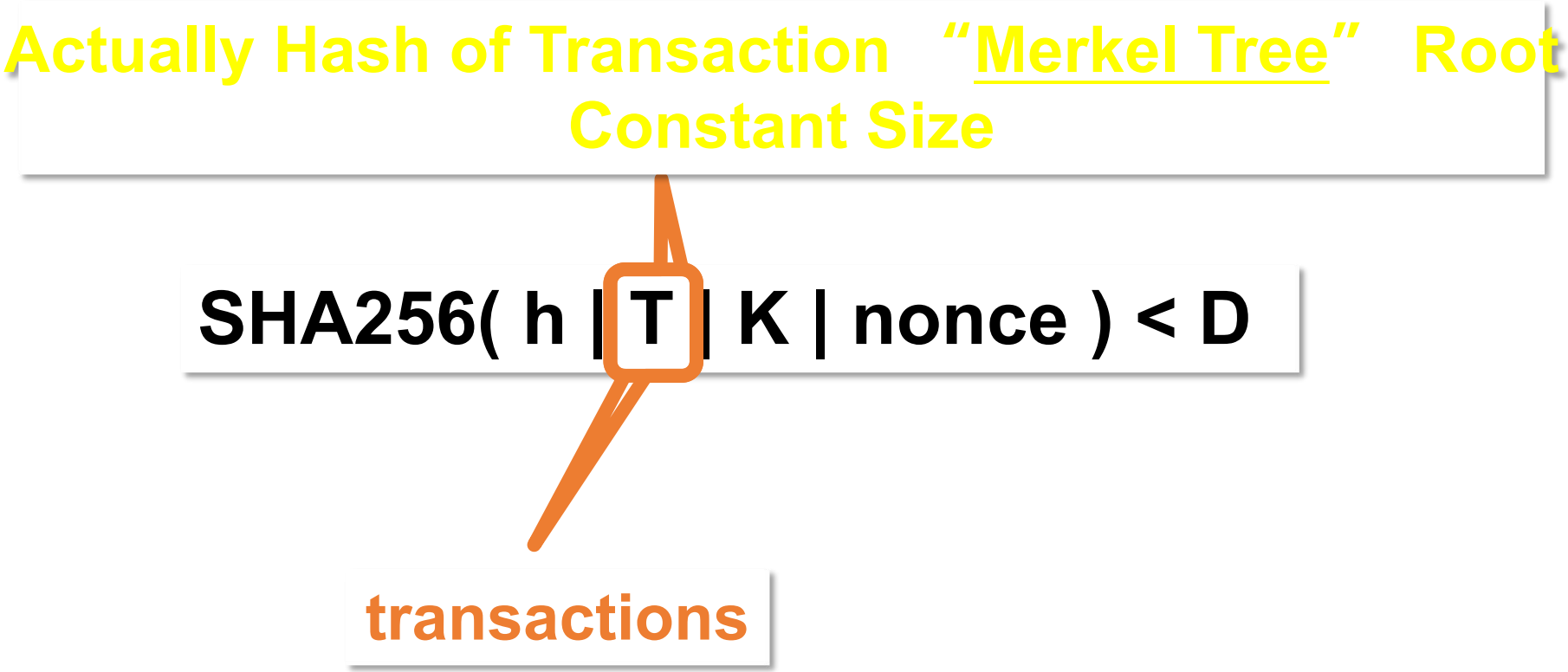
Nakamoto PoW Consensus in One Line



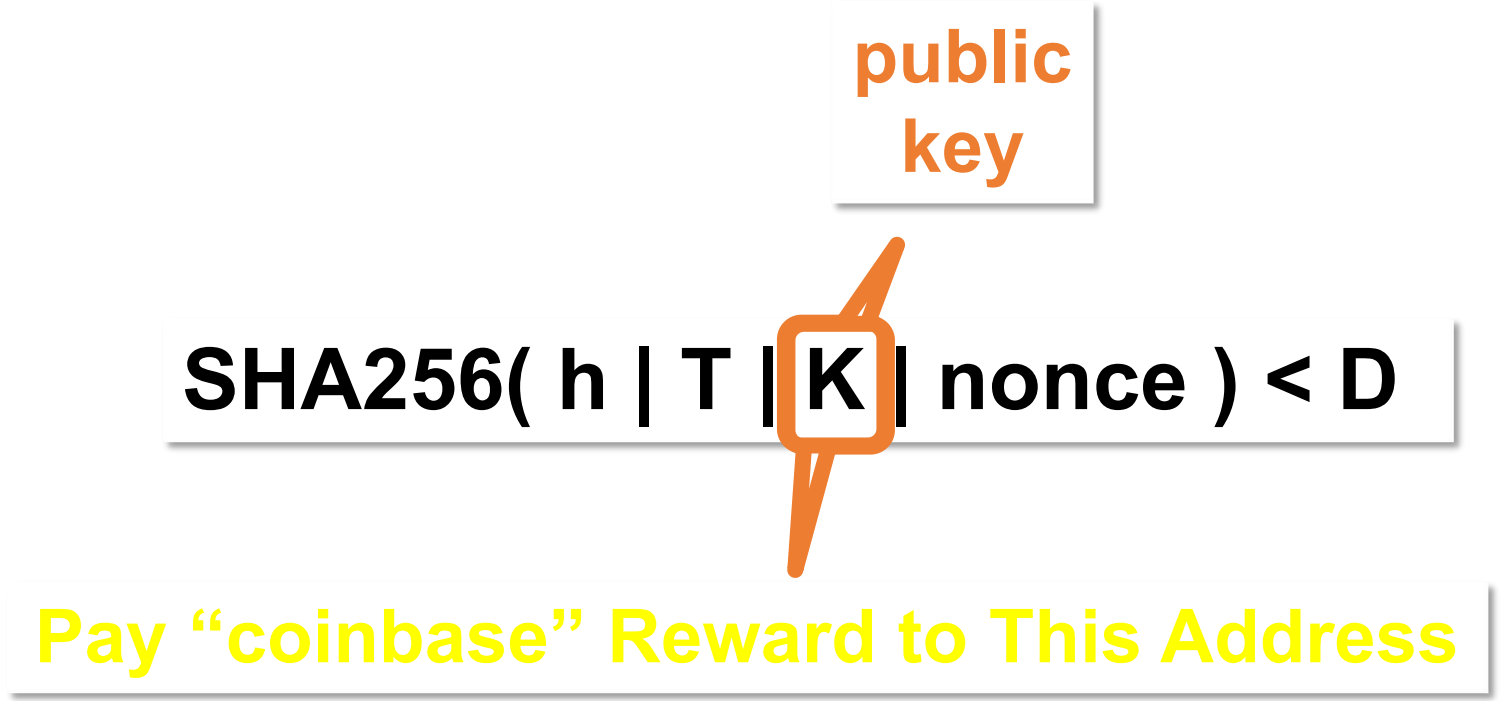
Nakamoto PoW Consensus in One Line



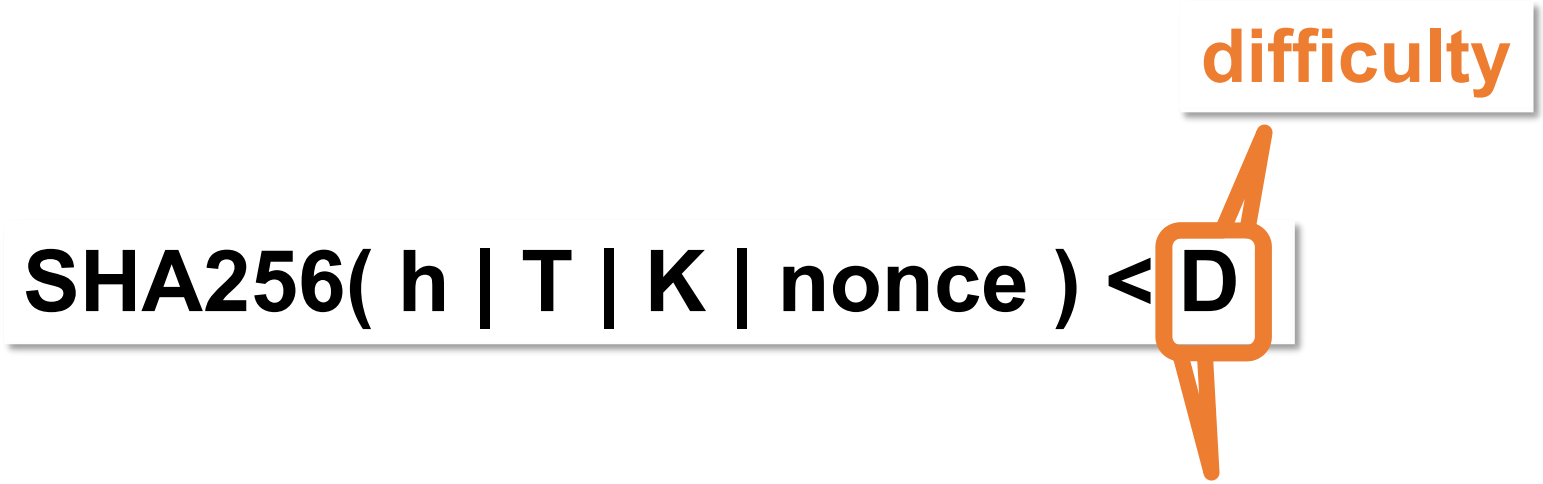
Nakamoto PoW Consensus in One Line



Nakamoto PoW Consensus in One Line



Nakamoto PoW Consensus in One Line

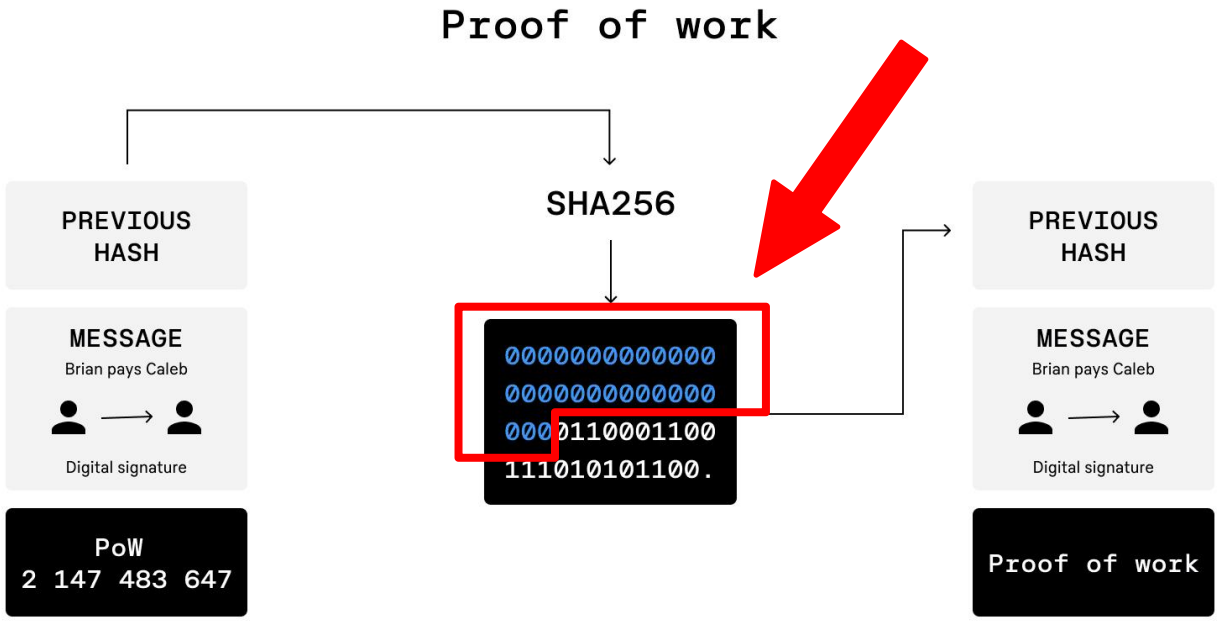


Bitcoin Cryptocurrency System

3. Solutions for Consistency in the Bitcoin System

3.2 PoW

- The Bitcoin system incorporates a parameter called "**difficulty**" that regulates the level of computational effort required to calculate a valid block hash. As the difficulty value increases, it signifies that a valid block hash must include a greater number of leading 0 in its hash value.



Bitcoin Cryptocurrency System

3. Solutions for Consistency in the Bitcoin System

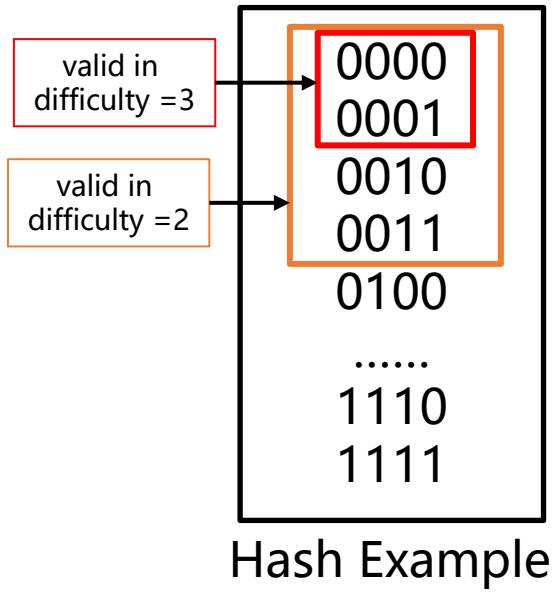
3.2 PoW

Example of Difficulty

- For example, let's consider calculating a hash consisting of 0 and 1 with a length of four digits. If the difficulty level specifies how many digits must be 0 at the beginning of this hash:

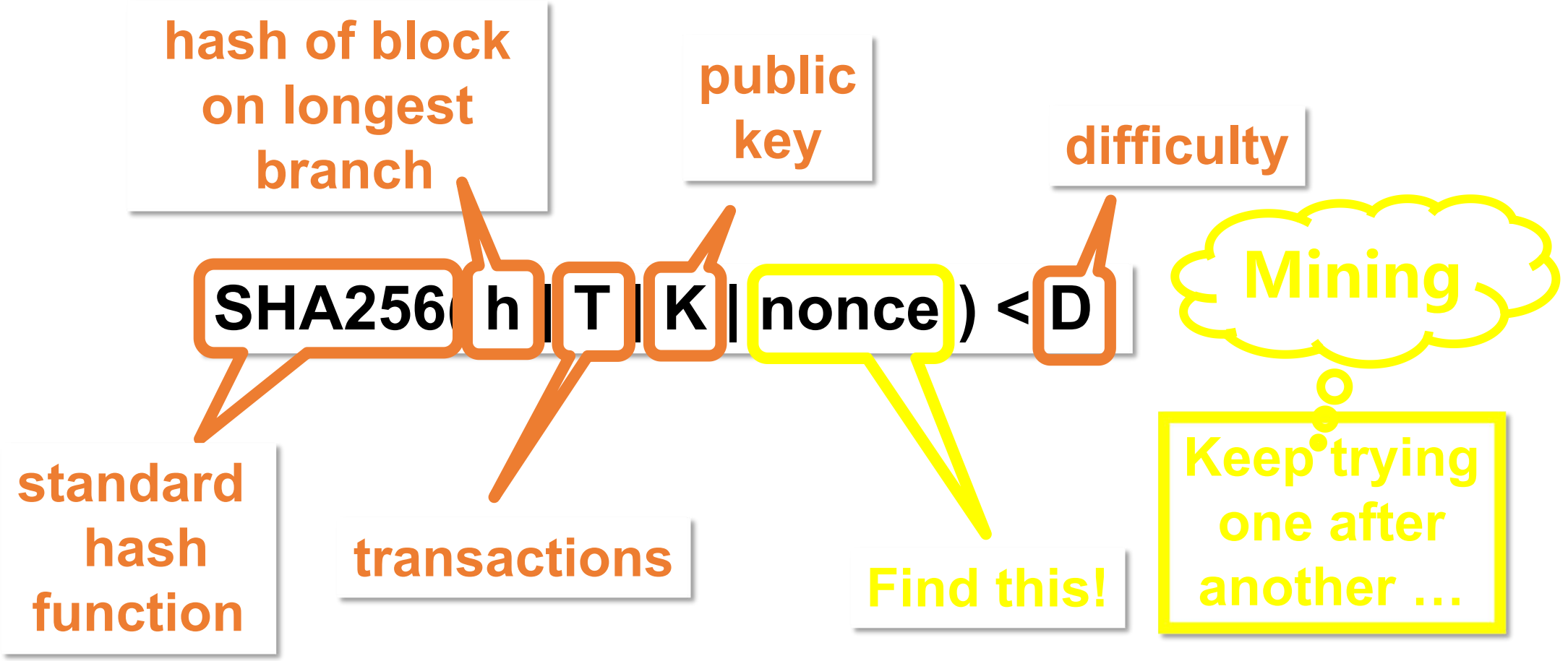
  When difficulty = 2, the probability of finding the valid hash in one computation is $\frac{4}{16} = \frac{1}{4}$

  When difficulty = 3, the probability of finding the valid hash in one computation is $\frac{2}{16} = \frac{1}{8}$



Hash Example

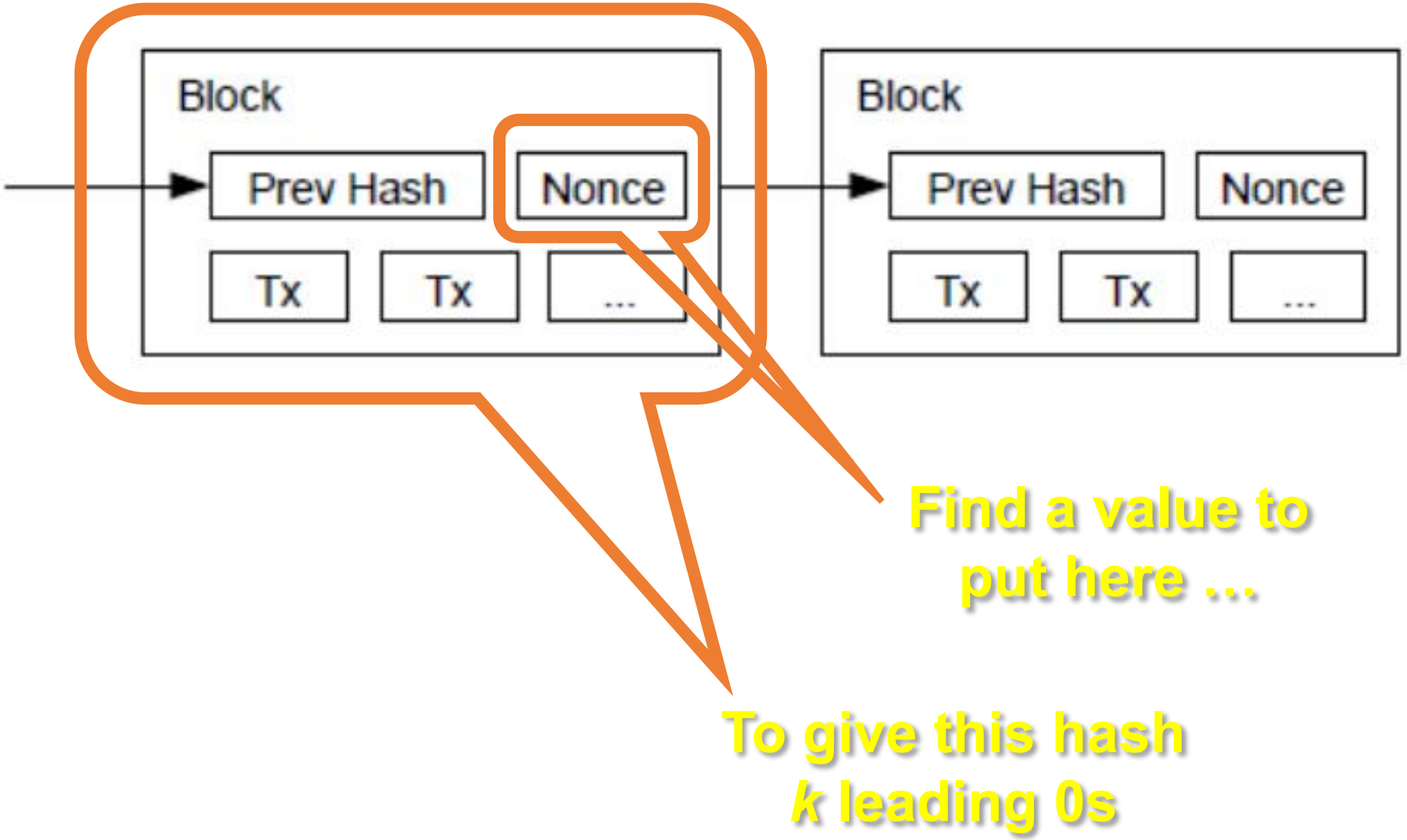
Nakamoto PoW Consensus in One Line



Philosophy in PoW Consensus Design

Hard to solve

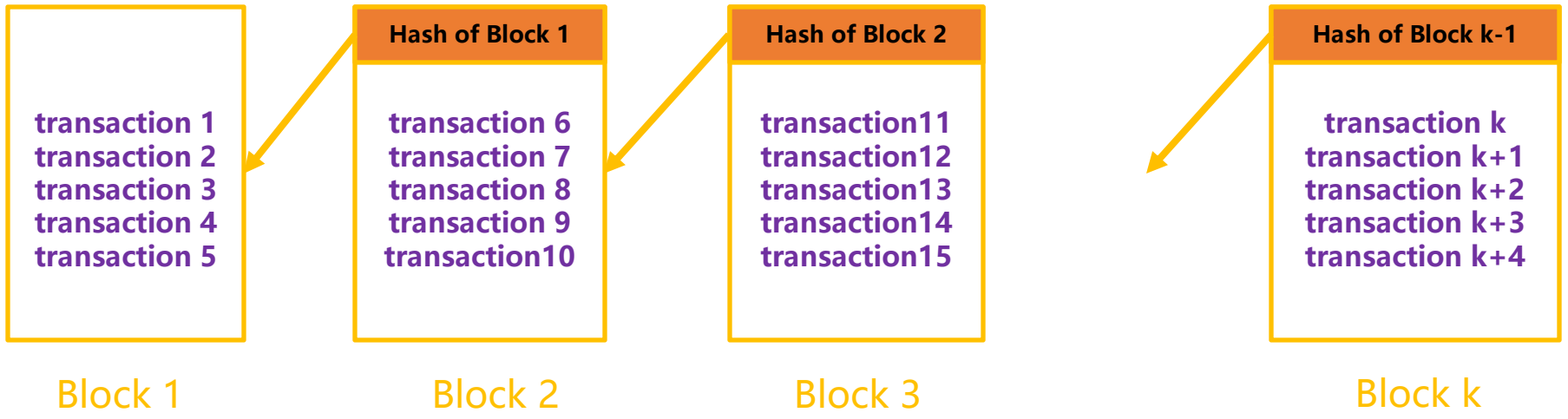
Easy to verify


Block
Prev Hash
Nonce
Tx
Tx
...
Block
Prev Hash
Nonce
Tx
Tx
...
Find a value to put here …
To give this hash *k* leading 0s

Bitcoin System

Block & Blockchain

- In BitCoin ledger, we write *transactions* to *blocks*



Bitcoin System

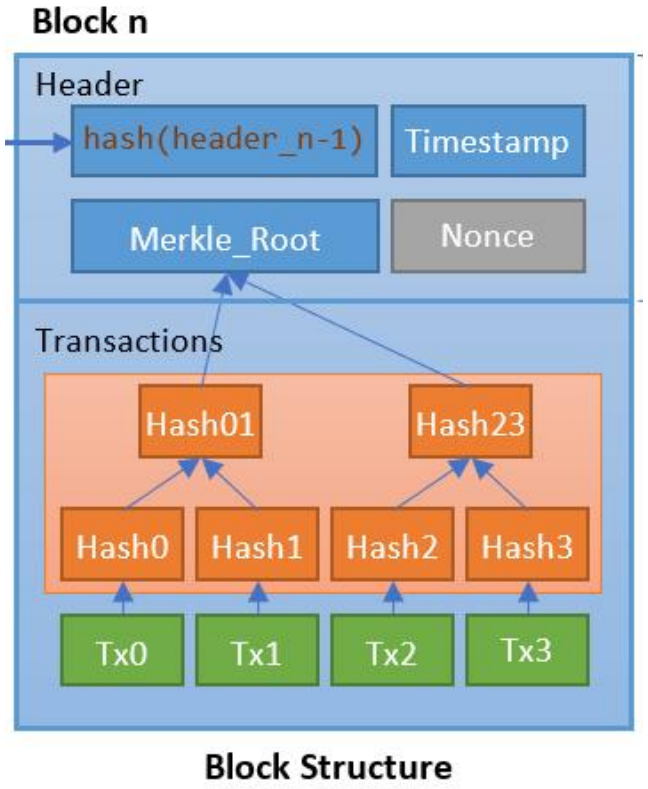
Block & Blockchain

A block's data structure is mainly divided into 2 parts:

- Transactions (It includes a bundle of transactions that need to be written into the global ledger.)
- Header (It includes extra information such as the previous block's hash, timestamp, nonce, and Merkle root for block validation and identification.)

- The hash of a block is obtained by applying a **cryptographic hash function** to the header parameters of the block.

  $block_hash_n$ = Hash($block_hash_{n-1}$, Merkle root, Timestamp, Nonce)



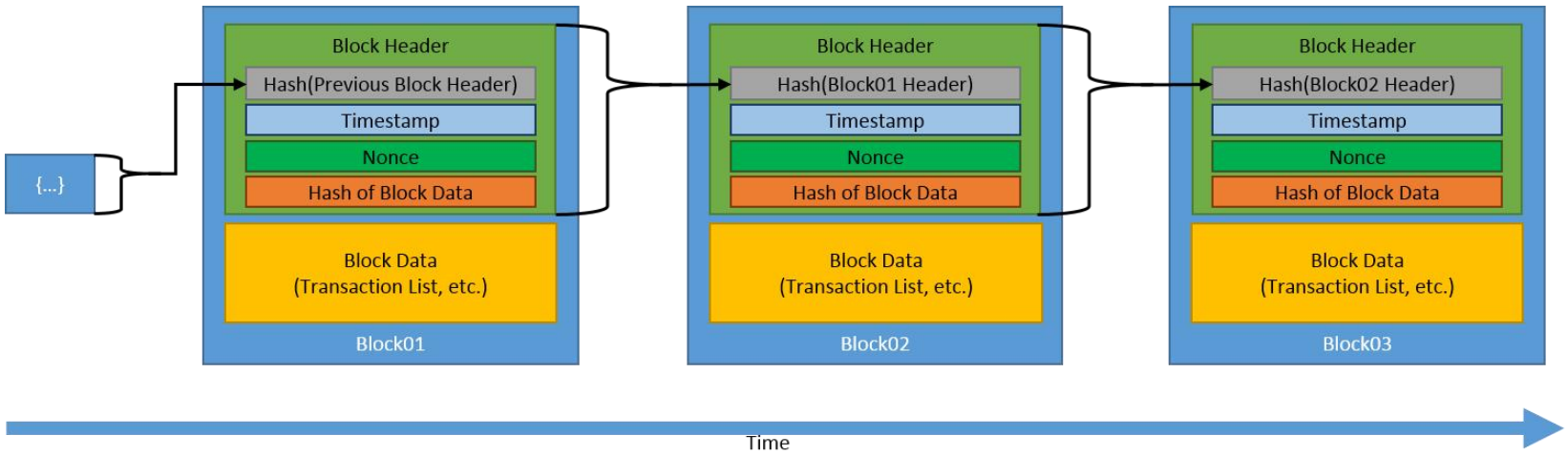
Block Structure

Bitcoin System

Block & Blockchain

- By including the hash of the previous block, it becomes challenging to alter any data in any block. If someone attempts to modify the data in a block, the block hash will change. Since the subsequent block includes the hash of the previous block, the hash of that following block will also change. Thus, when someone intends to modify any data in a block, they would need to rebuild all subsequent blocks.

- In this data structure, each block maintains a link to the previous block, which gives rise to the term **"blockchain"** .



Bitcoin System

Incentive Mechanism

As calculating the valid block hash have to use many computational resources, bitcoin system provides an Incentive Mechanism to encourage miners to contribute the system.

A miner receives 2 types of rewards when it broadcasts a valid block successfully:

1. **Incentive Reward**
2. **Transaction Fee**

Mining Reward

By convention, the first transaction in a block is a special transaction that starts a new coin owned by the creator of the block. This adds an incentive for nodes to support the network, and provides a way to initially distribute coins into circulation, since there is no central authority to issue them

You mine a block, you get paid

How new bitcoins are generated

"Coinbase" Transaction

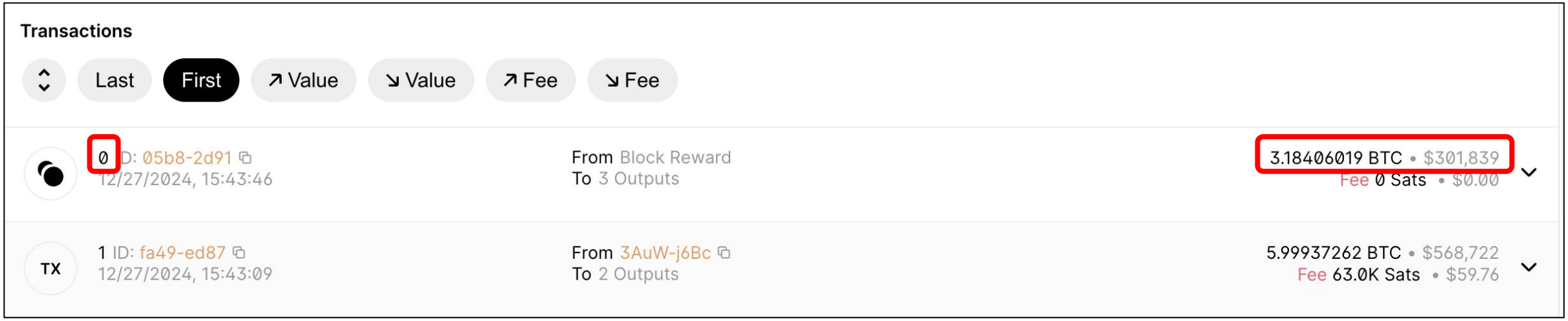
Bitcoin System

Incentive Mechanism

1. Incentive Reward

- Before a miner calculates the block hash, the Bitcoin protocol allows it to add a transaction that creates a specific value of Bitcoin to its address. This transaction usually appears as the first transaction (index = 0) in the block.

- When a miner successfully finds a valid block hash, this reward transaction is recorded in the global ledger, then the miner can claim the reward.

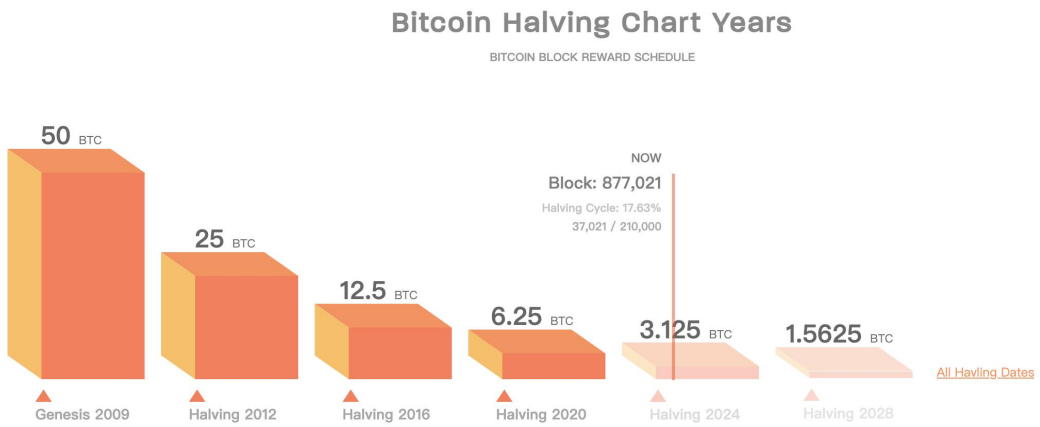


Bitcoin Cryptocurrency System

Incentive Mechanism

1. Incentive Reward

- In the Bitcoin system, the new block reward is cut in half after every 210,000 blocks have been generated.

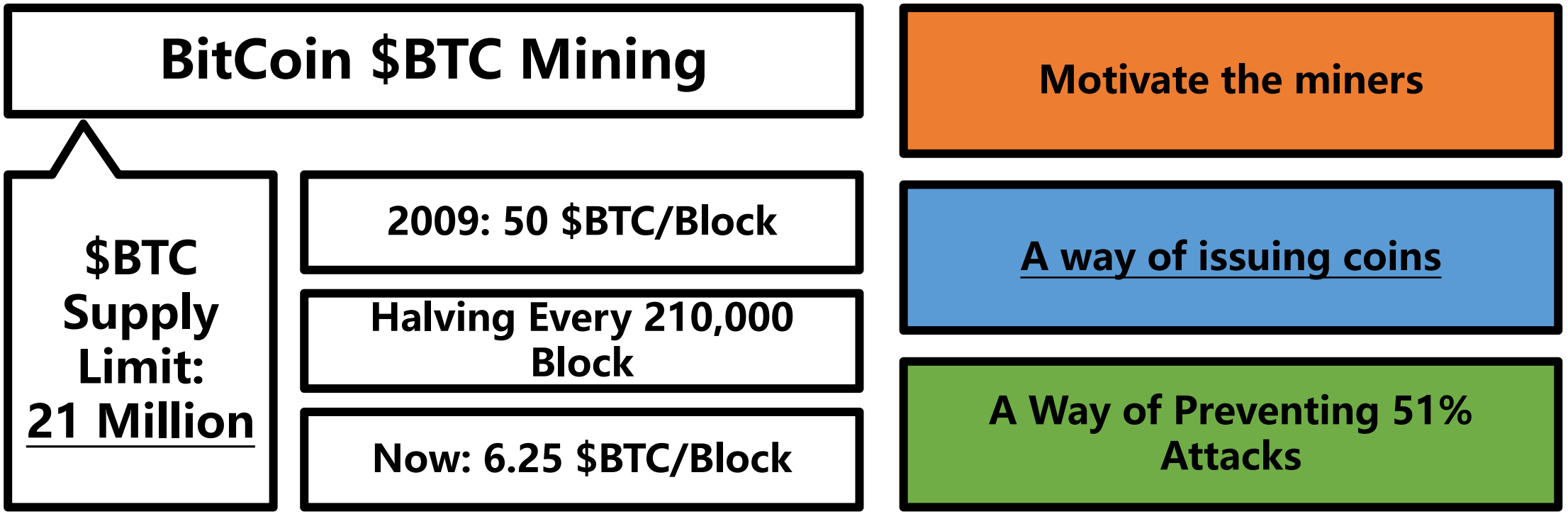


All Bitcoin Halving Dates

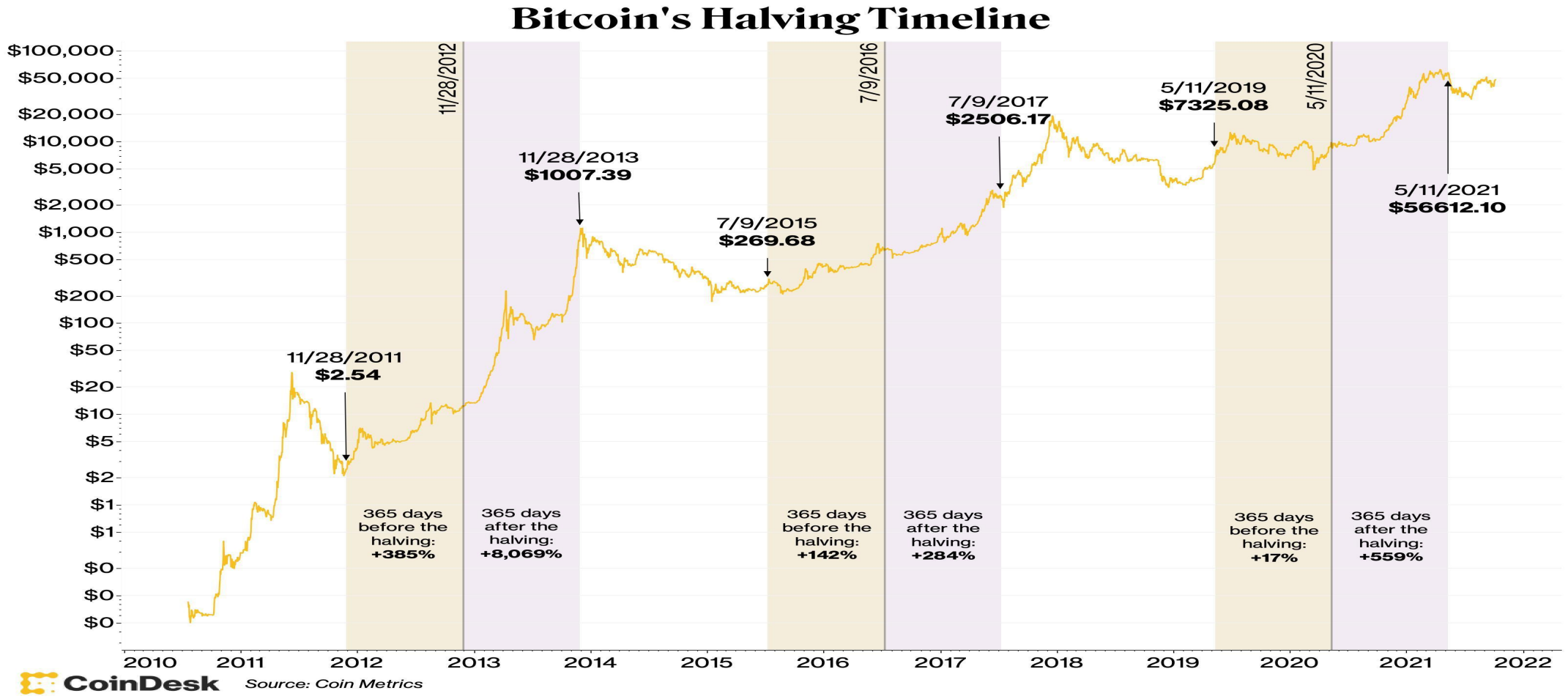
Halving	Year	Block Height	Block Reward	Date
		0 (Genesis Block)	50.00000000 BTC	January 3, 2009 6:15:05 PM UTC
1	2012	210,000	25.00000000 BTC	November 28, 2012 03:24:38 PM UTC
2	2016	420,000	12.50000000 BTC	July 9, 2016 4:46:13 PM UTC
3	2020	630,000	6.25000000 BTC	May 11, 2020 7:23:43 PM UTC
4	2024	840,000	3.12500000 BTC	April 20, 2024
5	2028	1,050,000	1.56250000 BTC	Mar 19, 2028 02:17:57 PM UTC (PROJECTED)
6	2032	1,260,000	0.78125000 BTC	

https://www.coinwarz.com/bitcoin-halving#dates

Bitcoin Mining Rewards



Bitcoin Halving and BitCoin Price



Another Incentive: Transaction Fee

The incentive can also be funded with transaction fees. If the output value of a transaction is less than its input value, the difference is a transaction fee that is added to the incentive value of the block containing the transaction. Once a predetermined number of coins have entered

Users can include transaction fee as *tips*

Higher fees buy lower latency?

This will be a problem later…

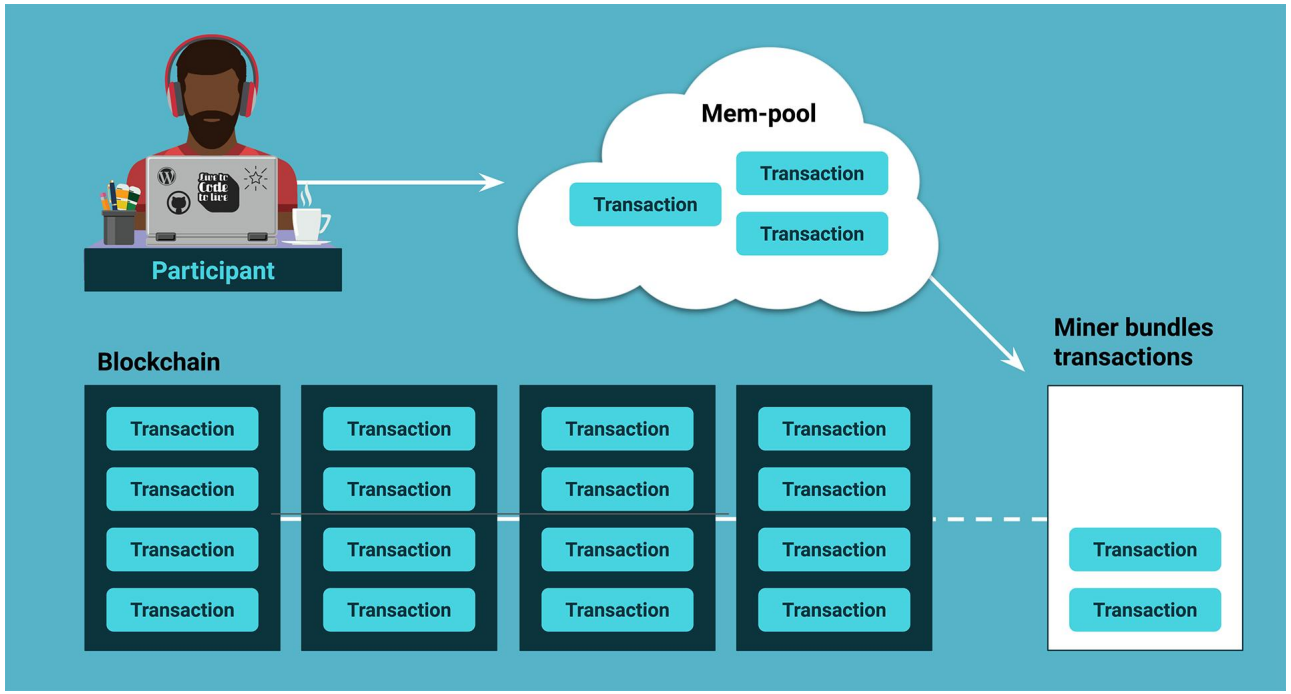
The only incentive after 2140 as Mining Reward become 0

Bitcoin System

Incentive Mechanism

2. Transaction Fee

- Since each block has a capacity or size limit, miners cannot include all received transactions in a single block. They must decide which transactions should be included.



Received Transactions: 3

Remaining capacity: 2

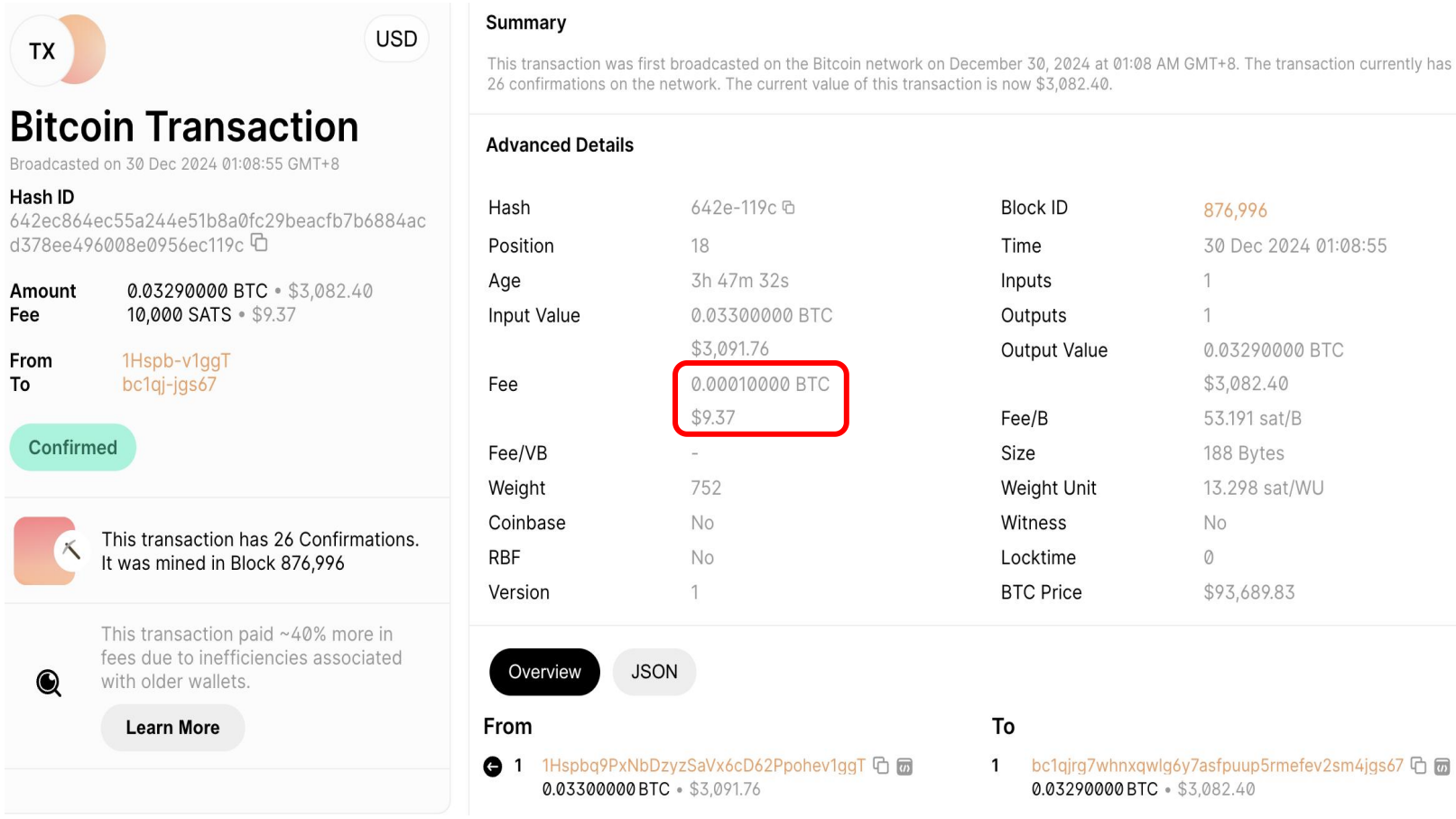
Bitcoin System

Incentive Mechanism

2. Transaction Fee

- The solution implemented in the Bitcoin system is to create an **auction system**.

- Each transaction sender is required to provide a transaction fee amount. When a miner includes a transaction, they can earn the transaction fee associated with it.



Fee = Input Value – Output Value

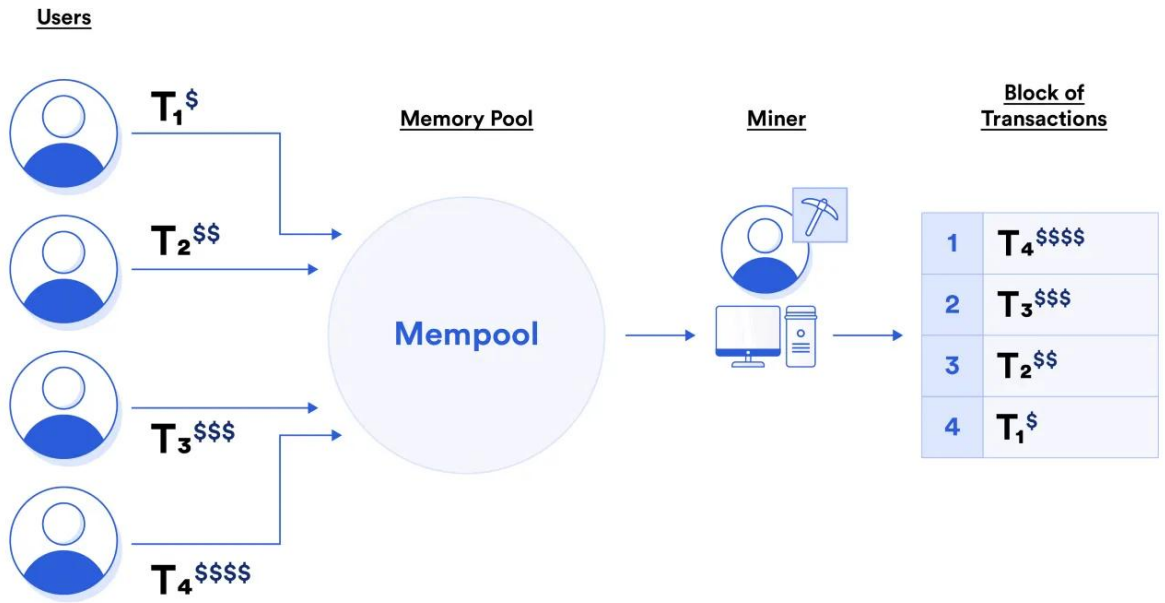
https://www.blockchain.com/explorer/transactions/btc/642ec864ec55a244e51b8a0fc29beacfb7b6884acd378ee496008e0956ec119c

Bitcoin System

Incentive Mechanism

2. Transaction Fee

- In this auction system, miners are motivated to include transactions with the highest transaction fees in order to maximize their rewards. This encourages transactions with higher fees can be processed and written into the global ledger faster.



Preventing 51% Attacks: Crime doesn’t Pay

The incentive may help encourage nodes to stay honest. If a greedy attacker is able to assemble more CPU power than all the honest nodes, he would have to choose between using it to defraud people by stealing back his payments, or using it to generate new coins. He ought to



Suppose dishonest party acquires lots of hashing power …

?

1. Unlimited Double Spending?

Vandalism destroys coin values!

2. Collect all the rewards?

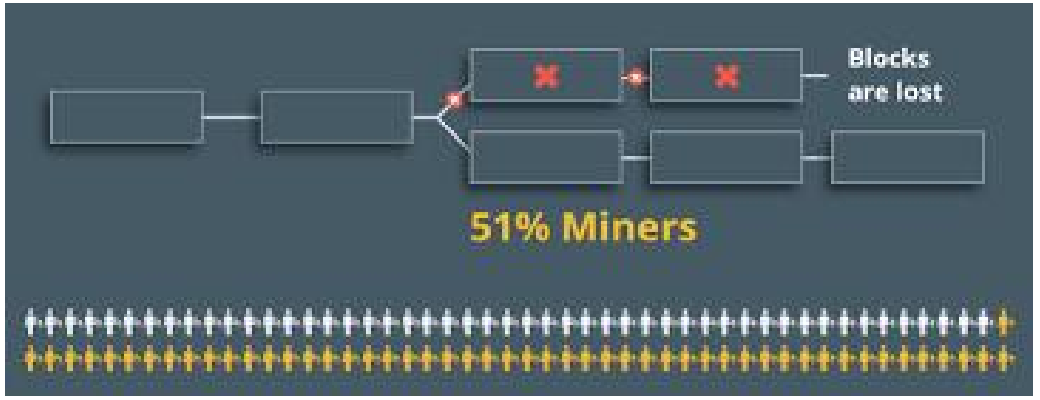
Bitcoin System

51% attack (f > $\frac{n}{2}$)

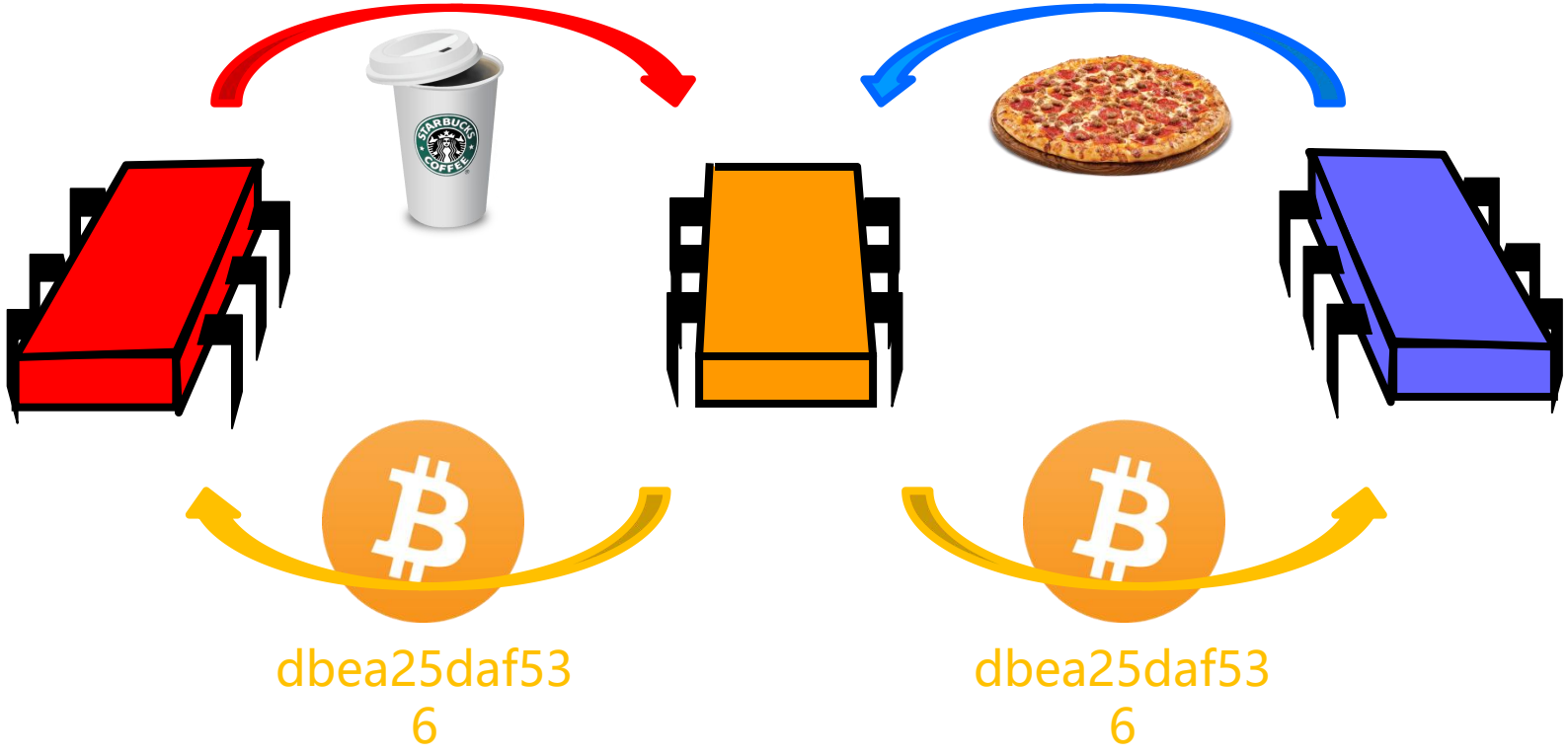
- Miners owned by the same group possess more than 50% of the computational power in the entire system.

- Due to their significant computational power, they can calculate valid block hashes at a faster rate compared to other miners. This allows them to control which transactions are included in a block and the order in which these transactions appear.

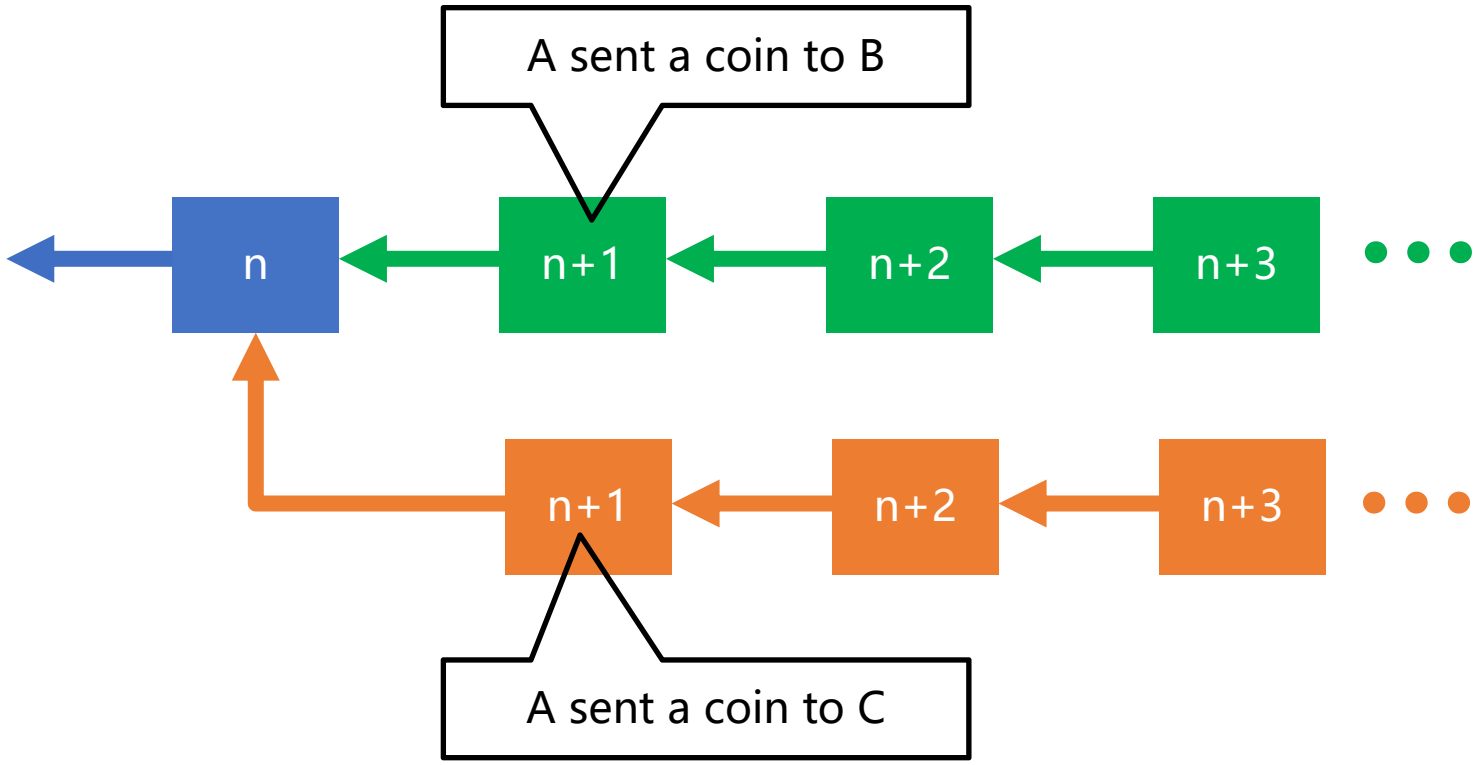
- Furthermore, they can create a new branch of the blockchain, enabling them to potentially rewrite transactions that have already been recorded in the global ledger.



Problem: Double Spending



The Essence of Double Spending



Which chain is the correct one?

Review: Longest Chain Rule

in it. If a majority of CPU power is controlled by honest nodes, the honest chain will grow the fastest and outpace any competing chains. To modify a past block, an attacker would have to

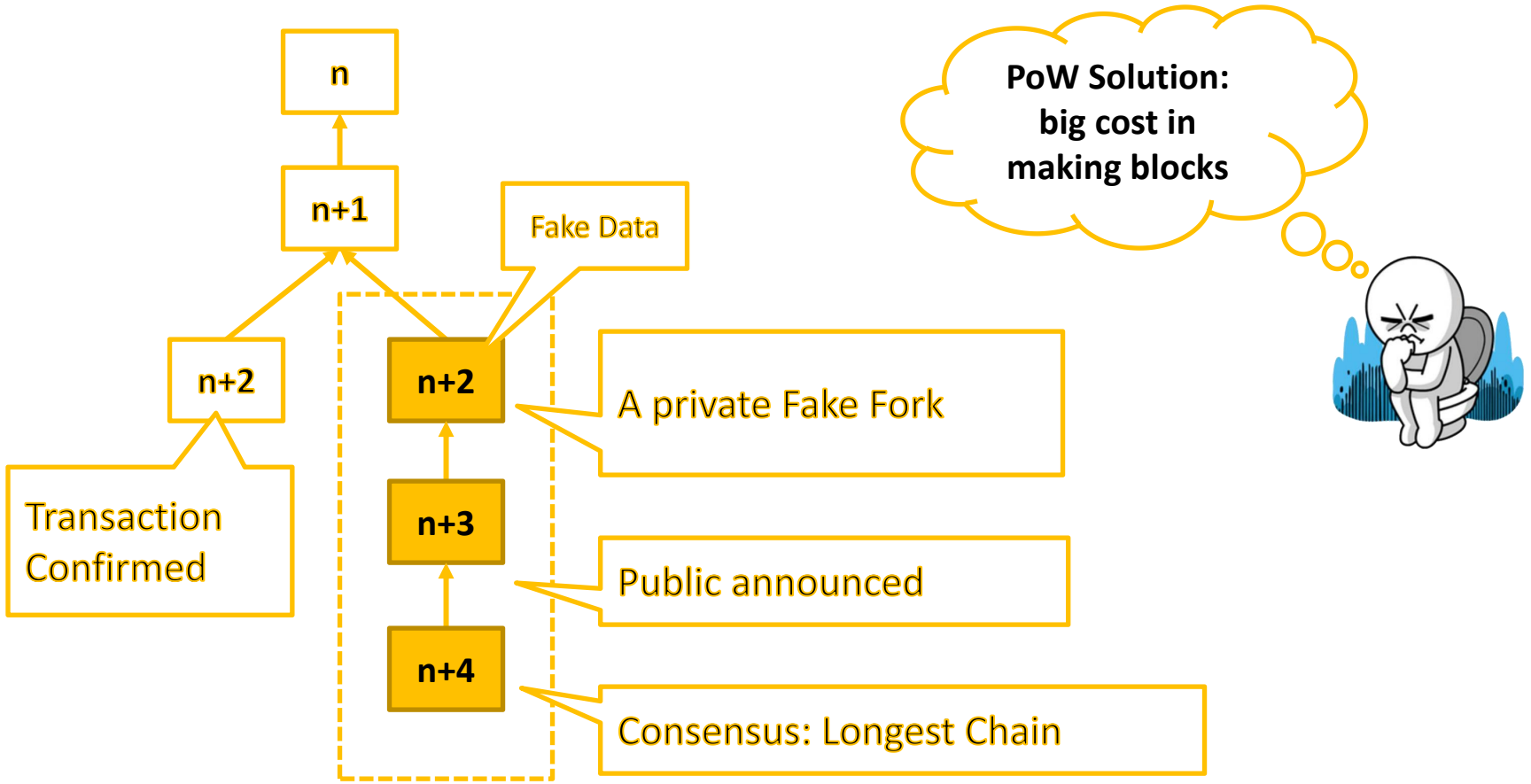
Honest miners build on longest chain …



… longest chain reflects will of honest miners

Dishonest miners would have to out-compute all honest miners

Double Spending Attack

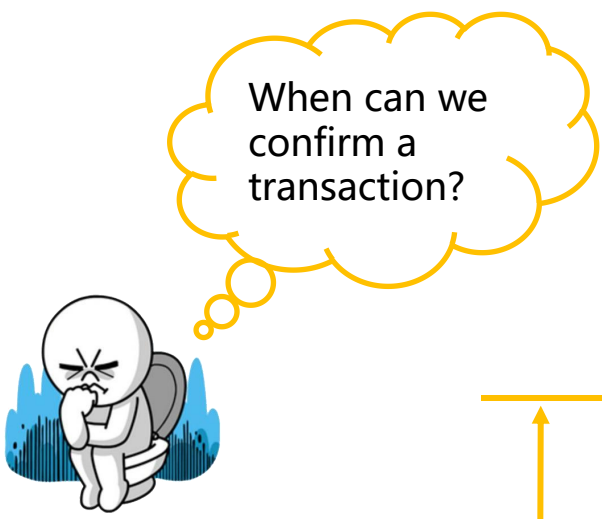


Chain Property

Control x% of miners, control x% of blocks

Not always true but close enough for now

51% Attack



Classic Bitcoin Client: 6 blocks
Assumption:
Adversaries' Hash Rate < 10%

depth k

n

n+1

n+2

n+3

n+4

n+2

n+3

n+4

Honest Blocks

Fake Blocks

One CPU One Vote

Cost: 51% Attack

Require:
Adversary controls < ½ power

How Many Blocks Needed to Confirm?

- An Naïve Calculation

p = probability an honest node finds the next block
q = probability the attacker finds the next block
① Probability the attacker will ever catch up from *z* blocks behind

$$q_z = \begin{cases} 1 & if\ p \leq q \\ \left(\frac{q}{p}\right)^z & if\ p > q \end{cases}$$

② Attacker ‘s potential progress:
a Poisson distribution with expected value $\lambda = z\ \frac{q}{p}$

③ The probability the attackers could catch up

$$\sum_{k=0}^{\infty} \frac{\lambda^k e^{-\lambda}}{k!} \cdot \begin{cases} \mathrm{q/p}^{(z-k)} & if\ k \leq z \\ 1 & if\ k > z \end{cases}$$

④ Rearranging to avoid summing the infinite tail of the distribution...

$$1 - \sum_{k=0}^{z} \frac{\lambda^k e^{-\lambda}}{k!} (1 - \mathrm{q/p}^{(z-k)})$$

Exponentially Reduced

How Many Blocks Needed to Confirm?

- Simulation Results

q=0.1	q=0.3
z=0 P=1.0000000	z=0 P=1.0000000
z=1 P=0.2045873	z=5 P=0.1773523
z=2 P=0.0509779	z=10 P=0.0416605
z=3 P=0.0131722	z=15 P=0.0101008
z=4 P=0.0034552	z=20 P=0.0024804
z=5 P=0.0009137	z=25 P=0.0006132
z=6 P=0.0002428	z=30 P=0.0001522
z=7 P=0.0000647	z=35 P=0.0000379
z=8 P=0.0000173	z=40 P=0.0000095
z=9 P=0.0000046	z=45 P=0.0000024
z=10 P=0.0000012	z=50 P=0.0000006

Solving for P less than 0.1%..

P < 0.001
q=0.10 z=5
q=0.15 z=8
q=0.20 z=11
q=0.25 z=15
q=0.30 z=24
q=0.35 z=41
q=0.40 z=89
q=0.45 z=340

10% Adversary Hash Rate
6 blocks achieves 99.9% finality

Users should set their own threshold according to the value of the trade: nominal or expensive

The Bitcoin Backbone Protocol: Analysis and Applications*

Juan A. Garay
Yahoo Research
garay@yahoo-inc.com

Aggelos Kiayias†‡
University of Edinburgh, IOHK
akiayias@inf.ed.ac.uk

Nikos Leonardos†§
National and Kapodistrian University of Athens.
nikos.leonardos@gmail.com

June 23, 2017

Abstract

Bitcoin is the first and most popular decentralized cryptocurrency to date. In this work, we extract and analyze the core of the Bitcoin protocol, which we term the Bitcoin *backbone*, and prove two of its fundamental properties which we call *common prefix* and *chain quality* in the static setting where the number of players remains fixed. Our proofs hinge on appropriate and novel assumptions on the "hashing power" of the adversary relative to network synchronicity; we show our results to be tight under high synchronization.

Next, we propose and analyze applications that can be built "on top" of the backbone protocol, specifically focusing on Byzantine agreement (BA) and on the notion of a public transaction ledger. Regarding BA, we observe that Nakamoto's suggestion falls short of solving it, and present a simple alternative which works assuming that the adversary's hashing power is bounded by 1/3. The public transaction ledger captures the essence of Bitcoin's operation as a cryptocurrency, in the sense that it guarantees the liveness and persistence of committed transactions. Based on this notion we describe and analyze the Bitcoin system as well as a more elaborate BA protocol, proving them secure assuming high network synchronicity and that the adversary's hashing power is strictly less than 1/2, while the adversarial bound needed for security decreases as the network desynchronizes.

Finally, we show that our analysis of the Bitcoin backbone protocol for synchronous networks extends with relative ease to the recently considered "partially synchronous" model, where there is an upper bound in the delay of messages that is unknown to the honest parties.

Here is a more complete calculation

Lots of Chernoff bounds ...

more precise statements of correctness

more precise bounds on hashing power

Bitcoin Governance



V.S.



Bitcoin Governance: Protocol

- Theory
  - All and only what Satoshi revealed is permitted
  - Fails to explain upgrades and bug fixes

- Actual Governance?
  - De facto governance by "*Core Bitcoin Devs*"
  - Commit access to *bitcoin core Github Repo*
  - Supported by *Bitcoin Foundation*



BITCOIN FOUNDATION

Bitcoin Governance: Miners

To follow or not to follow?

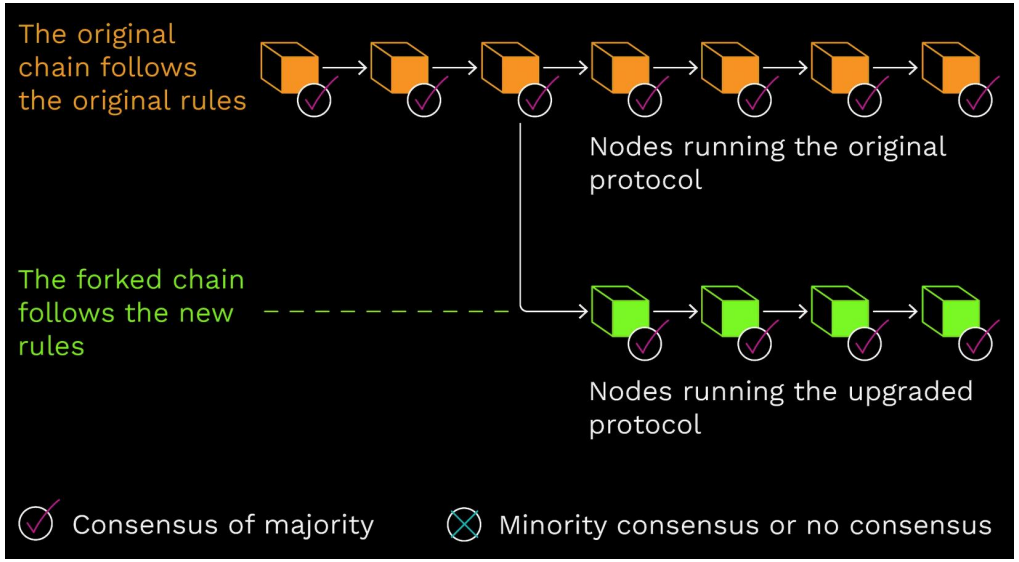


Decentralized Decision Making

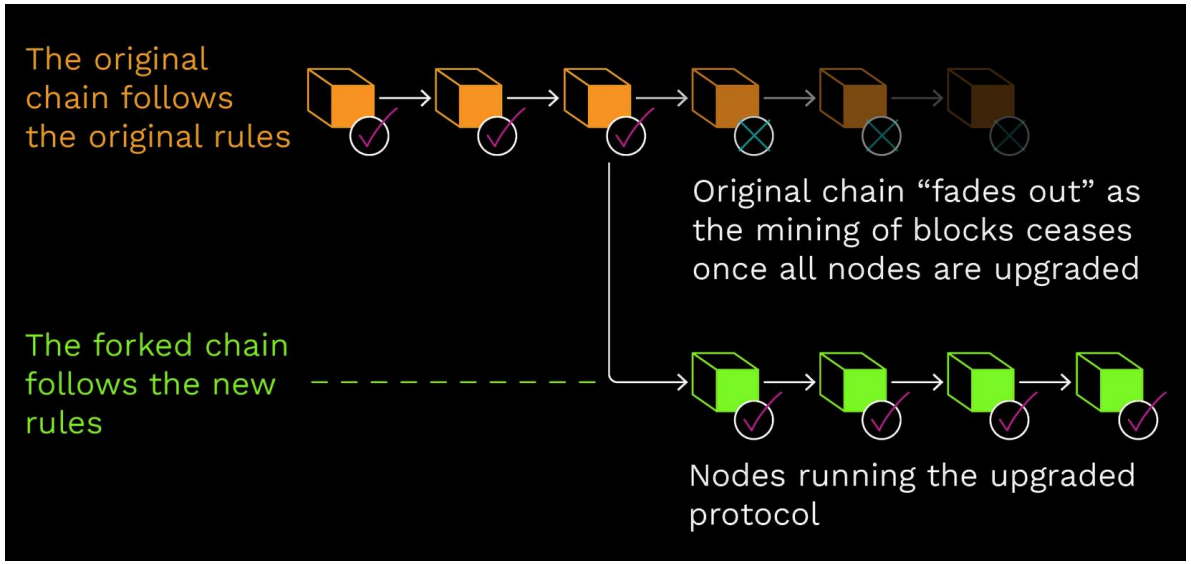
disagreement leads to forks

Soft Fork and Hard Fork

Main Difference: the change in protocol is gradual or not



Hard Fork: Backwards Incompatible



Soft Fork: Backwards Compatible

Forks on Bitcoin

100+ forks off the Bitcoin blockchain today



Case: Controversy on Bitcoin Block Size Limit

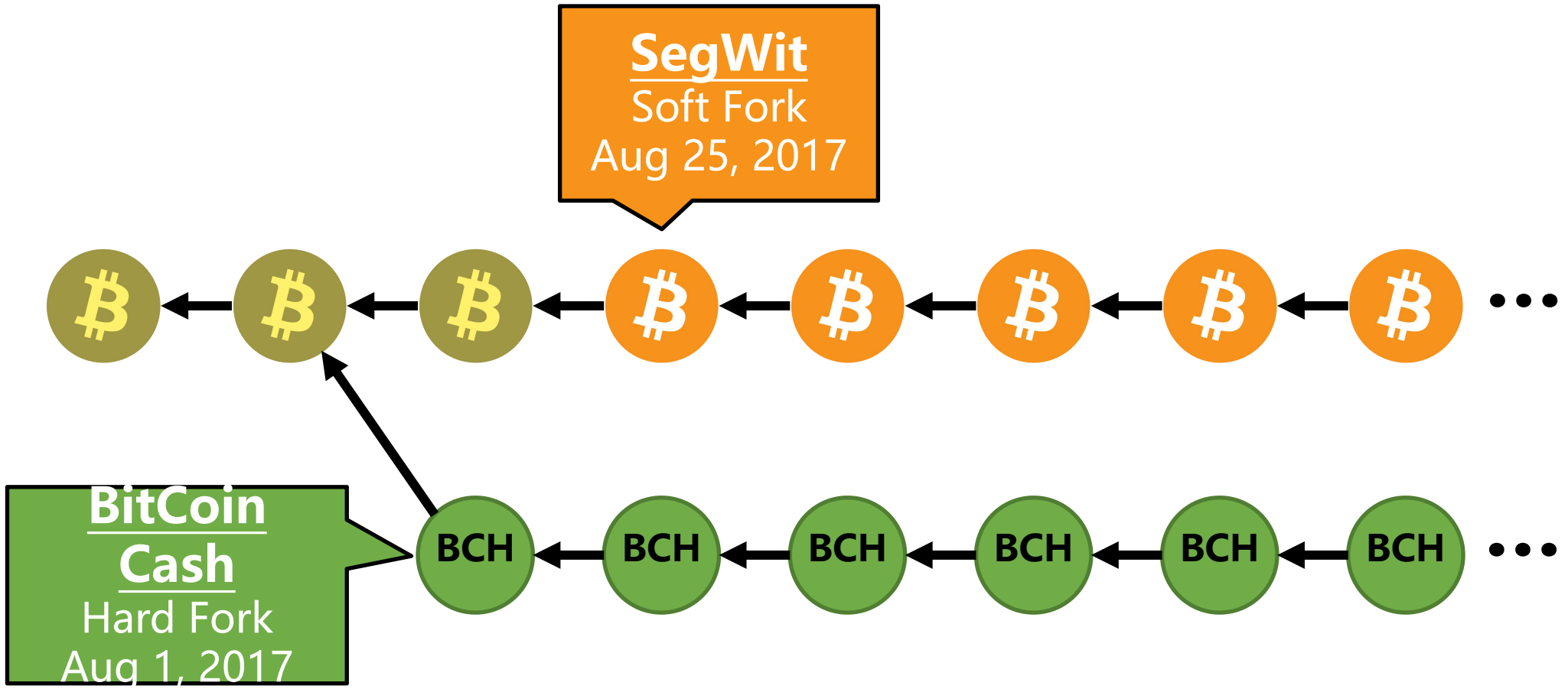
2009: genesis block does not have explicit block size limit
2010: Satoshi introduced 1MB limited

Dilemma

- Small Blocks: low Transactions per Second (TPS) ~4 TPS
- Large Blocks: requires a faster internet connection to sync

Leads to 2 major BitCoin Forks in 2017

Two Major Forks on Block Size Limit



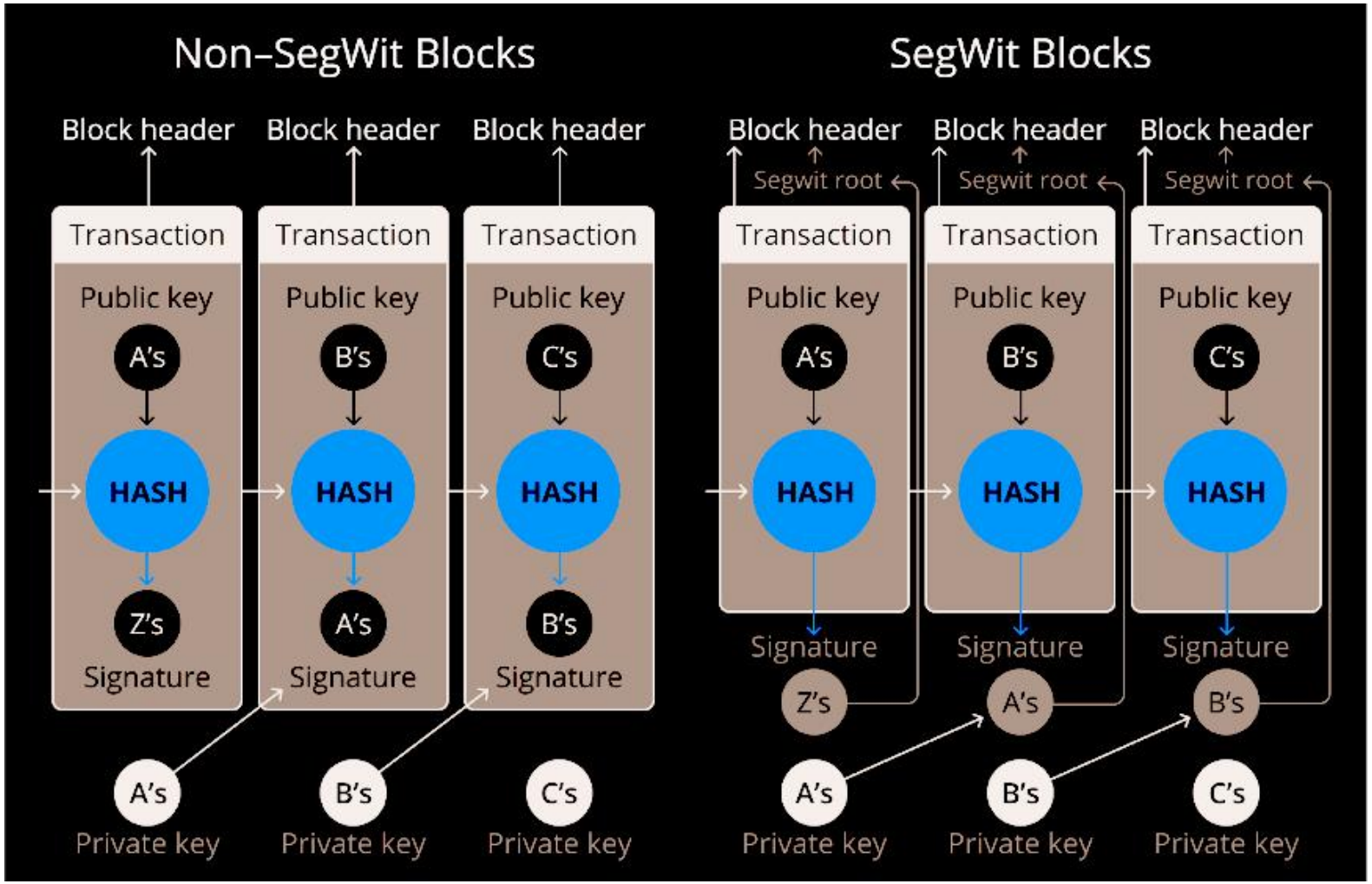
SEGWIT

An upgrade to the Bitcoin protocol that segregates witness data from the base transaction.



SegWit: Segregated Witness

SegWit Solution



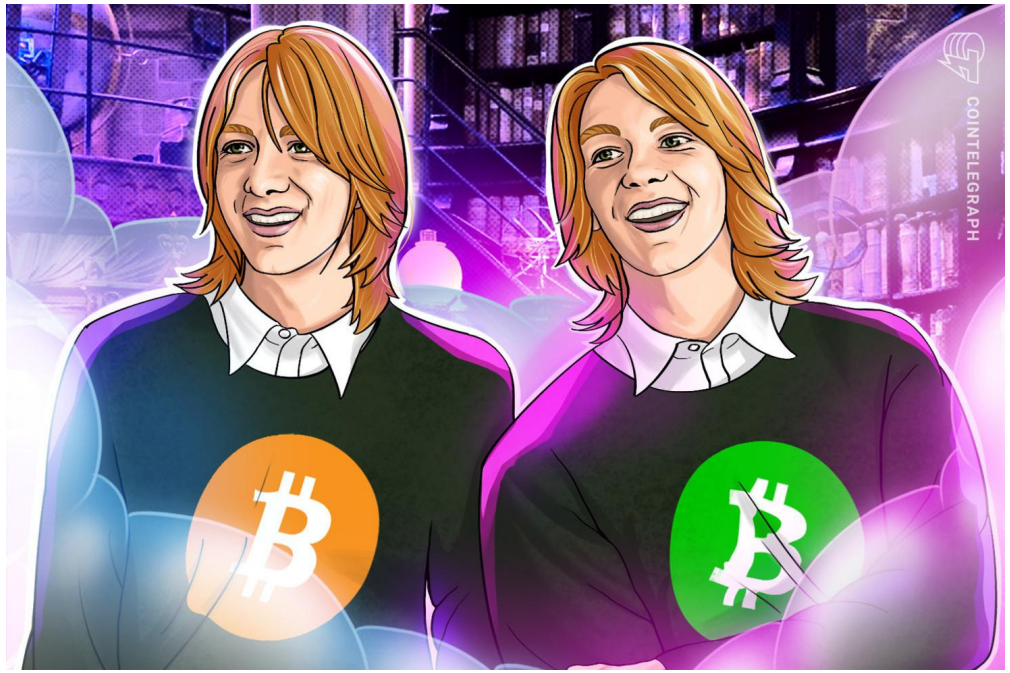
SegWit

Theory Max: 4MB

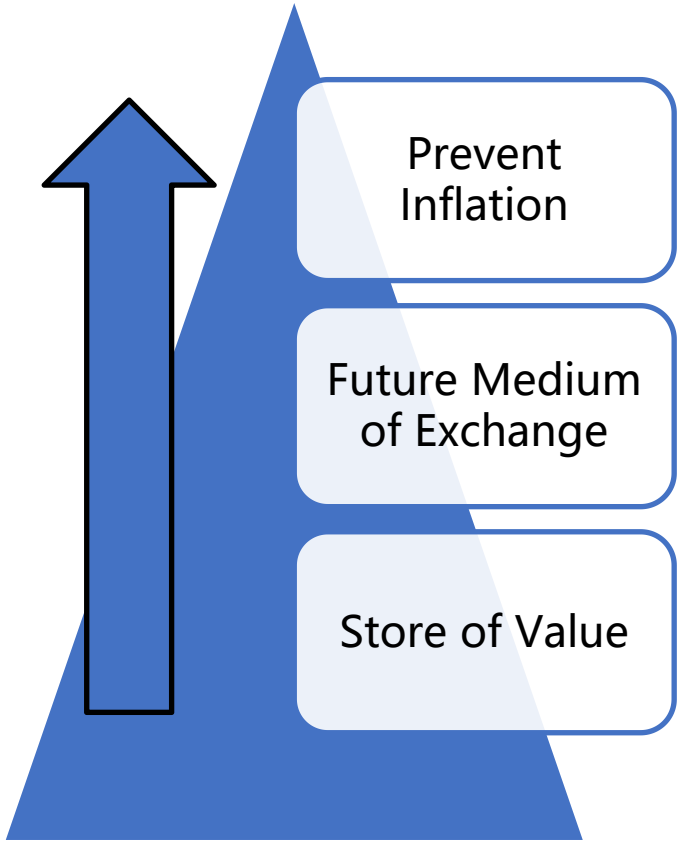
Realistic Max: 2MB

Bitcoin Cash (BCH)

- By whom against the SegWit - Upgrade Bitcoin Improvement Proposal (BIP91)
- 2017/08 - Bitcoin hard forks, increase the block size to 8 MB, leading to the creation of Bitcoin and Bitcoin Cash (BCH).
- 2018/05 - Bitcoin Cash increased block size from 8 MB to 32 MB, and added the potential to run smart contracts



Functionalities of Bitcoin

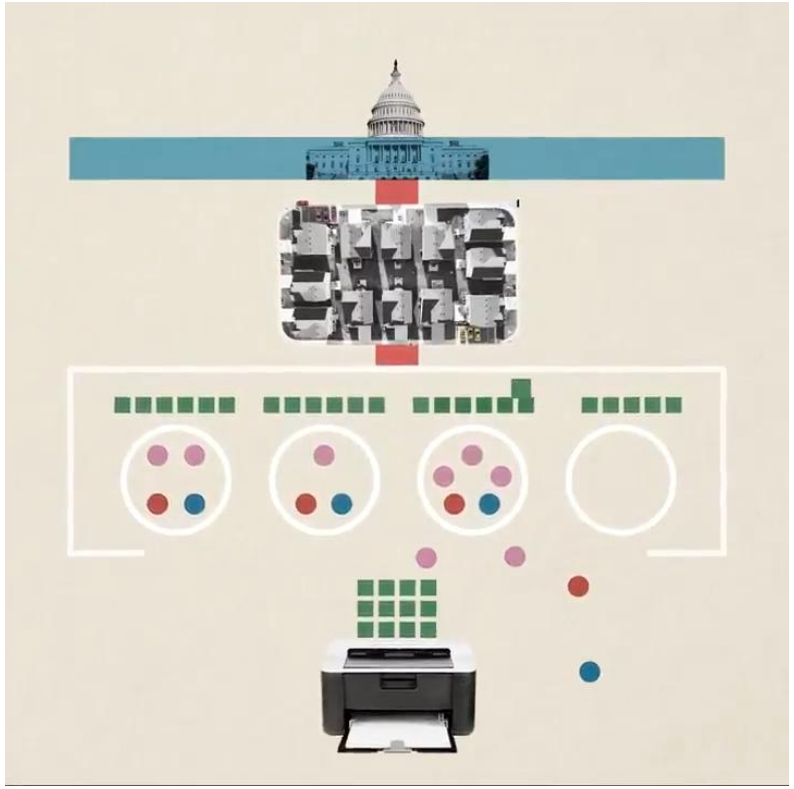


- Deflation on Bitcoin: Limited Supply and Destroy/Lost…
- Good or bad?

- Suffer from low TPS, volatility, fees, etc.
- Low Adoption Rate

- Anti-Double Spending: Make Digital Assets Reliable & Valuable
- Crazy Success So Far

Rational Players Don’ t Spend Deflationary Money



Babysitting Co-op Crisit
by Nobel Laureate Paul Krugman

If an asset is continually gaining value, why spend it?

“Deflationary spiral” (see Japan in 90s)

Workshop:

Using Bitcoin Wallet on Testnet

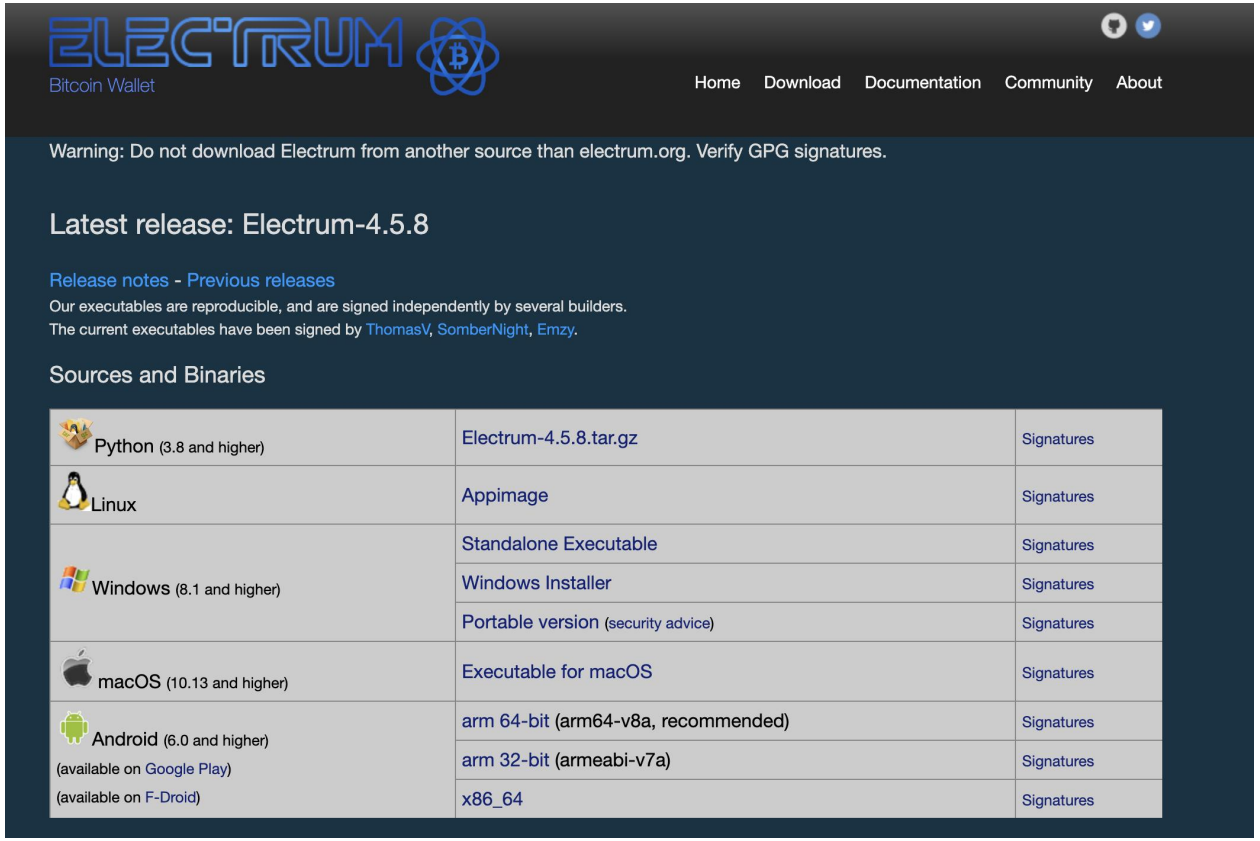
Workshop: Using Bitcoin Wallet on Testnet

1. Download Wallet Application

- We have selected **Electrum** as the example of a Bitcoin wallet application.

  You can download and install the wallet through the official link below:

  https://electrum.org/#download



Workshop: Using Bitcoin Wallet on Testnet

2. Open Wallet Application

- To open the wallet in production (mainnet) mode, simply launch the application as usual.

- If you want to open the wallet in testnet mode, you'll need to follow a few additional steps.



Workshop: Using Bitcoin Wallet on Testnet

2. Open Wallet Application on Testnet

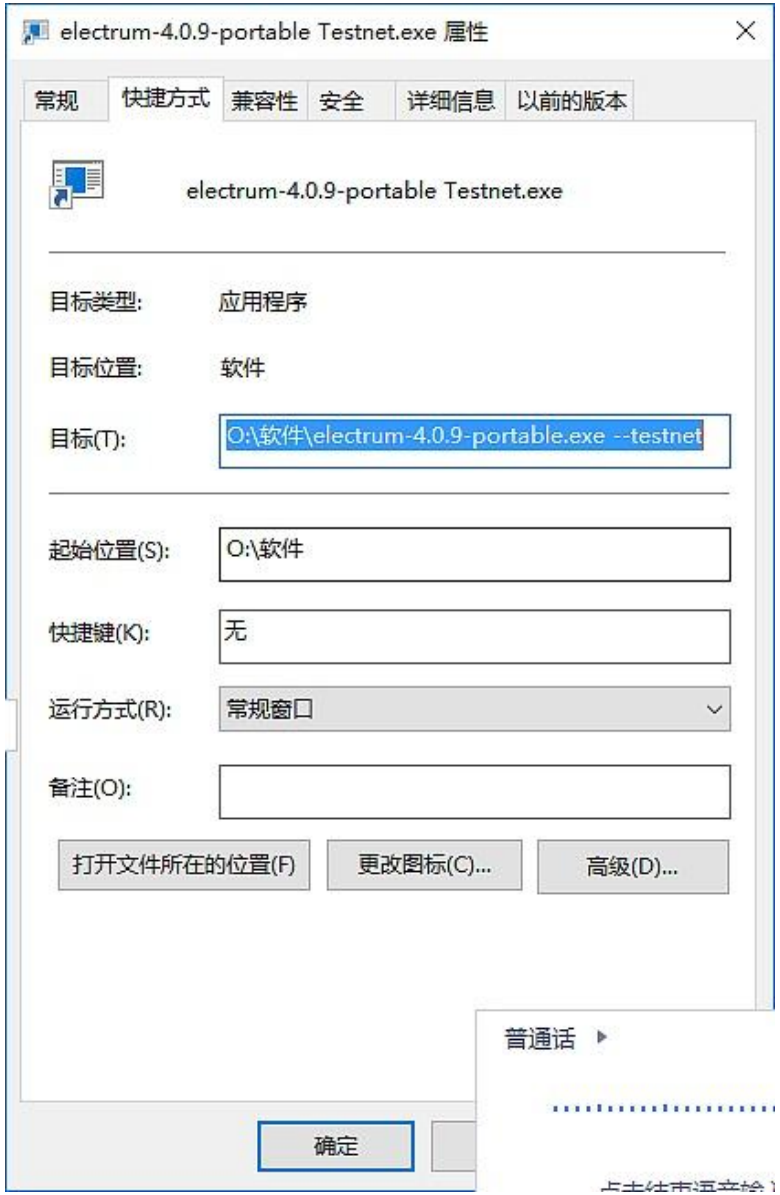
(For Windows OS)

1. Create a desktop shortcut of the application.
2. Open the Properties page of the shortcut.
3. Add an extra parameter after the target path:

   There are 2 testnets in bitcoin system.

   - use --testnet to open the **testnet3** Bitcoin system.
   - use --testnet4 to open the **testnet4** Bitcoin system.

Reference: https://www.jinse.cn/news/blockchain/1019059.html



Workshop: Using Bitcoin Wallet on Testnet

2. Open Wallet Application on Testnet

(For MacOS)

1. Open Terminal.
2. Launch the wallet application using the command provided below:

   There are 2 testnets in bitcoin system.

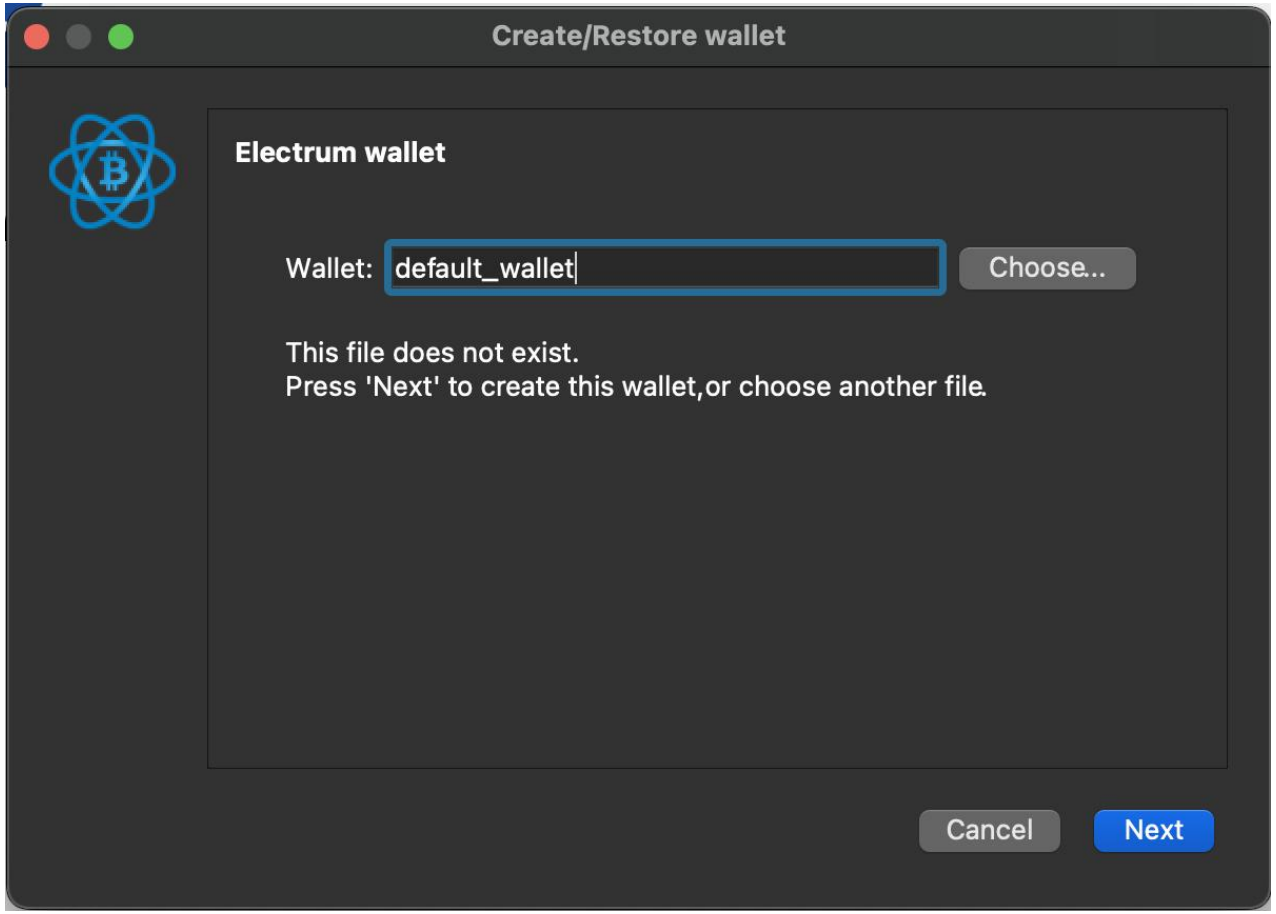
   - use --testnet to open the **testnet3** Bitcoin system.
   - use --testnet4 to open the **testnet4** Bitcoin system.

(location of your Electrum application)

```
[benlao@BendeMacBook-Air ~ % open /Applications/Electrum.app --args --testnet4
[benlao@BendeMacBook-Air ~ %
```

Workshop: Using Bitcoin Wallet on Testnet

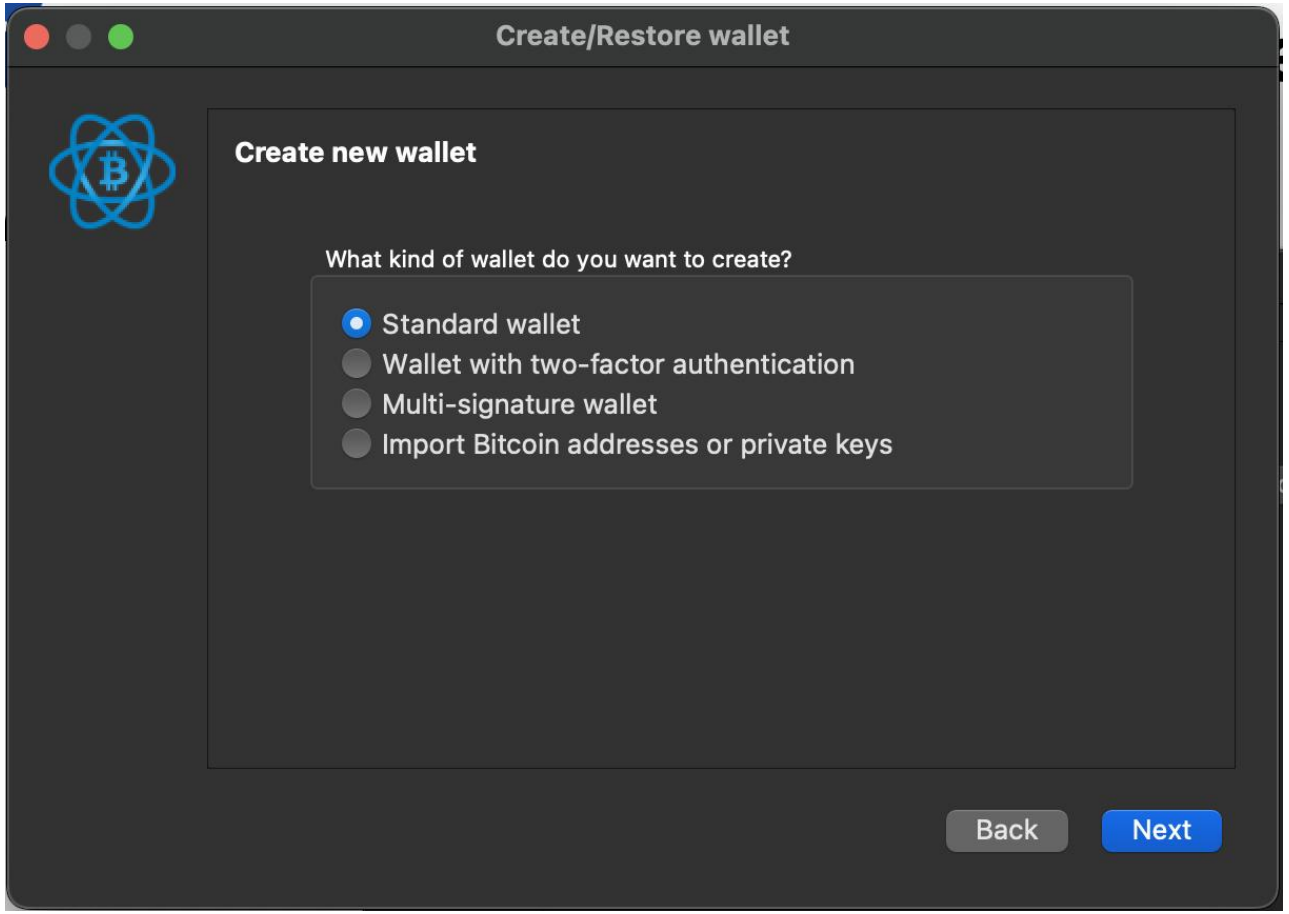
3. Create Wallet



Create a wallet name on your computer.

Workshop: Using Bitcoin Wallet on Testnet

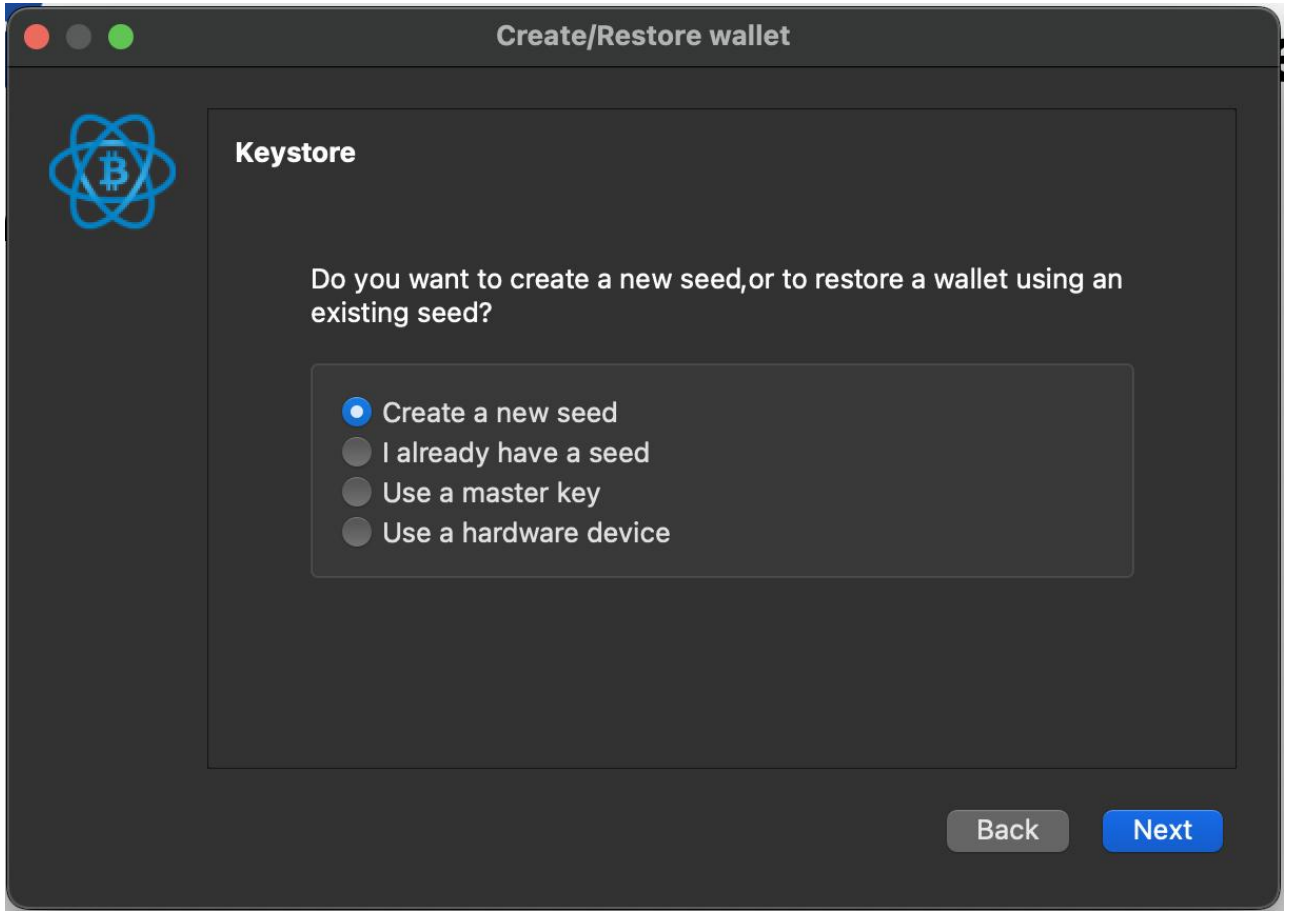
3. Create Wallet



Choose your wallet type.
(The standard wallet is sufficient for the workshop.)

Workshop: Using Bitcoin Wallet on Testnet

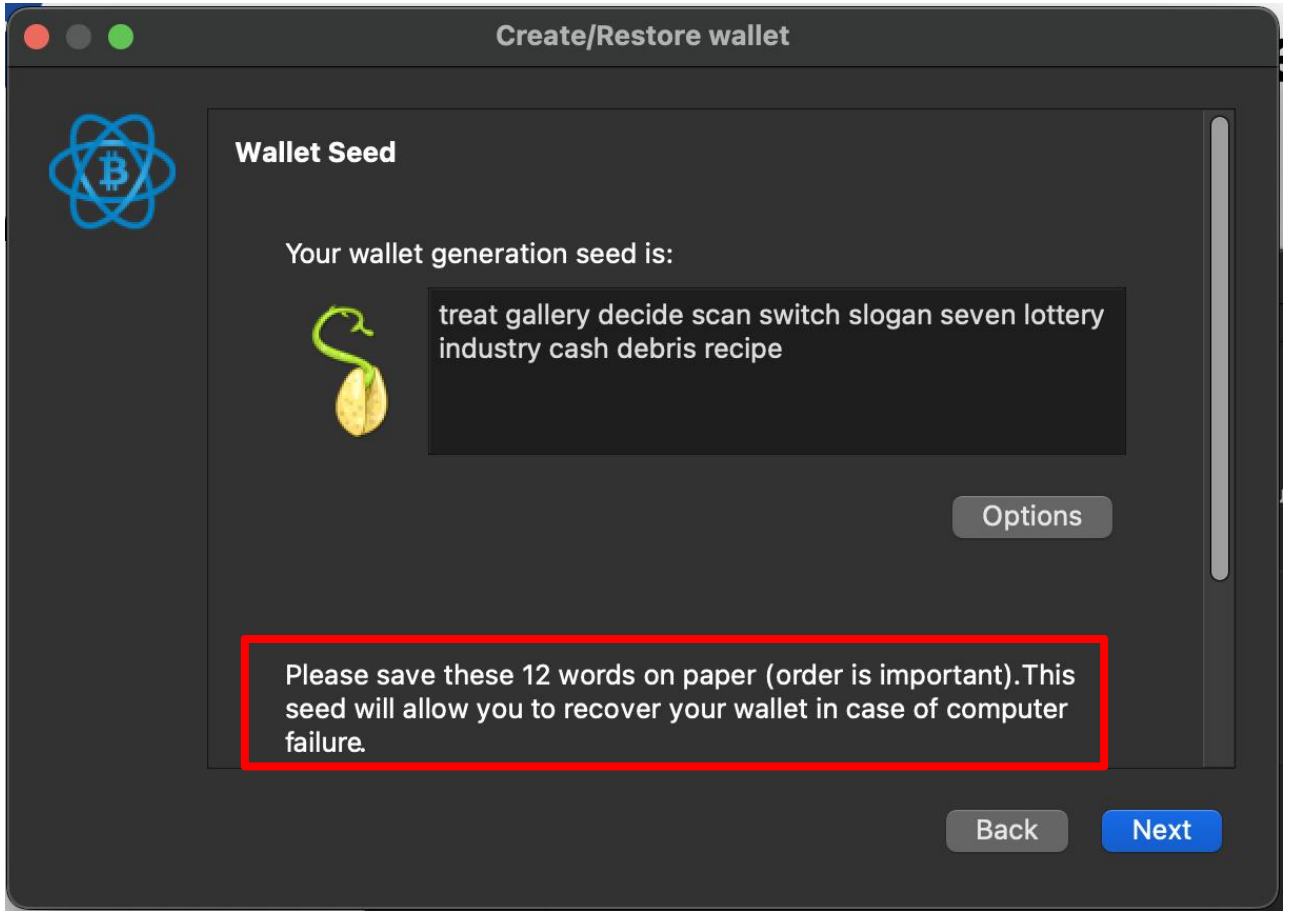
3. Create Wallet



Create a new wallet or restore an existing wallet using a seed phrase.

Workshop: Using Bitcoin Wallet on Testnet

3. Create Wallet

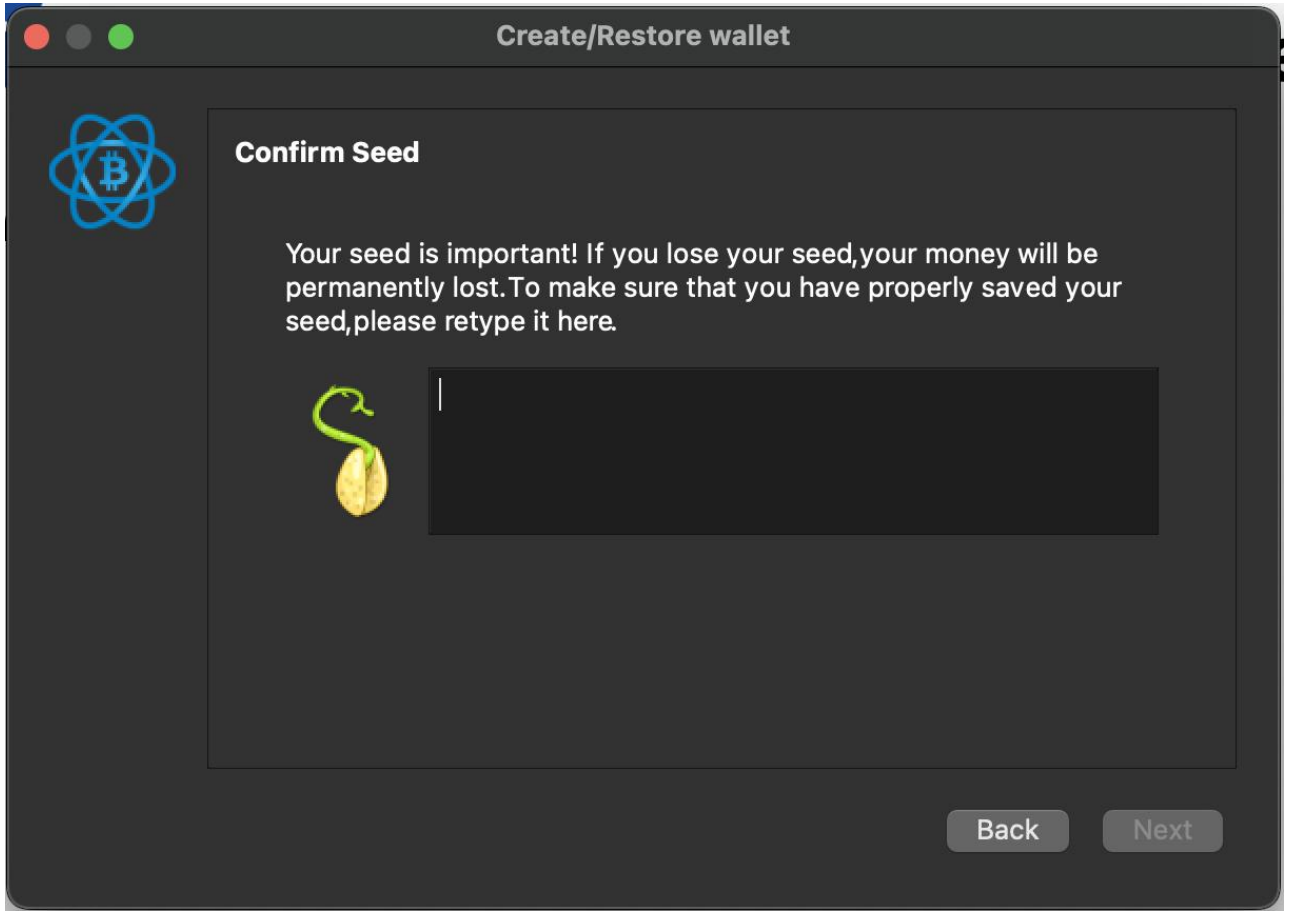


Generate a new seed for your wallet.

Important: Please keep the seed phrase in secret.
Anyone with this seed phrase can access your wallet!

Workshop: Using Bitcoin Wallet on Testnet

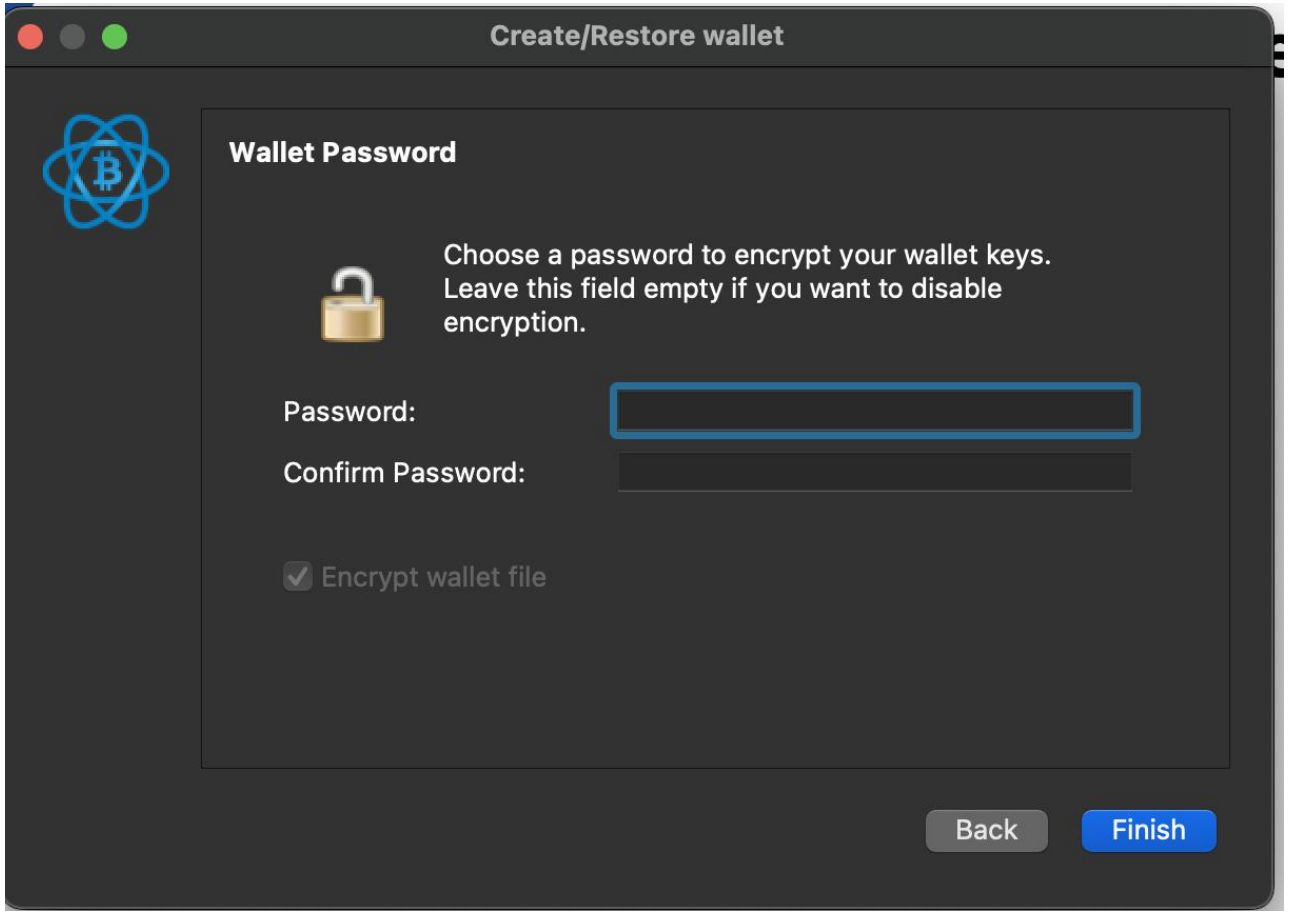
3. Create Wallet



Retype the seed.

Workshop: Using Bitcoin Wallet on Testnet

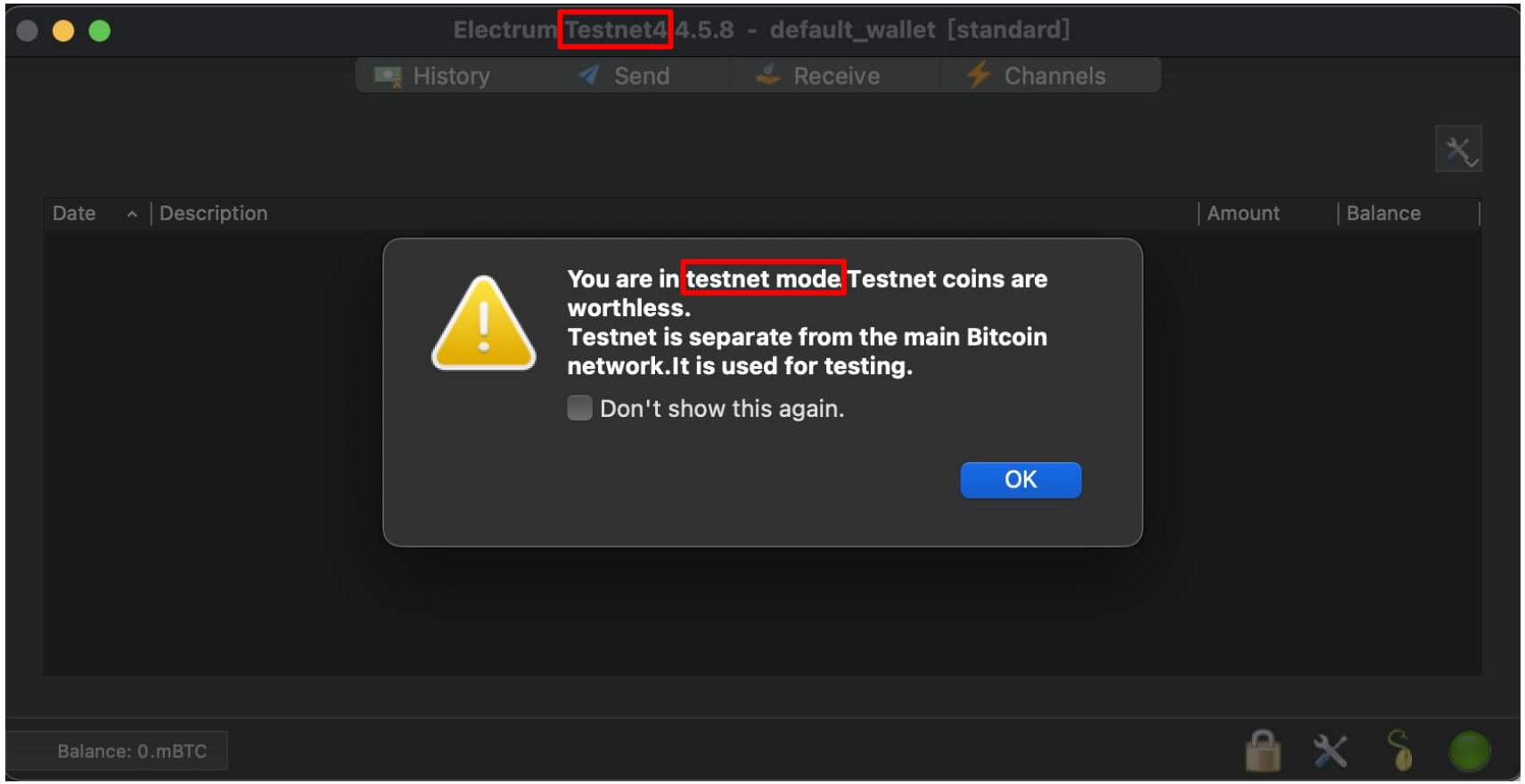
3. Create Wallet



Provide a password for the wallet on this computer.

Workshop: Using Bitcoin Wallet on Testnet

3. Create Wallet



Wallet created successfully.

Note: A testnet alert will appear in the title and an alert box when you login to testnet mode.

Workshop: Using Bitcoin Wallet on Testnet

4. Get testnet bitcoin

- You can obtain testnet Bitcoin from a faucet. One available faucet is: https://coinfaucet.eu/en/

- You can only request testnet Bitcoin once every 12 hours.

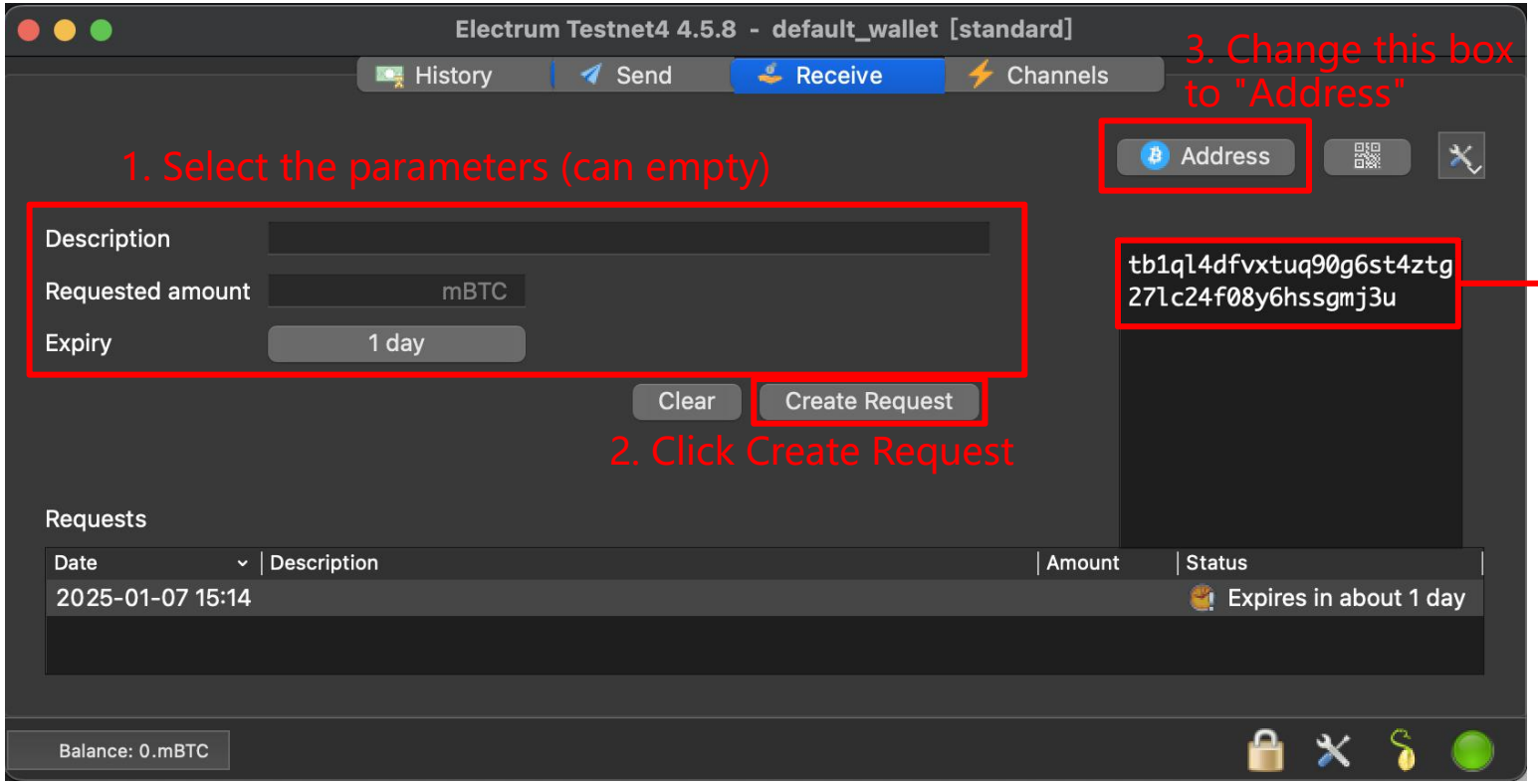
- **Reminder:**
  Testnet coins are a shared and limited resource. After you finish using the testnet coins, please donate the remaining coins to the address provided by the faucet to allow others to use them.

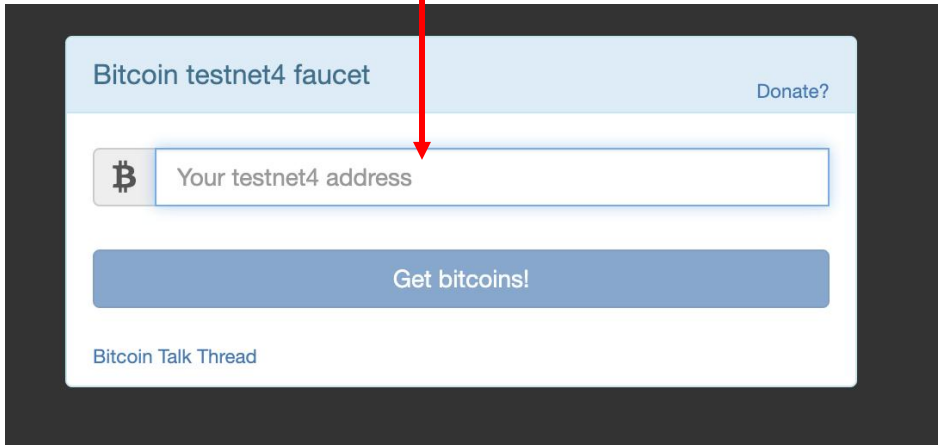


Workshop: Using Bitcoin Wallet on Testnet

4. Get testnet bitcoin

- You need to provide an address to the faucet, which you can obtain from the "Receive" section of your wallet.

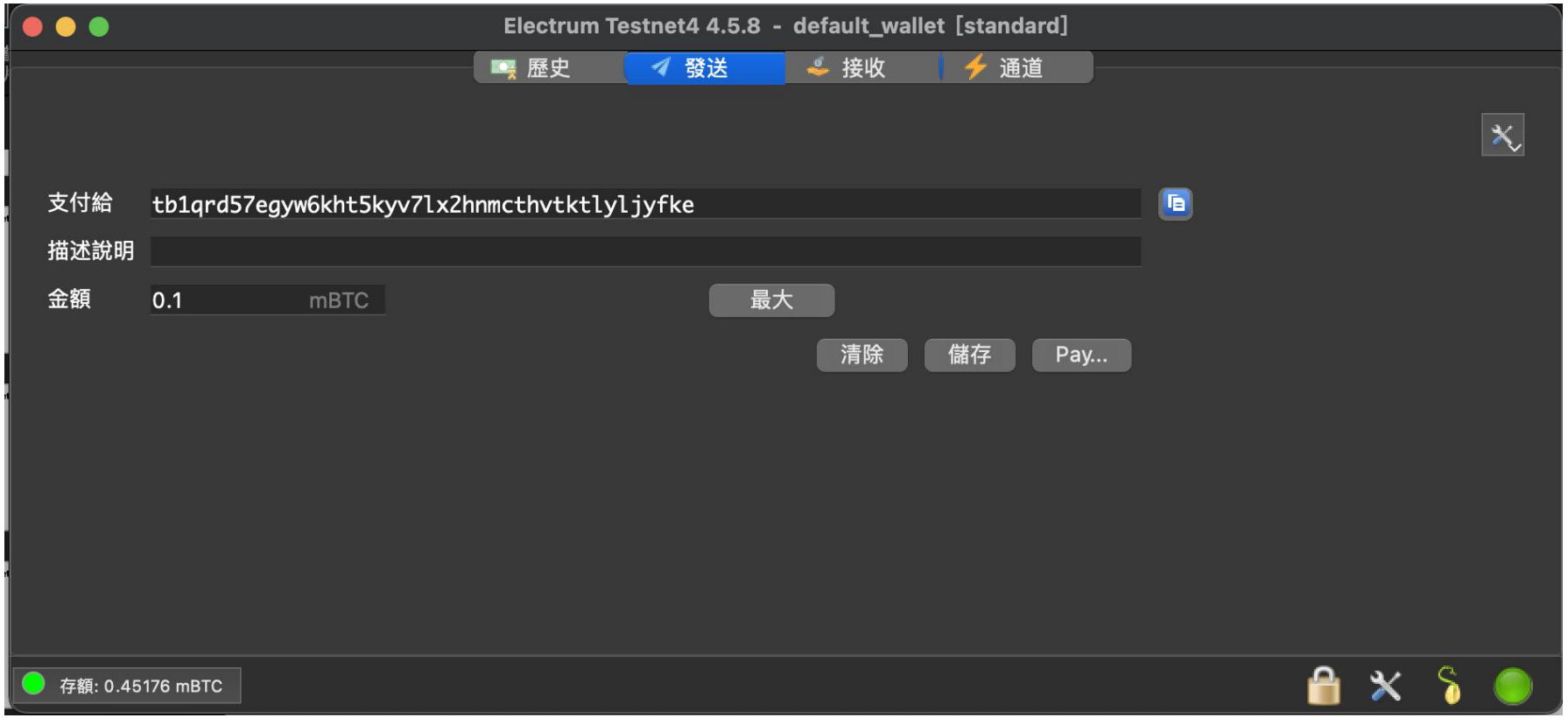




Workshop: Using Bitcoin Wallet on Testnet

5. Send testnet bitcoin

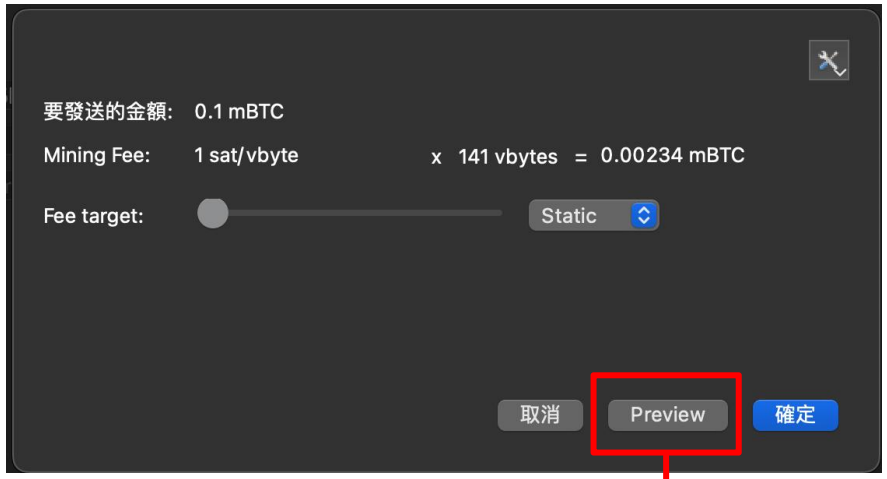
- By providing the target address and the amount, you can initiate a new transaction.

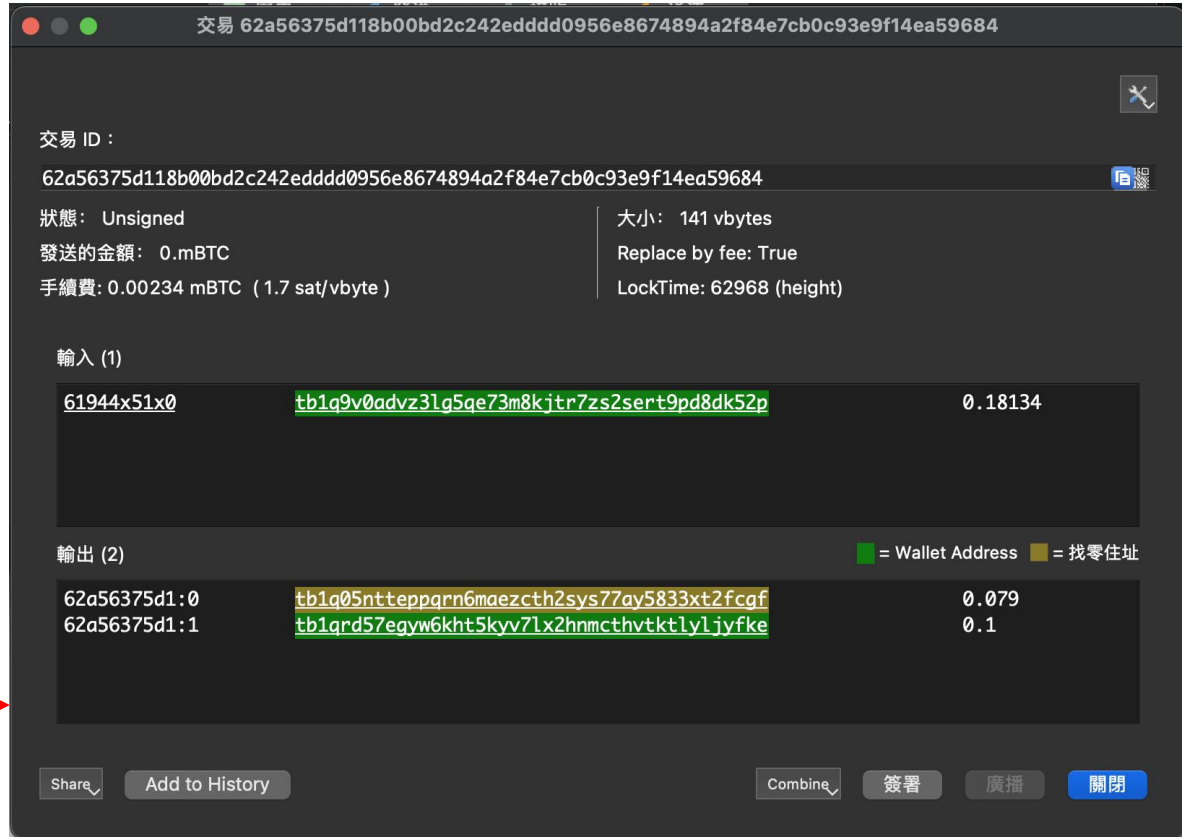


Workshop: Using Bitcoin Wallet on Testnet

5. Send testnet bitcoin

- Before sending the transaction, you can select the transaction fee and preview the transaction information.
- The wallet will automatically choose the appropriate UTXO to match your transaction request.





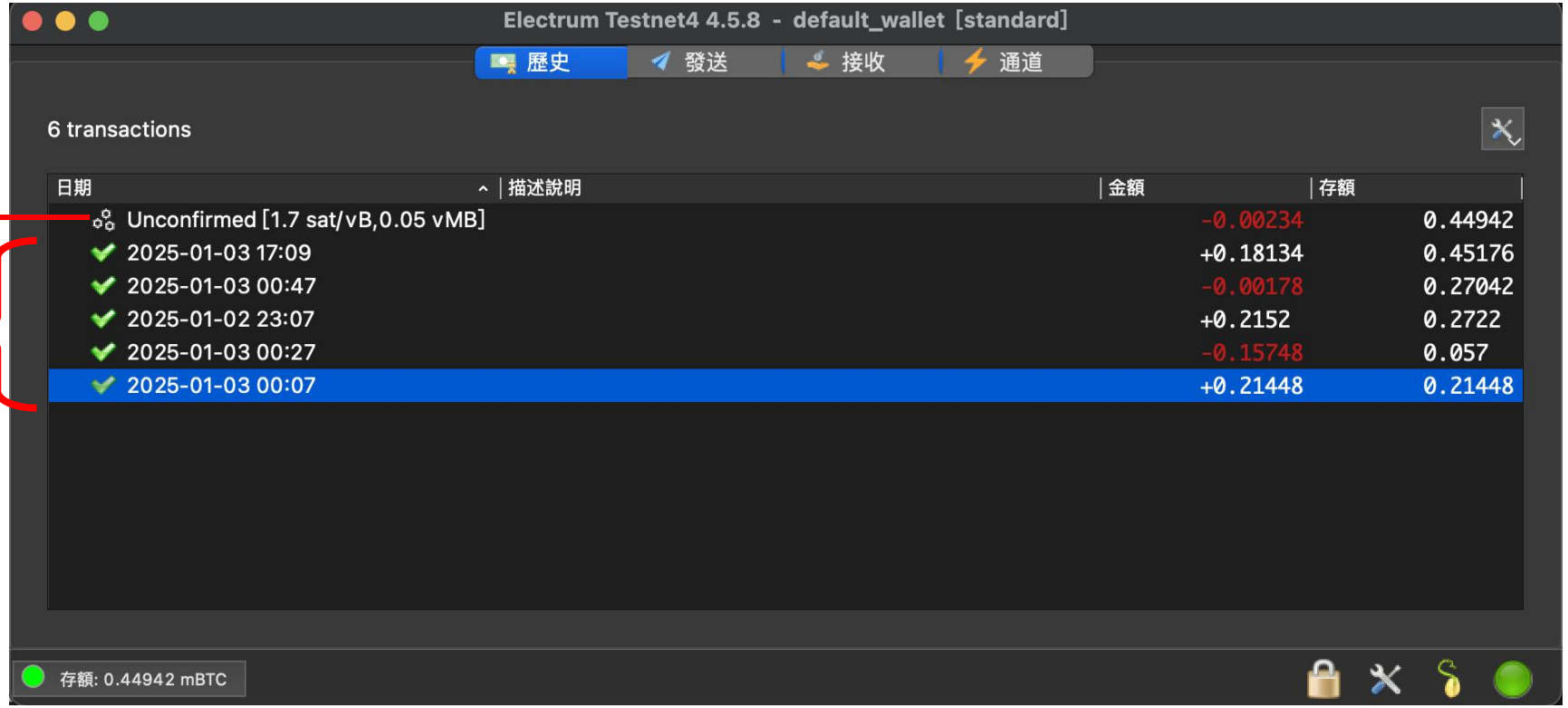
Workshop: Using Bitcoin Wallet on Testnet

6. Check Transaction Records

- You can view all transactions and their status on the history page.

unconfirmed transaction

confirmed transactions



Workshop: Using Bitcoin Wallet on Testnet

6. Check Transaction Records

- You can also check the detailed transaction information by clicking on the transaction in history page.
- This transaction can also be found in the online blockchain browser. One available online browser is: https://mempool.space/testnet4
- You can search for information by providing the transaction ID and UTXO ID on the online browser.

